

Minecraft Forge 1.20.1 Documentation

MinecraftForge

2026-06-22

Contents

MinecraftForge Documentation	7
Contributing to This Documentation	7
Style Guide	7
Getting Started with Forge	7
Prerequisites	7
From Zero to Modding	8
Customizing Your Mod Information	8
Building and Testing Your Mod	9
Mod Files	9
mods.toml	9
Mod Entrypoints	13
Structuring Your Mod	14
Packaging	14
Class Naming Schemes	15
Choose One Method from Many	15
Versioning	15
Examples	15
Registries	17
Methods for Registering	17
Referencing Registered Objects	18
Creating Custom Forge Registries	19
Handling Missing Entries	20
Sides in Minecraft	20
Different Kinds of Sides	20
Performing Side-Specific Operations	21
Common Mistakes	22
Writing One-Sided Mods	22
Events	23
Creating an Event Handler	23
Canceling	24
Results	25
Priority	25
Sub Events	25
Mod Event Bus	25

Mod Lifecycle	26
Registry Events	26
Data Generation	26
Common Setup	26
Sided Setup	26
InterModComms	26
Resources	27
ResourceLocation	27
Internationalization and Localization	27
Language files	28
Usage with Blocks and Items	28
Localization methods	28
Blocks	29
Creating a Block	29
Registering a Block	29
Further Reading	30
Block States	30
Legacy Behavior	30
Introduction of States	30
Proper Usage of Block States	30
Implementing Block States	31
Using <code>BlockState</code> 's	32
BlockEntities	32
Registering	32
Creating a <code>BlockEntity</code>	32
Storing Data within your <code>BlockEntity</code>	32
Ticking <code>BlockEntities</code>	33
Synchronizing the Data to the Client	33
BlockEntityRenderer	34
Creating a BER	34
Registering a BER	35
Items	35
Creating an Item	35
Creative Tabs	35
Registering an Item	36
BlockEntityWithoutLevelRenderer	36
Using <code>BlockEntityWithoutLevelRenderer</code>	36
Networking	37
SimpleImpl	37
Getting Started	37
The Version Checker	37
Registering Packets	37
Handling Packets	38
Sending Packets	39
Entities	39
Spawn Data	39
Dynamic Data	39

Particles	40
Creating a Particle	40
Spawning a Particle	42
Sounds	43
Terminology	43
sounds.json	43
Creating Sound Events	43
Playing Sounds	44
The Capability System	45
Forge-provided Capabilities	45
Using an Existing Capability	45
Exposing a Capability	46
Attaching Capabilities	47
Creating Your Own Capability	47
Persisting LevelChunk and BlockEntity capabilities	47
Synchronizing Data with Clients	48
Persisting across Player Deaths	48
Saved Data	48
Declaration	48
Attaching to a Level	48
Codecs	49
Using Codecs	49
Existing Codecs	50
Creating Codecs	51
Menus	55
MenuType	55
AbstractContainerMenu	56
Opening a Menu	59
Screens	61
Relative Coordinates	61
Gui Graphics	61
Renderable	62
GuiEventListener	62
NarratableEntry	63
The Screen Subtype	63
AbstractContainerScreen	65
Registering an AbstractContainerScreen	67
Root Transforms	67
Element-wise specification	67
Render Types	68
Vanilla Values	68
Custom Values	69
Part Visibility	70
Face Data	70
Elements Model	71
Generated Item Model	71
Parameters	71

Custom Model Loaders	72
WaveFront OBJ Models	72
BakedModel	73
Transform	74
ItemDisplayContext	74
The Vanilla Way	74
The Forge Way	75
ItemOverrides	75
BakedOverride	75
Resource Packs	76
Models	76
Model Files	76
Coloring Textures	76
Item Properties	77
Adding Properties to Items	77
Using Overrides	78
Datapacks	78
Recipes	78
Data-Driven Recipes	79
Recipe Manager	79
Additional Features	79
Custom Recipes	80
Recipe	80
RecipeType	81
RecipeSerializer	81
Building the JSON	81
Non-Item Logic	82
Data Generation	82
Ingredients	82
Custom Ingredients	82
Creating Custom Ingredients	84
Non-Datapack Recipes	85
Brewing Recipes	85
Anvil Recipes	85
Loom Recipes	86
Loot Tables	86
Data-Driven Tables	86
Using a Loot Table	86
Additional Features	86
Global Loot Modifiers	88
Registering a Global Loot Modifier	88
The <code>global_loot_modifiers.json</code>	88
The Serialized JSON	88
IGlobalLootModifier	89
The Loot Modifier Codec	90

Tags	90
Declaring Your Own Groupings	90
Using Tags In Code	91
Conventions	92
Migration from OreDictionary	92
Using Tags in Recipes and Advancements	92
Advancements	92
Advancement Criteria	92
Advancement Rewards	95
Conditionally-Loaded Data	95
Implementations	95
Conditions	95
Creating Custom Conditions	97
Data Generators	97
Existing Files	98
Generator Modes	98
Data Providers	98
Model Generation	99
Model Files	99
Model Builders	99
Model Providers	100
Block State Provider	101
Model Loader Builders	104
Custom Model Loader Builders	105
Custom Model Providers	105
Custom Model Consumers	106
Language Generation	106
LanguageProvider	106
Sound Definition Generation	107
Adding a Sound	107
Recipe Generation	108
RecipeBuilder	108
Non-RecipeBuilder Builders	109
Conditional Recipes	110
Custom Recipe Serializers	111
Loot Table Generation	111
LootTableSubProvider	111
Loot Table Builders	113
Tag Generation	114
TagsProvider	114
Custom Tag Providers	115
Advancement Generation	115
ForgeAdvancementProvider\$AdvancementGenerator	116
Advancement\$Builder	116
Global Loot Modifier Generation	117
Datapack Registry Object Generation	117

RegistrySetBuilder	117
Registering with BootstrapContext	118
Configuration	119
Creating a Configuration	119
Registering a Configuration	121
Configuration Events	121
Key Mappings	121
Registering a KeyMapping	121
Creating a KeyMapping	121
Checking a KeyMapping	123
Game Tests	124
Creating a Game Test	124
Registering a Game Test	126
Structure Templates	126
Running Game Tests	127
Buildscript Configurations	127
Forge Update Checker	128
Getting Started	128
Update JSON format	128
Retrieving Update Check Results	129
Debug Profiler	129
Using the Debug Profiler	129
Reading a Profiling result	129
Profiling your own code	130
Access Transformers	130
Adding ATs	130
Comments	130
Access Modifiers	131
Targets and Directives	131
Examples	132
Getting Started	132
Forking and Cloning the Repository	132
Checking out the Correct Branch	133
Setting Up the Environment	133
Making Changes and Pull Requests	134
Pull Request Guidelines	134
What Exactly is Forge?	134
Patches	135
Forge Code	135
Explain Yourself	135
Show that it Works	135
Breaking Changes in Forge	135
Be Patient, Civil, and Empathetic	136
Review	136
Documentation for Legacy Versions	136
Porting to Minecraft 1.20	137

MinecraftForge Documentation

This is the official documentation for MinecraftForge, the Minecraft modding API.

This documentation is *only* for Forge, **this is not a Java tutorial**.

If you would like to contribute to the docs, read Contributing to the Docs.

Contributing to This Documentation

You can make a contribution via a PR on GitHub.

This documentation is meant to be explanatory. Please explain how to do things, and break it down into reasonable chunks. We have a wiki elsewhere that can capture more comprehensive code examples.

Our audience is anyone who wants to understand how to build a mod using Forge.

Please don't try to turn this documentation into a tutorial on Java Development - it is intended for people who understand how a Java class works, and other fundamental structures of Java.

Style Guide

!!! important Please use **two spaces** to indent, not tabs.

Titles should be capitalized in the standard titling format. For example,

- Guide For Contributing to This Documentation
- Building and Testing Your Mod

Essentially, capitalize everything but unimportant words.

Spelling, grammar, and syntax should follow those of American English. Also, prefer using separate words over contractions (e.g. “are not” instead of “aren’t”).

Please use equals and dash underlines, instead of # and ##. For h3 and lower, ### etc. is fine. The source of this file contains an example for equals and dash underlining. Equals underlines create h1 text, and dash underlines create h2 text.

When referencing fields and methods outside of code block snippets, they should use a # separator (e.g. `ClassName#methodName`). Inner classes should use a \$ separator (e.g. `ClassName$InnerClassName`).

JSON code block snippets should use `js` syntax highlighting.

All links should have their location specified at the bottom of the page. Any internal links should reference the page via their relative path.

Admonitions (represented by !!! <type>) must be formatted as documented; otherwise they may end up rendering incorrectly.

Getting Started with Forge

If you have never made a Forge mod before, this section will provide the minimum amount of information needed to setup a Forge development environment. The rest of the documentation is about where to go from here.

Prerequisites

- An installation of the Java 17 Development Kit (JDK) and 64-bit Java Virtual Machine (JVM). Forge recommends and officially supports Eclipse Temurin.
!!! warning Make sure you are using a 64-bit JVM. One way of checking is to run `java -version` in a terminal. Using a 32-bit JVM will cause some problems when using ForgeGradle.
- Familiarity with an Integrated Development Environment (IDE).

- It is recommended to use an IDE with Gradle integration.

From Zero to Modding

1. Download the Mod Developer Kit (MDK) from the Forge file site by clicking ‘Mdk’ followed by the ‘Skip’ button in the top right after waiting for a period of time. It is recommended to download the latest version of Forge whenever possible.
2. Extract the downloaded MDK into an empty directory. This will be your mod’s directory, which should now contain some gradle files and a `src` subdirectory containing the example mod.

!!! note A number of files can be reused across different mods. These files are:

- * the ``gradle`` subdirectory
- * ``build.gradle``
- * ``gradlew``
- * ``gradlew.bat``
- * ``settings.gradle``

The ``src`` subdirectory does not need to be copied across workspaces; however, you may need to refresh

3. Open your selected IDE:
 - Forge only explicitly supports development on Eclipse and IntelliJ IDEA, but there are additional run configurations for Visual Studio Code. Regardless, any environment, from Apache NetBeans to Vim / Emacs, can be used.
 - Eclipse and IntelliJ IDEA’s Gradle integration, both installed and enabled by default, will handle the rest of the initial workspace setup on import or open. This includes downloading the necessary packages from Mojang, MinecraftForge, etc. The ‘Gradle for Java’ plugin is needed for Visual Studio Code to do the same.
 - Gradle will need to be invoked to re-evaluate the project for almost all changes to its associated files (e.g., `build.gradle`, `settings.gradle`, etc.). Some IDEs come with ‘Refresh’ buttons to do this; however, it can be done through the terminal via `gradlew`.
4. Generate run configurations for your selected IDE:
 - **Eclipse:** Run the `genEclipseRuns` task.
 - **IntelliJ IDEA:** Run the `genIntelliJRuns` task. If a “module not specified” error occurs, set the `ideaModule` property to your ‘main’ module (typically `${project.name}.main`).
 - **Visual Studio Code:** Run the `genVSCodeRuns` task.
 - **Other IDEs:** You can run the configurations directly using `gradle run*` (e.g., `runClient`, `runServer`, `runData`, `runGameTestServer`). These can also be used with the supported IDEs.

Customizing Your Mod Information

Edit the `build.gradle` file to customize how your mod is built (e.g., file name, artifact version, etc.).

!!! important Do **not** edit the `settings.gradle` unless you know what you are doing. The file specifies the repository that ForgeGradle is uploaded to.

Recommended `build.gradle` Customizations

Mod Id Replacement Replace all occurrences of `examplemod`, including `mods.toml` and the main mod file with the mod id of your mod. This also includes changing the name of the file you build by setting `base.archivesName` (this is typically set to your mod id).

```
// In some build.gradle
base.archivesName = 'mymod'
```

Group Id The `group` property should be set to your top-level package, which should either be a domain you own or your email address:

Type	Value	Top-Level Package
Domain	example.com	com.example
Subdomain	example.github.io	io.github.example
Email	example@gmail.com	com.gmail.example

```
// In some build.gradle
group = 'com.example'
```

The packages within your java source (`src/main/java`) should also now conform to this structure, with an inner package representing the mod id:

```
com
- example (top-level package specified in group property)
  - mymod (the mod id)
    - MyMod.java (renamed ExampleMod.java)
```

Version Set the `version` property to the current version of your mod. We recommend using a variation of Maven versioning.

```
// In some build.gradle
version = '1.20-1.0.0.0'
```

Additional Configurations

Additional configurations can be found on the ForgeGradle docs.

Building and Testing Your Mod

1. To build your mod, run `gradlew build`. This will output a file in `build/libs` with the name `[archivesBaseName]-[version].jar`, by default. This file can be placed in the `mods` folder of a Forge-enabled Minecraft setup or distributed.
2. To run your mod in a test environment, you can either use the generated run configurations or use the associated tasks (e.g. `gradlew runClient`). This will launch Minecraft from the run directory (default 'run') along with any source sets specified. The default MDK includes the `main` source set, so any code written in `src/main/java` will be applied.
3. If you are running a dedicated server, whether through the run configuration or `gradlew runServer`, the server will initially shut down immediately. You will need to accept the Minecraft EULA by editing the `eula.txt` file in the run directory. Once accepted, the server will load, which can then be accessed via a direct connect to `localhost`.

!!! note You should always test your mod in a dedicated server environment. This includes client-only mods as they should not do anything when loaded on the server.

Mod Files

The mod files are responsible for determining what mods are packaged into your JAR, what information to display within the 'Mods' menu, and how your mod should be loaded in the game.

mods.toml

The `mods.toml` file defines the metadata of your mod(s). It also contains additional information that is displayed within the 'Mods' menu and how your mod(s) should be loaded into the game.

The file uses the Tom's Obvious Minimal Language, or TOML, format. The file must be stored under the `META-INF` folder in the resource directory of the source set you are using (`src/main/resources/META-INF/mods.toml` for the `main` source set). A `mods.toml` file may look something like this:

```

modLoader="javafml"
loaderVersion="[46,)"

license="All Rights Reserved"
issueTrackerURL="https://github.com/MinecraftForge/MinecraftForge/issues"
showAsResourcePack=false

[[mods]]
  modId="examplemod"
  version="1.0.0.0"
  displayName="Example Mod"
  updateJSONURL="https://files.minecraftforge.net/net/minecraftforge/forge/promotions_slim.json"
  displayURL="https://minecraftforge.net"
  logoFile="logo.png"
  credits="I'd like to thank my mother and father."
  authors="Author"
  description=''
    Lets you craft dirt into diamonds. This is a traditional mod that has existed for eons. It is ancient. The holy
    Notch created it. Jeb rainbowfied it. Dinnerbone made it upside down. Etc.
  ''
  displayTest="MATCH_VERSION"

[[dependencies.examplemod]]
  modId="forge"
  mandatory=true
  versionRange="[46,)"
  ordering="NONE"
  side="BOTH"

[[dependencies.examplemod]]
  modId="minecraft"
  mandatory=true
  versionRange="[1.20]"
  ordering="NONE"
  side="BOTH"

```

`mods.toml` is broken into three parts: the non-mod-specific properties, which are linked to the mod file; the mod properties, with a section for each mod; and the dependency configurations, with a section for each mod's or mods' dependencies. Each of the properties associated with the `mods.toml` file will be explained below, where **required** means that a value must be specified or an exception will be thrown.

Non-Mod-Specific Properties

Non-mod-specific properties are properties associated with the JAR itself, indicating how to load the mod(s) and any additional global metadata.

Property	Type	Default	Description	Example
<code>modLoader</code>	string	mandatory	The language loader used by the mod(s). Can be used to support alternative language structures, such as Kotlin objects for the main file, or different methods of determining the entrypoint, such as an interface or method. Forge provides the Java loader <code>"javafml"</code> and low/no code loader <code>"lowcodefml"</code> .	<code>"javafml"</code>
<code>loaderVersion</code>	string	mandatory	The acceptable version range of the language loader, expressed as a Maven Version Range. For <code>javafml</code> and <code>lowcodefml</code> , the version is the major version of the Forge version.	<code>"[46,)"</code>

Property	Type	Default	Description	Example
<code>license</code>	string	mandatory	The license the mod(s) in this JAR are provided under. It is suggested that this is set to the SPDX identifier you are using and/or a link to the license. You can visit https://choosealicense.com/ to help pick the license you want to use.	"MIT"
<code>showAsResourcePack</code>	boolean	false	When true , the mod(s)'s resources will be displayed as a separate resource pack on the 'Resource Packs' menu, rather than being combined with the 'Mod resources' pack.	true
<code>services</code>	array	<code>[]</code>	An array of services your mod uses . This is consumed as part of the created module for the mod from Forge's implementation of the Java Platform Module System.	<code>["net.minecraftforge.forgespi.la</code>
<code>properties</code>	table	<code>{}</code>	A table of substitution properties. This is used by <code>StringSubstitutor</code> to replace <code>\${file.<key>}</code> with its corresponding value. This is currently only used to replace the version in the mod-specific properties.	<code>{ "example" = "1.2.3" }</code> referenced by <code>\${file.example}</code>
<code>issueTrackerURL</code>	string	<i>nothing</i>	A URL representing the place to report and track issues with the mod(s).	" https://forums.minecraftforge.n

!!! important The **services** property is functionally equivalent to specifying the **uses** directive in a module, which allows *loading* a service of a given type.

Mod-Specific Properties

Mod-specific properties are tied to the specified mod using the `[[mods]]` header. This is an array of tables; all key/value properties will be attached to that mod until the next header.

```
# Properties for examplemod1
[[mods]]
modId = "examplemod1"

# Properties for examplemod2
[[mods]]
modId = "examplemod2"
```

Property	Type	Default	Description	Example
<code>modId</code>	string	mandatory	The unique identifier representing this mod. The id must match <code>^[a-z][a-z0-9_]{1,63}\$</code> (a string 2-64 characters; starts with a lowercase letter; made up of lowercase letters, numbers, or underscores).	"examplemod"

Property	Type	Default	Description	Example
namespace	string	value of modId	An override namespace for the mod. The namespace must match <code>^[a-z][a-z0-9_.-]{1,63}\$</code> (a string 2-64 characters; starts with a lowercase letter; made up of lowercase letters, numbers, underscores, dots, or dashes). Currently unused.	"example"
version	string	"1"	The version of the mod, preferably in a variation of Maven versioning. When set to <code>\${file.jarVersion}</code> , it will be replaced with the value of the <code>Implementation-Version</code> property in the JAR's manifest (displays as 0.0.0 in a development environment).	"1.20-1.0.0.0"
displayName	string	value of modId	The pretty name of the mod. Used when representing the mod on a screen (e.g., mod list, mod mismatch).	"Example Mod"
description	string	"MISSING DESCRIPTION"	The description of the mod shown in the mod list screen. It is recommended to use a multiline literal string.	"This is an example."
logoFile	string	<i>nothing</i>	The name and extension of an image file used on the mods list screen. The logo must be in the root of the JAR or directly in the root of the source set (e.g., <code>src/main/resources</code> for the main source set).	"example_logo.png"
logoBlur	boolean	true	Whether to use <code>GL_LINEAR*</code> (true) or <code>GL_NEAREST*</code> (false) to render the <code>logoFile</code> .	false
updateJSONURL	string	<i>nothing</i>	A URL to a JSON used by the update checker to make sure the mod you are playing is the latest version.	"https://files.minecraftforge.net"
features	table	{}	See 'features'.	{ java_version = "17" }
modproperties	table	{}	A table of key/values associated with this mod. Currently unused by Forge, but is mainly for use by mods.	{ example = "value" }
modUrl	string	<i>nothing</i>	A URL to the download page of the mod. Currently unused.	"https://files.minecraftforge.net"
credits	string	<i>nothing</i>	Credits and acknowledges for the mod shown on the mod list screen.	"The person over here and there."
authors	string	<i>nothing</i>	The authors of the mod shown on the mod list screen.	"Example Person"
displayURL	string	<i>nothing</i>	A URL to the display page of the mod shown on the mod list screen.	"https://minecraftforge.net/"
displayTest	string	"MATCH_VERSION"	See 'sides'.	"NONE"

Property	Type	Default	Description	Example
----------	------	---------	-------------	---------

Features The features system allows mods to demand that certain settings, software, or hardware are available when loading the system. When a feature is not satisfied, mod loading will fail, informing the user about the requirement. Currently, Forge provides the following features:

Feature	Description	Example
<code>java_version</code>	The acceptable version range of the Java version, expressed as a Maven Version Range. This should be the supported version used by Minecraft.	"[17,)"

Dependency Configurations

Mods can specify their dependencies, which are checked by Forge before loading the mods. These configurations are created using the array of tables `[[dependencies.<modid>]]` where `modid` is the identifier of the mod the dependency is for.

Property	Type	Default	Description	Example
<code>modId</code>	string	mandatory	The identifier of the mod added as a dependency.	"example_library"
<code>mandatory</code>	boolean	mandatory	Whether the game should crash when this dependency is not met.	true
<code>versionRange</code>	string	""	The acceptable version range of the language loader, expressed as a Maven Version Range. An empty string matches any version.	"[1, 2)"
<code>ordering</code>	string	"NONE"	Defines if the mod must load before ("BEFORE") or after ("AFTER") this dependency. If the ordering does not matter, return "NONE"	"AFTER"
<code>side</code>	string	"BOTH"	The physical side the dependency must be present on: "CLIENT", "SERVER", or "BOTH".	"CLIENT"
<code>referralUrl</code>	string	<i>nothing</i>	A URL to the download page of the dependency. Currently unused.	"https://library.example.com/"

!!! warning The **ordering** of two mods may cause a crash due to a cyclic dependency: for example, mod A must load "BEFORE" mod B and mod B "BEFORE" mod A.

Mod Entrypoints

Now that the `mods.toml` is filled out, we need to provide an endpoint to being programming the mod. Entrypoints are essentially the starting point for executing the mod. The endpoint itself is determined by the language loader used in the `mods.toml`.

javafml and @Mod

`javafml` is a language loader provided by Forge for the Java programming language. The endpoint is defined using a public class with the `@Mod` annotation. The value of `@Mod` must contain one of the mod ids specified within the `mods.toml`. From there, all initialization logic (e.g., registering events, adding `DeferredRegisters`) can be specified within the constructor of the class. The mod bus can be obtained from `FMLJavaModLoadingContext`.

```

@Mod("examplemod") // Must match mods.toml
public class Example {

    public Example() {
        // Initialize logic here
        var modBus = FMLJavaModLoadingContext.get().getModEventBus();

        // ...
    }
}

```

lowcodefml

`lowcodefml` is a language loader used as a way to distribute datapacks and resource packs as mods without the need of an in-code entrypoint. It is specified as `lowcodefml` rather than `nocodefml` for minor additions in the future that might require minimal coding.

Structuring Your Mod

Structured mods are beneficial for maintenance, making contributions, and providing a clearer understanding of the underlying codebase. Some of the recommendations from Java, Minecraft, and Forge are listed below.

!!! note You do not have to follow the advice below; you can structure your mod any way you see fit. However, it is still highly recommended to do so.

Packaging

When structuring your mod, pick a unique, top-level package structure. Many programmers will use the same name for different classes, interfaces, etc. Java allows classes to have the same name as long as they are in different packages. As such, if two classes have the same package with the same name, only one would be loaded, most likely causing the game to crash.

```

a.jar
- com.example.ExampleClass
b.jar
- com.example.ExampleClass // This class will not normally be loaded

```

This is even more relevant when it comes to loading modules. If there are class files in two packages under the same name in separate modules, this will cause the mod loader **to crash on startup** since mod modules are exported to the game and other mods.

```

module A
- package X
- class I
- class J
module B
- package X // This package will cause the mod loader to crash, as there already is a module with package X
- class R
- class S
- class T

```

As such, your top level package should be something that you own: a domain, email address, a subdomain of where your website, etc. It can even be your name or username as long as you can guarantee that it will be uniquely identifiable within the expected target.

Type	Value	Top-Level Package
Domain	example.com	com.example
Subdomain	example.github.io	io.github.example
Email	example@gmail.com	com.gmail.example

The next level package should then be your mod's id (e.g. `com.example.examplemod` where `examplemod` is the mod id). This will guarantee that, unless you have two mods with the same id (which should never be the case), your packages should not have any issues loading.

You can find some additional naming conventions on Oracle's tutorial page.

Sub-package Organization

In addition to the top-level package, it is highly recommend to break your mod's classes between subpackages. There are two major methods on how to do so:

- **Group By Function:** Make subpackages for classes with a common purpose. For example, blocks can be under `block` or `blocks`, entities under `entity` or `entities`, etc. Mojang uses this structure with the singular version of the word.
- **Group By Logic:** Make subpackages for classes with a common logic. For example, if you were creating a new type of crafting table, you would put its block, menu, item, and more under `feature.crafting_table`.

Client, Server, and Data Packages In general, code only for a given side or runtime should be isolated from the other classes in a separate subpackage. For example, code related to data generation should go in a `data` package while code only on the dedicated server should go in a `server` package.

However, it is highly recommended that client-only code should be isolated in a `client` subpackage. This is because dedicated servers have no access to any of the client-only packages in Minecraft. As such, having a dedicated package would provide a decent sanity check to verify you are not reaching across sides within your mod.

Class Naming Schemes

A common class naming scheme makes it easier to decipher the purpose of the class or to easily locate specific classes.

Classes are commonly suffixed with its type, for example:

- An `Item` called `PowerRing` -> `PowerRingItem`.
- A `Block` called `NotDirt` -> `NotDirtBlock`.
- A menu for an `Oven` -> `OvenMenu`.

!!! note Mojang typically follows a similar structure for all classes except entities. Those are represented by just their names (e.g. `Pig`, `Zombie`, etc.).

Choose One Method from Many

There are many methods for performing a certain task: registering an object, listening for events, etc. It's generally recommended to be consistent by using a single method to accomplish a given task. While this does improve code formatting, it also avoid any weird interactions or redundancies that may occur (e.g. your event listener executing twice).

Versioning

In general projects, semantic versioning is often used (which has the format `MAJOR.MINOR.PATCH`). However, in the case of modding it may be more beneficial to use the format `MCVERSION-MAJORMOD.MAJORAPI.MINOR.PATCH` to be able to differentiate between world-breaking and API-breaking changes of a mod.

!!! important Forge uses Maven version ranges to compare version strings, which is not fully compatible with the Semantic Versioning 2.0.0 spec, such as the 'prerelease' tag.

Examples

Here is a list of examples that can increment the various variables.

- `MCVERSION`

- Always matches the Minecraft version the mod is for.
- **MAJORMOD**
 - Removing items, blocks, block entities, etc.
 - Changing or removing previously existing mechanics.
 - Updating to a new Minecraft version.
- **MAJORAPI**
 - Changing the order or variables of enums.
 - Changing return types of methods.
 - Removing public methods altogether.
- **MINOR**
 - Adding items, blocks, block entities, etc.
 - Adding new mechanics.
 - Deprecating public methods. (This is not a MAJORAPI increment since it doesn't break an API.)
- **PATCH**
 - Bugfixes.

When incrementing any variable, all lesser variables should reset to 0. For instance, if MINOR would increment, PATCH would become 0. If MAJORMOD would increment, all other variables would become 0.

Work In Progress

If you are in the initial development stage of your mod (before any official releases), the MAJORMOD and MAJORAPI should always be 0. Only MINOR and PATCH should be updated every time you build your mod. Once you build an official release (most of the time with a stable API), you should increment MAJORMOD to version 1.0.0.0. For any further development stages, refer to the Prereleases and Release candidates section of this document.

Multiple Minecraft Versions

If the mod upgrades to a new version of Minecraft, and the old version will only receive bug fixes, the PATCH variable should be updated based on the version before the upgrade. If the mod is still in active development in both the old and the new version of Minecraft, it is advised to append the version to **both** build numbers. For example, if the mod is upgraded to version 3.0.0.0 due to a Minecraft version change, the old mod should also be updated to 3.0.0.0. The old version will become, for example, version 1.7.10-3.0.0.0, while the new version will become 1.8-3.0.0.0. If there are no changes at all when building for a newer Minecraft version, all variables except for the Minecraft version should stay the same.

Final Release

When dropping support for a Minecraft version, the last build for that version should get the **-final** suffix. This denotes that the mod will no longer be supported for the denoted MCVERSION and that players should upgrade to a newer version of the mod to continue receiving updates and bug fixes.

Pre-releases

It is also possible to prerelease work-in-progress features, which means new features are released that are not quite done yet. These can be seen as a sort of “beta”. These versions should be appended with **-betaX**, where X is the number of the prerelease. (This guide does not use **-pre** since, at the time of writing, it is not a valid alias for **-beta**.) Note that already released versions and versions before the initial release can not go into prerelease; variables (mostly MINOR, but MAJORAPI and MAJORMOD can also prerelease) should be updated accordingly before adding the **-beta** suffix. Versions before the initial release are simply work-in-progress builds.

Release Candidates

Release candidates act as prereleases before an actual version change. These versions should be appended with **-rcX**, where X is the number of the release candidate which should, in theory, only be increased for bugfixes. Already released versions can not receive release candidates; variables (mostly MINOR, but MAJORAPI and MAJORMOD can also prerelease) should be updated accordingly before adding the **-rc** suffix. When releasing a release candidate as stable build, it can either be exactly the same as the last release candidate or have a few more bug fixes.

Registries

Registration is the process of taking the objects of a mod (such as items, blocks, sounds, etc.) and making them known to the game. Registering things is important, as without registration the game will simply not know about these objects, which will cause unexplainable behaviors and crashes.

Most things that require registration in the game are handled by the Forge registries. A registry is an object similar to a map that assigns values to keys. Forge uses registries with `ResourceLocation` keys to register objects. This allows the `ResourceLocation` to act as the “registry name” for objects.

Every type of registrable object has its own registry. To see all registries wrapped by Forge, see the `ForgeRegistries` class. All registry names within a registry must be unique. However, names in different registries will not collide. For example, there’s a `Block` registry, and an `Item` registry. A `Block` and an `Item` may be registered with the same name `example:thing` without colliding; however, if two different `Blocks` or `Items` were registered with the same exact name, the second object will override the first.

Methods for Registering

There are two proper ways to register objects: the `DeferredRegister` class, and the `RegisterEvent` lifecycle event.

DeferredRegister

`DeferredRegister` is the recommended way to register objects. It allows the use and convenience of static initializers while avoiding the issues associated with it. It simply maintains a list of suppliers for entries and registers the objects from those suppliers during `RegisterEvent`.

An example of a mod registering a custom block:

```
private static final DeferredRegister<Block> BLOCKS = DeferredRegister.create(ForgeRegistries.BLOCKS, MODID);

public static final RegistryObject<Block> ROCK_BLOCK = BLOCKS.register("rock", () -> new
Block(BlockBehaviour.Properties.of().mapColor(MapColor.STONE)));

public ExampleMod() {
    BLOCKS.register(FMLJavaModLoadingContext.get().getModEventBus());
}
```

RegisterEvent

`RegisterEvent` is the second way to register objects. This event is fired for each registry after the mod constructors and before the loading of configs. Objects are registered using `#register` by passing in the registry key, the name of the registry object, and the object itself. There is an additional `#register` overload which takes in a consumed helper to register an object with a given name. It is recommended to use this method to avoid unnecessary object creation.

Here is an example: (the event handler is registered on the *mod event bus*)

```
@SubscribeEvent
public void register(RegisterEvent event) {
    event.register(ForgeRegistries.Keys.BLOCKS,
        helper -> {
            helper.register(new ResourceLocation(MODID, "example_block_1"), new Block(...));
            helper.register(new ResourceLocation(MODID, "example_block_2"), new Block(...));
            helper.register(new ResourceLocation(MODID, "example_block_3"), new Block(...));
            // ...
        }
    );
}
```

Registries that aren’t Forge Registries

Not all registries are wrapped by Forge. These can be static registries, like `LootItemConditionType`, which are safe to use. There are also dynamic registries, like `ConfiguredFeature` and some other worldgen registries, which are typically represented in JSON. `DeferredRegister#create` has an overload which allows modders to specify

the registry key of which vanilla registry to create a `RegistryObject` for. The registry method and attaching to the mod event bus is the same as other `DeferredRegisters`.

!!! important Dynamic registry objects can **only** be registered through data files (e.g. JSON). They **cannot** be registered in-code.

```
private static final DeferredRegister<LootItemConditionType> REGISTER =
DeferredRegister.create(Registries.LOOT_CONDITION_TYPE, "examplemod");

public static final RegistryObject<LootItemConditionType> EXAMPLE_LOOT_ITEM_CONDITION_TYPE =
REGISTER.register("example_loot_item_condition_type", () -> new LootItemConditionType(...));
```

!!! note Some classes cannot by themselves be registered. Instead, `*Type` classes are registered, and used in the formers' constructors. For example, `BlockEntity` has `BlockEntityType`, and `Entity` has `EntityType`. These `*Type` classes are factories that simply create the containing type on demand.

These factories are created through the use of their `*Type$Builder` classes. An example: (`REGISTER` reference)
```java  
public static final RegistryObject<BlockEntityType<ExampleBlockEntity>> EXAMPLE\_BLOCK\_ENTITY = REGISTER.register("example\_block\_entity", () -> BlockEntityType.Builder.of(ExampleBlockEntity::new, EXAMPLE\_BLOCK.get()).build());  
```

Referencing Registered Objects

Registered objects should not be stored in fields when they are created and registered. They are to be always newly created and registered whenever `RegisterEvent` is fired for that registry. This is to allow dynamic loading and unloading of mods in a future version of Forge.

Registered objects must always be referenced through a `RegistryObject` or a field with `@ObjectHolder`.

Using RegistryObjects

`RegistryObjects` can be used to retrieve references to registered objects once they are available. These are used by `DeferredRegister` to return a reference to the registered objects. Their references are updated after `RegisterEvent` is called for their registry, along with the `@ObjectHolder` annotations.

To get a `RegistryObject`, call `RegistryObject#create` with a `ResourceLocation` and the `IForgeRegistry` of the registrable object. Custom registries can also be used by supplying the registry name instead. Store the `RegistryObject` in a `public static final` field, and call `#get` whenever you need the registered object.

An example of using `RegistryObject`:

```
public static final RegistryObject<Item> BOW = RegistryObject.create(new ResourceLocation("minecraft:bow"),
ForgeRegistries.ITEMS);

// assume that 'neomagicae:mana_type' is a valid registry, and 'neomagicae:coffeinum' is a valid object within that registry
public static final RegistryObject<ManaType> COFFEINUM = RegistryObject.create(new ResourceLocation("neomagicae",
"coffeinum"), new ResourceLocation("neomagicae", "mana_type"), "neomagicae");
```

Using @ObjectHolder

Registered objects from registries can be injected into the `public static` fields by annotating classes or fields with `@ObjectHolder` and supplying enough information to construct a `ResourceLocation` to identify a specific object in a specific registry.

The rules for `@ObjectHolder` are as follows:

- If the class is annotated with `@ObjectHolder`, its value will be the default namespace for all fields within if not explicitly defined
- If the class is annotated with `@Mod`, the modid will be the default namespace for all annotated fields within if not explicitly defined
- A field is considered for injection if:

- it has at least the modifiers `public static`;
- the **field** is annotated with `@ObjectHolder`, and:
 - * the name value is explicitly defined; and
 - * the registry name value is explicitly defined
- A compile-time exception is thrown if a field does not have a corresponding registry or name.
- An exception is thrown if the resulting `ResourceLocation` is incomplete or invalid (non-valid characters in path)
- If no other errors or exceptions occur, the field will be injected
- If all of the above rules do not apply, no action will be taken (and a message may be logged)

`@ObjectHolder`-annotated fields are injected with their values after `RegisterEvent` is fired for their registry, along with the `RegistryObjects`.

!!! note If the object does not exist in the registry when it is to be injected, a debug message will be logged and no value will be injected.

As these rules are rather complicated, here are some examples:

```
class Holder {
    @ObjectHolder(registryName = "minecraft:enchantment", value = "minecraft:flame")
    public static final Enchantment flame = null;    // Annotation present. [public static] is required. [final] is optional.
                                                    // Registry name is explicitly defined: "minecraft:enchantment"
                                                    // Resource location is explicitly defined: "minecraft:flame"
                                                    // To inject: "minecraft:flame" from the [Enchantment] registry

    public static final Biome ice_flat = null;      // No annotation on the field.
                                                    // Therefore, the field is ignored.

    @ObjectHolder("minecraft:creeper")
    public static Entity creeper = null;            // Annotation present. [public static] is required.
                                                    // The registry has not been specified on the field.
                                                    // Therefore, THIS WILL PRODUCE A COMPILE-TIME EXCEPTION.

    @ObjectHolder(registryName = "potion")
    public static final Potion levitation = null;    // Annotation present. [public static] is required. [final] is optional.
                                                    // Registry name is explicitly defined: "minecraft:potion"
                                                    // Resource location is not specified on the field
                                                    // Therefore, THIS WILL PRODUCE A COMPILE-TIME EXCEPTION.
}
```

Creating Custom Forge Registries

Custom registries can usually just be a simple map of key to value. This is a common style; however, it forces a hard dependency on the registry being present. It also requires that any data that needs to be synced between sides must be done manually. Custom Forge Registries provide a simple alternative for creating soft dependents along with better management and automatic syncing between sides (unless told otherwise). Since the objects also use a Forge registry, registration becomes standardized in the same way.

Custom Forge Registries are created with the help of a `RegistryBuilder`, through either `NewRegistryEvent` or the `DeferredRegister`. The `RegistryBuilder` class takes various parameters (such as the registry's name, id range, and various callbacks for different events happening on the registry). New registries are registered to the `RegistryManager` after `NewRegistryEvent` finishes firing.

Any newly created registry should use its associated registration method to register the associated objects.

Using `NewRegistryEvent`

When using `NewRegistryEvent`, calling `#create` with a `RegistryBuilder` will return a supplier-wrapped registry. The supplied registry can be accessed after `NewRegistryEvent` has finished posting to the mod event bus. Getting the custom registry from the supplier before `NewRegistryEvent` finishes firing will result in a `null` value.

New Datapack Registries New datapack registries can be added using the `DataPackRegistryEvent$NewRegistry` event on the mod event bus. The registry is created via `#dataPackRegistry`

by passing in the `ResourceKey` representing the registry name and the `Codec` used to encode and decode the data from JSON. An optional `Codec` can be provided to sync the datapack registry to the client.

!!! important Datapack Registries cannot be created with `DeferredRegister`. They can only be created through the event.

With DeferredRegister

The `DeferredRegister` method is once again another wrapper around the above event. Once a `DeferredRegister` is created in a constant field using the `#create` overload which takes in the registry name and the mod id, the registry can be constructed via `DeferredRegister#makeRegistry`. This takes in a supplied `RegistryBuilder` containing any additional configurations. The method already populates `#setName` by default. Since this method can be returned at any time, a supplied version of an `IForgeRegistry` is returned instead. Getting the custom registry from the supplier before `NewRegistryEvent` is fired will result in a `null` value.

!!! important `DeferredRegister#makeRegistry` must be called before the `DeferredRegister` is added to the mod event bus via `#register`. `#makeRegistry` also uses the `#register` method to create the registry during `NewRegistryEvent`.

Handling Missing Entries

There are cases where certain registry objects will cease to exist whenever a mod is updated or, more likely, removed. It is possible to specify actions to handle the missing mapping through the third of the registry events: `MissingMappingsEvent`. Within this event, a list of missing mappings can be obtained either by `#getMappings` given a registry key and mod id or all mappings via `#getAllMappings` given a registry key.

!!! important `MissingMappingsEvent` is fired on the **Forge** event bus.

For each `Mapping`, one of four mapping types can be selected to handle the missing entry:

Action	Description
IGNORE	Ignores the missing entry and abandons the mapping.
WARN	Generates a warning in the log.
FAIL	Prevents the world from loading.
REMAP	Remaps the entry to an already registered, non-null object.

If no action is specified, then the default action will occur by notifying the user about the missing entry and whether they still would like to load the world. All actions besides remapping will prevent any other registry object from taking the place of the existing id in case the associated entry ever gets added back into the game.

Sides in Minecraft

A very important concept to understand when modding Minecraft are the two sides: *client* and *server*. There are many, many common misconceptions and mistakes regarding siding, which can lead to bugs that might not crash the game, but can rather have unintended and often unpredictable effects.

Different Kinds of Sides

When we say “client” or “server”, it usually follows with a fairly intuitive understanding of what part of the game we are talking about. After all, a client is what the user interacts with, and a server is where the user connects for a multiplayer game. Easy, right?

As it turns out, there can be some ambiguity even with two such terms. Here we disambiguate the four possible meanings of “client” and “server”:

- Physical client - The *physical client* is the entire program that runs whenever you launch Minecraft from the launcher. All threads, processes, and services that run during the game’s graphical, interactable lifetime are part of the physical client.

- Physical server - Often known as the dedicated server, the *physical server* is the entire program that runs whenever you launch any sort of `minecraft_server.jar` that does not bring up a playable GUI.
- Logical server - The *logical server* is what runs game logic: mob spawning, weather, updating inventories, health, AI, and all other game mechanics. The logical server is present within a physical server, but it also can run inside a physical client together with a logical client, as a single player world. The logical server always runs in a thread named the **Server Thread**.
- Logical client - The *logical client* is what accepts input from the player and relays it to the logical server. In addition, it also receives information from the logical server and makes it available graphically to the player. The logical client runs in the **Render Thread**, though often several other threads are spawned to handle things like audio and chunk render batching.

In the MinecraftForge codebase, the physical side is represented by an enum called `Dist`, while the logical side is represented by an enum called `LogicalSide`.

Performing Side-Specific Operations

`Level#isClientSide`

This boolean check will be your most used way to check sides. Querying this field on a `Level` object establishes the **logical** side the level belongs to. That is, if this field is `true`, the level is currently running on the logical client. If the field is `false`, the level is running on the logical server. It follows that the physical server will always contain `false` in this field, but we cannot assume that `false` implies a physical server, since this field can also be `false` for the logical server inside a physical client (in other words, a single player world).

Use this check whenever you need to determine if game logic and other mechanics should be run. For example, if you want to damage the player every time they click your block, or have your machine process dirt into diamonds, you should only do so after ensuring `#isClientSide` is `false`. Applying game logic to the logical client can cause desynchronization (ghost entities, desynchronized stats, etc.) in the best case, and crashes in the worst case.

This check should be used as your go-to default. Aside from `DistExecutor`, rarely will you need the other ways of determining side and adjusting behavior.

`DistExecutor`

Considering the use of a single “universal” jar for client and server mods, and the separation of the physical sides into two jars, an important question comes to mind: How do we use code that is only present on one physical side? All code in `net.minecraft.client` is only present on the physical client. If any class you write references those names in any way, they will crash the game when that respective class is loaded in an environment where those names do not exist. A very common mistake in beginners is to call `Minecraft.getInstance().<doStuff>()` in block or block entity classes, which will crash any physical server as soon as the class is loaded.

How do we resolve this? Luckily, FML has `DistExecutor`, which provides various methods to run different methods on different physical sides, or a single method only on one side.

!!! note It is important to understand that FML checks based on the **physical** side. A single player world (logical server + logical client within a physical client) will always use `Dist.CLIENT`!

`DistExecutor` works by taking in a supplied supplier executing a method, effectively preventing classloading by taking advantage of the `invokedynamic` JVM instruction. The executed method should be static and within a different class. Additionally, if no parameters are present for the static method, a method reference should be used instead of a supplier executing a method.

There are two main methods within `DistExecutor`: `#runWhenOn` and `#callWhenOn`. The methods take in the physical side the executing method should run on and the supplied executing method which either runs or returns a result respectively.

These two methods are subdivided further into `#safe*` and `#unsafe*` variants. Safe and unsafe variants are misnomers for their purposes. The main difference is that when in a development environment, the `#safe*` methods will validate that the supplied executing method is a lambda returning a method reference to another class with an error being thrown otherwise. Within the production environment, `#safe*` and `#unsafe*` are functionally the same.

```
// In a client class: ExampleClass
public static void unsafeRunMethodExample(Object param1, Object param2) {
    // ...
}

public static Object safeCallMethodExample() {
    // ...
}

// In some common class
DistExecutor.unsafeRunWhenOn(Dist.CLIENT, () -> ExampleClass.unsafeRunMethodExample(var1, var2));

DistExecutor.safeCallWhenOn(Dist.CLIENT, () -> ExampleClass::safeCallMethodExample);
```

!!! warning Due to a change in how `invokedynamic` works in Java 9+, all `#safe*` variants of the `DistExecutor` methods throw the original exception wrapped within a `BootstrapMethodError` in the development environment. `#unsafe*` variants or a check to `FMLEnvironment#dist` should be used instead.

Thread Groups

If `Thread.currentThread().getThreadGroup() == SidedThreadGroups.SERVER` is true, it is likely the current thread is on the logical server. Otherwise, it is likely on the logical client. This is useful to retrieve the **logical** side when you do not have access to a `Level` object to check `isClientSide`. It *guesses* which logical side you are on by looking at the group of the currently running thread. Because it is a guess, this method should only be used when other options have been exhausted. In nearly every case, you should prefer checking `Level#isClientSide`.

FMLEnvironment#dist and @OnlyIn

`FMLEnvironment#dist` holds the **physical** side your code is running on. Since it is determined at startup, it does not rely on guessing to return its result. The number of use cases for this is limited, however.

Annotating a method or field with the `@OnlyIn(Dist)` annotation indicates to the loader that the respective member should be completely stripped out of the definition not on the specified **physical** side. Usually, these are only seen when browsing through the decompiled Minecraft code, indicating methods that the Mojang obfuscator stripped out. There is **NO** reason for using this annotation directly. Use `DistExecutor` or a check on `FMLEnvironment#dist` instead.

Common Mistakes

Reaching Across Logical Sides

Whenever you want to send information from one logical side to another, you must **always** use network packets. It is incredibly tempting, when in a single player scenario, to directly transfer data from the logical server to the logical client.

This is actually very commonly inadvertently done through static fields. Since the logical client and logical server share the same JVM in a single player scenario, both threads writing to and reading from static fields will cause all sorts of race conditions and the classical issues associated with threading.

This mistake can also be made explicitly by accessing physical client-only classes such as `Minecraft` from common code that runs or can run on the logical server. This mistake is easy to miss for beginners who debug in a physical client. The code will work there, but it will immediately crash on a physical server.

Writing One-Sided Mods

In recent versions, Minecraft Forge has removed a “sidedness” attribute from the `mods.toml`. This means that your mods are expected to work whether they are loaded on the physical client or the physical server. So for one-sided mods, you would typically register your event handlers inside a `DistExecutor#safeRunWhenOn` or `DistExecutor#unsafeRunWhenOn` instead of directly calling the relevant registration methods in your mod constructor. Basically, if your mod is loaded on the wrong side, it should simply do nothing, listen to no events, and so on. A one-sided mod by nature should not register blocks, items, ... since they would need to be available on the other side, too.

Additionally, if your mod is one-sided, it typically does not forbid the user from joining a server that is lacking that mod. Therefore, you should set the `displayTest` property in your `mods.toml` to whatever value is necessary.

```
[[mods]]
# ...

# MATCH_VERSION means that your mod will cause a red X if the versions on client and server differ. This is the
# default behaviour and should be what you choose if you have server and client elements to your mod.
# IGNORE_SERVER_VERSION means that your mod will not cause a red X if it's present on the server but not on the
# client. This is what you should use if you're a server only mod.
# IGNORE_ALL_VERSION means that your mod will not cause a red X if it's present on the client or the server. This is
# a special case and should only be used if your mod has no server component.
# NONE means that no display test is set on your mod. You need to do this yourself, see IExtensionPoint.DisplayTest
# for more information. You can define any scheme you wish with this value.
# IMPORTANT NOTE: this is NOT an instruction as to which environments (CLIENT or DEDICATED SERVER) your mod loads
# on. Your mod should load (and maybe do nothing!) wherever it finds itself.
displayTest="IGNORE_ALL_VERSION" # MATCH_VERSION is the default if nothing is specified (#optional)
```

If a custom display test is to be used, then the `displayTest` option should be set to `NONE`, and an `IExtensionPoint$DisplayTest` extension should be registered:

```
//Make sure the mod being absent on the other network side does not cause the client to display the server as
incompatible
ModLoadingContext.get().registerExtensionPoint(IExtensionPoint.DisplayTest.class, () -> new
IExtensionPoint.DisplayTest(() -> NetworkConstants.IGNORESERVERONLY, (a, b) -> true));
```

This tells the client that it should ignore the server version being absent, and the server that it should not tell the client this mod should be present. So this snippet works both for client- and server-only-sided mods.

Events

Forge uses an event bus that allows mods to intercept events from various Vanilla and mod behaviors.

Example: An event can be used to perform an action when a Vanilla stick is right clicked.

The main event bus used for most events is located at `MinecraftForge#EVENT_BUS`. There is another event bus for mod specific events located at `FMLJavaModLoadingContext#getModEventBus` that you should only use in specific cases. More information about this bus can be found below.

Every event is fired on one of these buses: most events are fired on the main forge event bus, but some are fired on the mod specific event buses.

An event handler is some method that has been registered to an event bus.

Creating an Event Handler

Event handlers methods have a single parameter and do not return a result. The method could be static or instance depending on implementation.

Event handlers can be directly registered using `IEventBus#addListener` for or `IEventBus#addGenericListener` for generic events (as denoted by subclassing `GenericEvent<T>`). Either listener adder takes in a consumer representing the method reference. Generic event handlers need to specify the class of the generic as well. Event handlers must be registered within the constructor of the main mod class.

```
// In the main mod class ExampleMod

// This event is on the mod bus
private void modEventHandler(RegisterEvent event) {
    // Do things here
}

// This event is on the forge bus
private static void forgeEventHandler(AttachCapabilitiesEvent<Entity> event) {
    // ...
}

// In the mod constructor
```



```
modEventBus.addListener(this::modEventHandler);
forgeEventBus.addGenericListener(Entity.class, ExampleMod::forgeEventHandler);
```

Instance Annotated Event Handlers

This event handler listens for the `EntityItemPickupEvent`, which is, as the name states, posted to the event bus whenever an `Entity` picks up an item.

```
public class MyForgeEventHandler {
    @SubscribeEvent
    public void pickupItem(EntityItemPickupEvent event) {
        System.out.println("Item picked up!");
    }
}
```

To register this event handler, use `MinecraftForge.EVENT_BUS.register(...)` and pass it an instance of the class the event handler is within. If you want to register this handler to the mod specific event bus, you should use `FMLJavaModLoadingContext.get().getModEventBus().register(...)` instead.

Static Annotated Event Handlers

An event handler may also be static. The handling method is still annotated with `@SubscribeEvent`. The only difference from an instance handler is that it is also marked `static`. In order to register a static event handler, an instance of the class won't do. The `Class` itself has to be passed in. An example:

```
public class MyStaticForgeEventHandler {
    @SubscribeEvent
    public static void arrowNocked(ArrowNockEvent event) {
        System.out.println("Arrow nocked!");
    }
}
```

which must be registered like this: `MinecraftForge.EVENT_BUS.register(MyStaticForgeEventHandler.class)`.

Automatically Registering Static Event Handlers

A class may be annotated with the `@Mod$EventBusSubscriber` annotation. Such a class is automatically registered to `MinecraftForge#EVENT_BUS` when the `@Mod` class itself is constructed. This is essentially equivalent to adding `MinecraftForge.EVENT_BUS.register(AnnotatedClass.class)`; at the end of the `@Mod` class's constructor.

You can pass the bus you want to listen to the `@Mod$EventBusSubscriber` annotation. It is recommended you also specify the mod id, since the annotation process may not be able to figure it out, and the bus you are registering to, since it serves as a reminder to make sure you are on the correct one. You can also specify the `Dists` or physical sides to load this event subscriber on. This can be used to not load client specific event subscribers on the dedicated server.

An example for a static event listener listening to `RenderLevelStageEvent` which will only be called on the client:

```
@Mod.EventBusSubscriber(modid = "mymod", bus = Bus.FORGE, value = Dist.CLIENT)
public class MyStaticClientOnlyEventHandler {
    @SubscribeEvent
    public static void drawLast(RenderLevelStageEvent event) {
        System.out.println("Drawing!");
    }
}
```

!!! note This does not register an instance of the class; it registers the class itself (i.e. the event handling methods must be static).

Canceling

If an event can be canceled, it will be marked with the `@Cancelable` annotation, and the method `Event#isCancelable()` will return `true`. The cancel state of a cancelable event may be modified by calling `Event#setCanceled(boolean canceled)`, wherein passing the boolean value `true` is interpreted as canceling the

event, and passing the boolean value `false` is interpreted as “un-canceling” the event. However, if the event cannot be canceled (as defined by `Event#isCancelable()`), an `UnsupportedOperationException` will be thrown regardless of the passed boolean value, since the cancel state of a non-cancelable event is considered immutable.

!!! important Not all events can be canceled! Attempting to cancel an event that is not cancelable will result in an unchecked `UnsupportedOperationException` being thrown, which is expected to result in the game crashing! Always check that an event can be canceled using `Event#isCancelable()` before attempting to cancel it!

Results

Some events have an `Event$Result`. A result can be one of three things: `DENY` which stops the event, `DEFAULT` which uses the Vanilla behavior, and `ALLOW` which forces the action to take place, regardless if it would have originally. The result of an event can be set by calling `#setResult` with an `Event$Result` on the event. Not all events have results; an event with a result will be annotated with `@HasResult`.

!!! important Different events may use results in different ways, refer to the event’s JavaDoc before using the result.

Priority

Event handler methods (marked with `@SubscribeEvent`) have a priority. You can set the priority of an event handler method by setting the `priority` value of the annotation. The priority can be any value of the `EventPriority` enum (`HIGHEST`, `HIGH`, `NORMAL`, `LOW`, and `LOWEST`). Event handlers with priority `HIGHEST` are executed first and from there in descending order until `LOWEST` events which are executed last.

Sub Events

Many events have different variations of themselves. These can be different but all based around one common factor (e.g. `PlayerEvent`) or can be an event that has multiple phases (e.g. `PotionBrewEvent`). Take note that if you listen to the parent event class, you will receive calls to your method for *all* subclasses.

Mod Event Bus

The mod event bus is primarily used for listening to lifecycle events in which mods should initialize. Each event on the mod bus is required to implement `IModBusEvent`. Many of these events are also ran in parallel so mods can be initialized at the same time. This does mean you can’t directly execute code from other mods in these events. Use the `InterModComms` system for that.

These are the four most commonly used lifecycle events that are called during mod initialization on the mod event bus:

- `FMLCommonSetupEvent`
- `FMLClientSetupEvent` & `FMLDedicatedServerSetupEvent`
- `InterModEnqueueEvent`
- `InterModProcessEvent`

!!! note The `FMLClientSetupEvent` and `FMLDedicatedServerSetupEvent` are only called on their respective distribution.

These four lifecycle events are all ran in parallel since they all are a subclass of `ParallelDispatchEvent`. If you want to run code on the main thread during any `ParallelDispatchEvent`, you can use the `#enqueueWork` to do so.

Next to the lifecycle events, there are a few miscellaneous events that are fired on the mod event bus where you can register, set up, or initialize various things. Most of these events are not ran in parallel in contrast to the lifecycle events. A few examples:

- `RegisterColorHandlersEvent`
- `ModelEvent$BakingCompleted`
- `TextureStitchEvent`
- `RegisterEvent`

A good rule of thumb: events are fired on the mod event bus when they should be handled during initialization of a mod.

Mod Lifecycle

During the mod loading process, the various lifecycle events are fired on the mod-specific event bus. Many actions are performed during these events, such as registering objects, preparing for data generation, or communicating with other mods.

Event listeners should be registered either using `@EventBusSubscriber(bus = Bus.MOD)` or in the mod constructor:

```
@Mod.EventBusSubscriber(modid = "mymod", bus = Mod.EventBusSubscriber.Bus.MOD)
public class MyModEventSubscriber {
    @SubscribeEvent
    static void onCommonSetup(FMLCommonSetupEvent event) { ... }
}

@Mod("mymod")
public class MyMod {
    public MyMod() {
        FMLModLoadingContext.get().getModEventBus().addListener(this::onCommonSetup);
    }

    private void onCommonSetup(FMLCommonSetupEvent event) { ... }
}
```

!!! warning Most of the lifecycle events are fired in parallel: all mods will concurrently receive the same event.

Mods *must* take care to be thread-safe, like when calling other mods' APIs or accessing vanilla systems.

Registry Events

The registry events are fired after the mod instance construction. There are three: `NewRegistryEvent`, `DataPackRegistryEvent$NewRegistry` and `RegisterEvent`. These events are fired synchronously during mod loading.

`NewRegistryEvent` allows modders to register their own custom registries, using the `RegistryBuilder` class.

`DataPackRegistryEvent$NewRegistry` allows modders to register custom datapack registries by providing a `Codec` to encode and decode the object from JSON.

`RegisterEvent` is for registering objects into the registries. The event is fired for each registry.

Data Generation

If the game is setup to run data generators, then the `GatherDataEvent` will be the last event to fire. This event is for registering mods' data providers to their associated data generator. This event is also fired synchronously.

Common Setup

`FMLCommonSetupEvent` is for actions that are common to both physical client and server, such as registering capabilities.

Sided Setup

The sided-setup events are fired on their respective physical sides: `FMLClientSetupEvent` on the physical client, and `FMLDedicatedServerSetupEvent` for the dedicated server. This is where physical side-specific initialization should occur, such as registering client-side key bindings.

InterModComms

This is where messages can be sent to mods for cross-mod compatibility. There are two events: `InterModEnqueueEvent` and `InterModProcessEvent`.

`InterModComms` is the class responsible for holding messages for mods. The methods are safe to call during the lifecycle events, as it is backed by a `ConcurrentMap`.

During the `InterModEnqueueEvent`, use `InterModComms#sendTo` to send messages to different mods. These methods take in the mod id that will be sent the message, the key associated with the message data, and a supplier holding the message data. Additionally, the sender of the message can also be specified, but by default it will be the mod id of the caller.

Then during the `InterModProcessEvent`, use `InterModComms#getMessages` to get a stream of all received messages. The mod id supplied will almost always be the mod id of the mod the method is called on. Additionally, a predicate can be specified to filter out the message keys. This will return a stream of `IMCMessages` which hold the sender of the data, the receiver of the data, the data key, and the supplied data itself.

!!! note There are two other lifecycle events: `FMLConstructModEvent`, fired directly after mod instance construction but before the `RegisterEvent`, and `FMLLoadCompleteEvent`, fired after the `InterModComms` events, for when the mod loading process is complete.

Resources

A resource is extra data used by the game, and is stored in a data file, instead of being in the code. Minecraft has two primary resource systems active: one on the logical client used for visuals such as models, textures, and localization called **assets**, and one on the logical server used for gameplay such as recipes and loot tables called **data**. Resource packs control the former, while Datapacks control the latter.

In the default mod development kit, assets and data directories are located under the `src/main/resources` directory of the project.

When multiple resource packs or data packs are enabled, they are merged. Generally, files from packs at the top of the stack override those below; however, for certain files, such as localization files and tags, data is actually merged contentwise. Mods define resource and data packs in their `resources` directories, but they are seen as subsets of the “Mod Resources” pack. Mod resource packs cannot be disabled, but they can be overridden by other resource packs. Mod datapacks can be disabled with the vanilla `/datapack` command.

All resources should have snake case paths and filenames (lowercase, using “_” for word boundaries), which is enforced in 1.11 and above.

ResourceLocation

Minecraft identifies resources using `ResourceLocations`. A `ResourceLocation` contains two parts: a namespace and a path. It generally points to the resource at `assets/<namespace>/<ctx>/<path>`, where `ctx` is a context-specific path fragment that depends on how the `ResourceLocation` is being used. When a `ResourceLocation` is written/read as from a string, it is seen as `<namespace>:<path>`. If the namespace and the colon are left out, then when the string is read into an `ResourceLocation` the namespace will always default to “minecraft”. A mod should put its resources into a namespace with the same name as its mod id (e.g. a mod with the id `examplemod` should place its resources in `assets/examplemod` and `data/examplemod` respectively, and `ResourceLocations` pointing to those files would look like `examplemod:<path>`). This is not a requirement, and in some cases it can be desirable to use a different (or even more than one) namespace. `ResourceLocations` are used outside the resource system, too, as they happen to be a great way to uniquely identify objects (e.g. registries).

Internationalization and Localization

Internationalization, i18n for short, is a way of designing code so that it requires no changes to be adapted for various languages. Localization is the process of adapting displayed text to the user’s language.

I18n is implemented using *translation keys*. A translation key is a string that identifies a piece of displayable text in no specific language. For example, `block.minecraft.dirt` is the translation key referring to the name of the Dirt block. This way, displayable text may be referenced with no concern for a specific language. The code requires no changes to be adapted in a new language.

Localization will happen in the game's locale. In a Minecraft client the locale is specified by the language settings. On a dedicated server, the only supported locale is `en_us`. A list of available locales can be found on the Minecraft Wiki.

Language files

Language files are located by `assets/[namespace]/lang/[locale].json` (e.g. all US English translations provided by `examplemod` would be within `assets/examplemod/lang/en_us.json`). The file format is simply a json map from translation keys to values. The file must be encoded in UTF-8. Old `.lang` files can be converted to json using a converter.

```
{
  "item.examplemod.example_item": "Example Item Name",
  "block.examplemod.example_block": "Example Block Name",
  "commands.examplemod.examplecommand.error": "Example Command Errored!"
}
```

Usage with Blocks and Items

Block, Item and a few other Minecraft classes have built-in translation keys used to display their names. These translation keys are specified by overriding `#getDescriptionId`. Item also has `#getDescriptionId(ItemStack)` which can be overridden to provide different translation keys depending on `ItemStack` NBT.

By default, `#getDescriptionId` will return `block.` or `item.` prepended to the registry name of the block or item, with the colon replaced by a dot. `BlockItems` override this method to take their corresponding `Block`'s translation key by default. For example, an item with ID `examplemod:example_item` effectively requires the following line in a language file:

```
{
  "item.examplemod.example_item": "Example Item Name"
}
```

!!! note The only purpose of a translation key is internationalization. Do not use them for logic. Use registry names instead.

Localization methods

!!! warning A common issue is having the server localize for clients. The server can only localize in its own locale, which does not necessarily match the locale of connected clients.

To respect the language settings of clients, the server should have clients localize text in their own locale.

`net.minecraft.client.resources.language.I18n` (client only)

This `I18n` class can only be found on a Minecraft client! It is intended to be used by code that only runs on the client. Attempts to use this on a server will throw exceptions and crash.

- `get(String, Object...)` localizes in the client's locale with formatting. The first parameter is a translation key, and the rest are formatting arguments for `String.format(String, Object...)`.

TranslatableContents

`TranslatableContents` is a `ComponentContents` that is localized and formatted lazily. It is very useful when sending messages to players because it will be automatically localized in their own locale.

The first parameter of the `TranslatableContents(String, Object...)` constructor is a translation key, and the rest are used for formatting. The only supported format specifiers are `%s` and `%1$s`, `%2$s`, `%3$s` etc. Formatting arguments may be `Components` that will be inserted into the resulting formatted text with all their attributes preserved.

A `MutableComponent` can be created using `Component#translatable` by passing in the `TranslatableContents`'s parameters. It can also be created using `MutableComponent#create` by passing in the `ComponentContents` itself.

TextComponentHelper

- `createComponentTranslation(CommandSource, String, Object...)` creates a localized and formatted `MutableComponent` depending on a receiver. The localization and formatting is done eagerly if the receiver is a vanilla client. If not, the localization and formatting is done lazily with a `Component` containing `TranslatableContents`. This is only useful if the server should allow vanilla clients to connect.

Blocks

Blocks are, obviously, essential to the Minecraft world. They make up all of the terrain, structures, and machines. Chances are if you are interested in making a mod, then you will want to add some blocks. This page will guide you through the creation of blocks, and some of the things you can do with them.

Creating a Block

Basic Blocks

For simple blocks, which need no special functionality (think cobblestone, wooden planks, etc.), a custom class is not necessary. You can create a block by instantiating the `Block` class with a `BlockBehaviour$Properties` object. This `BlockBehaviour$Properties` object can be made using `BlockBehaviour$Properties#of`, and it can be customized by calling its methods. For instance:

- **strength** - The hardness controls the time it takes to break the block. It is an arbitrary value. For reference, stone has a hardness of 1.5, and dirt 0.5. If the block should be unbreakable a hardness of -1.0 should be used, see the definition of `Blocks#BEDROCK` as an example. The resistance controls the explosion resistance of the block. For reference, stone has a resistance of 6.0, and dirt 0.5.
- **sound** - Controls the sound the block makes when it is punched, broken, or placed. Requires a `SoundType` argument, see the sounds page for more details.
- **lightLevel** - Controls the light emission of the block. Takes a function with a `BlockState` parameter that returns a value from zero to fifteen.
- **friction** - Controls how slippery the block is. For reference, ice has a slipperiness of 0.98.

All these methods are *chainable* which means you can call them in series. See the `Blocks` class for examples of this.

!!! note `Blocks` have no setter for their `CreativeModeTab`. This is handled by the `BuildCreativeModeTabContentsEvent` if the block has an associated item (e.g. `BlockItem`). Furthermore, there is no setter for translation key of the block as it is generated from the registry name via `Block#getDescriptionId`.

Advanced Blocks

Of course, the above only allows for extremely basic blocks. If you want to add functionality, like player interaction, a custom class is required. However, the `Block` class has many methods and unfortunately not every single one can be documented here. See the rest of the pages in this section for things you can do with blocks.

Registering a Block

Blocks must be registered to function.

!!! important A block in the level and a “block” in an inventory are very different things. A block in the level is represented by an `BlockState`, and its behavior defined by an instance of `Block`. Meanwhile, an item in an inventory is an `ItemStack`, controlled by an `Item`. As a bridge between the different worlds of `Block` and `Item`, there exists the class `BlockItem`. `BlockItem` is a subclass of `Item` that has a field `block` that holds a reference to the `Block` it represents. `BlockItem` defines some of the behavior of a “block” as an item, like how a right click places the block. It’s possible to have a `Block` without an `BlockItem`. (E.g. `minecraft:water` exists a block, but not an item. It is therefore impossible to hold it in an inventory as one.)

When a block is registered, *only* a block is registered. The block does not automatically have an `BlockI`

Optionally Registering Blocks In the past there have been several mods that have allowed users to disable blocks/items in a configuration file. However, you shouldn't do this. There is no limit on the amount of blocks that can be register, so register all blocks in your mod! If you want a block to be disabled through a configuration file, you should disable the crafting recipe. If you would like to disable the block in the creative tab, use a `FeatureFlag` when building the contents within `BuildCreativeModeTabContentsEvent`.

Further Reading

For information about block properties, such as those used for vanilla blocks like fences, walls, and many more, see the section on blockstates.

Block States

Legacy Behavior

In Minecraft 1.7 and previous versions, blocks which need to store placement or state data that did not have `BlockEntities` used **metadata**. Metadata was an extra number stored with the block, allowing different rotations, facings, or even completely separate behaviors within a block.

However, the metadata system was confusing and limited, since it was stored as only a number alongside the block ID, and had no meaning except what was commented in the code. For example, to implement a block that can face a direction and be on either the upper or lower half of a block space (such as a stair):

```
switch (meta) {
    case 0: { ... } // south and on the lower half of the block
    case 1: { ... } // south on the upper side of the block
    case 2: { ... } // north and on the lower half of the block
    case 3: { ... } // north and on the upper half of the block
    // ... etc. ...
}
```

Because the numbers carry no meaning by themselves, no one could know what they represent unless they had access to the source code and comments.

Introduction of States

In Minecraft 1.8 and above, the metadata system, along with the block ID system, was deprecated and eventually replaced with the **block state system**. The block state system abstracts out the details of the block's properties from the other behaviors of the block.

Each *property* of a block is described by an instance of `Property<?>`. Examples of block properties include instruments (`EnumProperty<NoteBlockInstrument>`), facing (`DirectionProperty`), poweredness (`Property<Boolean>`), etc. Each property has the value of the type `T` parametrized by `Property<T>`.

A unique pair can be constructed from the `Block` and a map of the `Property<?>` to their associated values. This unique pair is called a `BlockState`.

The previous system of meaningless metadata values were replaced by a system of block properties, which are easier to interpret and deal with. Previously, a stone button which is facing east and is powered or held down is represented by "minecraft:stone_button with metadata 9. Now, this is represented by "minecraft:stone_button[facing=east,powered=true]".

Proper Usage of Block States

The `BlockState` system is a flexible and powerful system, but it also has limitations. `BlockStates` are immutable, and all combinations of their properties are generated on startup of the game. This means that having a `BlockState` with many properties and possible values will slow down the loading of the game, and befuddle anyone trying to make sense of your block logic.

Not all blocks and situations require the usage of `BlockState`; only the most basic properties of a block should be put into a `BlockState`, and any other situation is better off with having a `BlockEntity` or being a separate `Block`.

Always consider if you actually need to use blockstates for your purposes.

!!! note A good rule of thumb is: **if it has a different name, it should be a separate block.**

An example is making chair blocks: the *direction* of the chair should be a *property*, while the different *types of wood* should be separated into different blocks. An “Oak Chair” facing east (`oak_chair[facing=east]`) is different from a “Spruce Chair” facing west (`spruce_chair[facing=west]`).

Implementing Block States

In your Block class, create or reference `static final Property<?>` objects for every property that your Block has. You are free to make your own `Property<?>` implementations, but the means to do that are not covered in this article. The vanilla code provides several convenience implementations:

- `IntegerProperty`
 - Implements `Property<Integer>`. Defines a property that holds an integer value.
 - Created by calling `IntegerProperty#create(String propertyName, int minimum, int maximum)`.
- `BooleanProperty`
 - Implements `Property<Boolean>`. Defines a property that holds a `true` or `false` value.
 - Created by calling `BooleanProperty#create(String propertyName)`.
- `EnumProperty<E extends Enum<E>>`
 - Implements `Property<E>`. Defines a property that can take on the values of an Enum class.
 - Created by calling `EnumProperty#create(String propertyName, Class<E> enumClass)`.
 - It is also possible to use only a subset of the Enum values (e.g. 4 out of 16 `DyeColors`). See the overloads of `EnumProperty#create`.
- `DirectionProperty`
 - This is a convenience implementation of `EnumProperty<Direction>`
 - Several convenience predicates are also provided. For example, to get a property that represents the cardinal directions, call `DirectionProperty.create("<name>", Direction.Plane.HORIZONTAL)`; to get the X directions, `DirectionProperty.create("<name>", Direction.Axis.X)`.

The class `BlockStateProperties` contains shared vanilla properties which should be used or referenced whenever possible, in place of creating your own properties.

When you have your desired `Property<>` objects, override `Block#createBlockStateDefinition(StateDefinition$Builder)` in your Block class. In that method, call `StateDefinition$Builder#add(...)`; with the parameters as every `Property<?>` you wish the block to have.

Every block will also have a “default” state that is automatically chosen for you. You can change this “default” state by calling the `Block#registerDefaultState(BlockState)` method from your constructor. When your block is placed it will become this “default” state. An example from `DoorBlock`:

```
this.registerDefaultState(
    this.stateDefinition.any()
        .setValue(FACING, Direction.NORTH)
        .setValue(OPEN, false)
        .setValue(HINGE, DoorHingeSide.LEFT)
        .setValue(POWERED, false)
        .setValue(HALF, DoubleBlockHalf.LOWER)
);
```

If you wish to change what `BlockState` is used when placing your block, you can overwrite `Block#getStateForPlacement(BlockPlaceContext)`. This can be used to, for example, set the direction of your block depending on where the player is standing when they place it.

Because `BlockStates` are immutable, and all combinations of their properties are generated on startup of the game, calling `BlockState#setValue(Property<T>, T)` will simply go to the Block’s `StateHolder` and request the `BlockState` with the set of values you want.

Because all possible `BlockStates` are generated at startup, you are free and encouraged to use the reference equality operator (`==`) to check if two `BlockStates` are equal.

Using BlockState's

You can get the value of a property by calling `BlockState#getValue(Property<?>)`, passing it the property you want to get the value of. If you want to get a `BlockState` with a different set of values, simply call `BlockState#setValue(Property<T>, T)` with the property and its value.

You can get and place `BlockState`'s in the level using `Level#setBlockAndUpdate(BlockPos, BlockState)` and `Level#getBlockState(BlockPos)`. If you are placing a `Block`, call `Block#defaultBlockState()` to get the "default" state, and use subsequent calls to `BlockState#setValue(Property<T>, T)` as stated above to achieve the desired state.

BlockEntities

`BlockEntities` are like simplified `Entities` that are bound to a `Block`. They are used to store dynamic data, execute tick based tasks, and dynamic rendering. Some examples from vanilla Minecraft would be handling of inventories on chests, smelting logic on furnaces, or area effects on beacons. More advanced examples exist in mods, such as quarries, sorting machines, pipes, and displays.

!!! note `BlockEntities` aren't a solution for everything and they can cause lag when used wrongly. When possible, try to avoid them.

Registering

`Block Entities` are created and removed dynamically and as such are not registry objects on their own.

In order to create a `BlockEntity`, you need to extend the `BlockEntity` class. As such, another object is registered instead to easily create and refer to the *type* of the dynamic object. For a `BlockEntity`, these are known as `BlockEntityType`s.

A `BlockEntityType` can be registered like any other registry object. To construct a `BlockEntityType`, its builder form can be used via `BlockEntityType$Builder#of`. This takes in two arguments: a `BlockEntityType$BlockEntitySupplier` which takes in a `BlockPos` and `BlockState` to create a new instance of the associated `BlockEntity`, and a varargs of `Blocks` which this `BlockEntity` can be attached to. Building the `BlockEntityType` is done by calling `BlockEntityType$Builder#build`. This takes in a `Type` which represents the type-safe reference used to refer to this registry object in a `DataFixer`. Since `DataFixers` are an optional system to use for mods, this can be passed as `null`.

```
// For some DeferredRegister<BlockEntityType<?>> REGISTER
public static final RegistryObject<BlockEntityType<MyBE>> MY_BE = REGISTER.register("mybe", () ->
BlockEntityType.Builder.of(MyBE::new, validBlocks).build(null));

// In MyBE, a BlockEntity subclass
public MyBE(BlockPos pos, BlockState state) {
    super(MY_BE.get(), pos, state);
}
```

Creating a BlockEntity

To create a `BlockEntity` and attach it to a `Block`, the `EntityBlock` interface must be implemented on your `Block` subclass. The method `EntityBlock#newBlockEntity(BlockPos, BlockState)` must be implemented and return a new instance of your `BlockEntity`.

Storing Data within your BlockEntity

In order to save data, override the following two methods:

```
BlockEntity#saveAdditional(CompoundTag tag)

BlockEntity#load(CompoundTag tag)
```

These methods are called whenever the `LevelChunk` containing the `BlockEntity` gets loaded from/saved to a tag. Use them to read and write to the fields in your block entity class.

!!! note Whenever your data changes, you need to call `BlockEntity#setChanged`; otherwise, the `LevelChunk` containing your `BlockEntity` might be skipped while the level is saved.

!!! important It is important that you call the `super` methods!

The tag names ``id``, ``x``, ``y``, ``z``, ``ForgeData`` and ``ForgeCaps`` are reserved by the ``super`` methods.

Ticking BlockEntities

If you need a ticking `BlockEntity`, for example to keep track of the progress during a smelting process, another method must be implemented and overridden within `EntityBlock`: `EntityBlock#getTicker(Level, BlockState, BlockEntityType)`. This can implement different tickers depending on which logical side the user is on, or just implement one general ticker. In either case, a `BlockEntityTicker` must be returned. Since this is a functional interface, it can just take in a method representing the ticker instead:

```
// Inside some Block subclass
@Nullable
@Override
public <T extends BlockEntity> BlockEntityTicker<T> getTicker(Level level, BlockState state, BlockEntityType<T> type)
{
    return type == MyBlockEntityTypes.MYBE.get() ? MyBlockEntity::tick : null;
}

// Inside MyBlockEntity
public static void tick(Level level, BlockPos pos, BlockState state, MyBlockEntity blockEntity) {
    // Do stuff
}
```

!!! note This method is called each tick; therefore, you should avoid having complicated calculations in here. If possible, you should make more complex calculations every X ticks. (The amount of ticks in a second may be lower than 20 (twenty) but won't be higher)

Synchronizing the Data to the Client

There are three ways of syncing data to the client: synchronizing on chunk load, on block updates, and with a custom network message.

Synchronizing on LevelChunk Load

For this you need to override

```
BlockEntity#getUpdateTag()

IForgeBlockEntity#handleUpdateTag(CompoundTag tag)
```

Again, this is pretty simple, the first method collects the data that should be sent to the client, while the second one processes that data. If your `BlockEntity` doesn't contain much data, you might be able to use the methods out of the Storing Data within your `BlockEntity` section.

!!! important Synchronizing excessive/useless data for block entities can lead to network congestion. You should optimize your network usage by sending only the information the client needs when the client needs it. For instance, it is more often than not unnecessary to send the inventory of a block entity in the update tag, as this can be synchronized via its `AbstractContainerMenu`.

Synchronizing on Block Update

This method is a bit more complicated, but again you just need to override two or three methods. Here is a tiny example implementation of it:

```
@Override
public CompoundTag getUpdateTag() {
    CompoundTag tag = new CompoundTag();
    //Write your data into the tag
    return tag;
}
```

```

@Override
public Packet<ClientGamePacketListener> getUpdatePacket() {
    // Will get tag from #getUpdateTag
    return ClientboundBlockEntityDataPacket.create(this);
}

// Can override IForgeBlockEntity#onDataPacket. By default, this will defer to the #load.

```

The static constructors `ClientboundBlockEntityDataPacket#create` takes:

- The `BlockEntity`.
- An optional function to get the `CompoundTag` from the `BlockEntity`. By default, this uses `BlockEntity#getUpdateTag`.

Now, to send the packet, an update notification must be given on the server.

```

Level#sendBlockUpdated(BlockPos pos, BlockState oldState, BlockState newState, int flags)

```

The `pos` should be your `BlockEntity`'s position. For `oldState` and `newState`, you can pass the current `BlockState` at that position. `flags` is a bitmask that should contain 2, which will sync the changes to the client. See `Block` for more info as well as the rest of the flags. The flag 2 is equivalent to `Block#UPDATE_CLIENTS`.

Synchronizing Using a Custom Network Message

This way of synchronizing is probably the most complicated but is usually the most optimized, as you can make sure that only the data you need to be synchronized is actually synchronized. You should first check out the `Networking` section and especially `SimpleImpl` before attempting this. Once you've created your custom network message, you can send it to all users that have the `BlockEntity` loaded with `SimpleChannel#send(PacketDistributor$PacketTarget, MSG)`.

!!! warning It is important that you do safety checks, the `BlockEntity` might already be destroyed/replaced when the message arrives at the player! You should also check if the chunk is loaded (`Level#hasChunkAt(BlockPos)`).

BlockEntityRenderer

A `BlockEntityRenderer` or BER is used to render blocks in a way that cannot be represented with a static baked model (JSON, OBJ, B3D, others). A block entity renderer requires the block to have a `BlockEntity`.

Creating a BER

To create a BER, create a class that inherits from `BlockEntityRenderer`. It takes a generic argument specifying the block's `BlockEntity` class. The generic argument is used in the BER's `render` method.

Only one BER exists for a given `BlockEntityType`. Therefore, values that are specific to a single instance in the level should be stored in the block entity being passed to the renderer rather than in the BER itself. For example, an integer that increments every frame, if stored in the BER, will increment every frame for every block entity of this type in the level.

render

This method is called every frame in order to render the block entity.

Parameters

- `blockEntity`: This is the instance of the block entity being rendered.
- `partialTick`: The amount of time, in fractions of a tick, that has passed since the last full tick.
- `poseStack`: A stack holding four-dimensional matrix entries offset to the current position of the block entity.
- `bufferSource`: A rendering buffer able to access a vertex consumer.
- `combinedLight`: An integer of the current light value on the block entity.

- `combinedOverlay`: An integer set to the current overlay of the block entity, usually `OverlayTexture#NO_OVERLAY` or 655,360.

Registering a BER

In order to register a BER, you must subscribe to the `EntityRenderersEvent$RegisterRenderers` event on the mod event bus and call `#registerBlockEntityRenderer`.

Items

Along with blocks, items are a key component of most mods. While blocks make up the level around you, items exist within inventories.

Creating an Item

Basic Items

Basic items that need no special functionality (think sticks or sugar) do not need custom classes. You can create an item by instantiating the `Item` class with an `Item$Properties` object. This `Item$Properties` object can be made via the constructor and customized by calling its methods. For instance:

Method	Description
<code>requiredFeatures</code>	Sets the required <code>FeatureFlags</code> needed to see this item in the <code>CreativeModeTab</code> it is added to.
<code>durability</code>	Sets the maximum damage value for this item. If it is over 0, two item properties “damaged” and “damage” are added.
<code>stacksTo</code>	Sets the maximum stack size. You cannot have an item that is both damageable and stackable.
<code>setNoRepair</code>	Makes this item impossible to repair, even if it is damageable.
<code>craftRemainder</code>	Sets this item’s container item, the way that lava buckets give you back an empty bucket when they are used.

The above methods are chainable, meaning they `return this` to facilitate calling them in series.

Advanced Items

Setting the properties of an item as above only works for simple items. If you want more complicated items, you should subclass `Item` and override its methods.

Creative Tabs

An item can be added to a `CreativeModeTab` via `BuildCreativeModeTabContentsEvent` on the mod event bus. An item(s) can be added without any additional configurations via `#accept`.

```
// Registered on the MOD event bus
// Assume we have RegistryObject<Item> and RegistryObject<Block> called ITEM and BLOCK
@SubscribeEvent
public void buildContents(BuildCreativeModeTabContentsEvent event) {
    // Add to ingredients tab
    if (event.getTabKey() == CreativeModeTabs.INGREDIENTS) {
        event.accept(ITEM);
        event.accept(BLOCK); // Takes in an ItemLike, assumes block has registered item
    }
}
```

You can also enable or disable items being added through a `FeatureFlag` in the `FeatureFlagSet` or a boolean determining whether the player has permissions to see operator creative tabs.

Custom Creative Tabs

A custom `CreativeModeTab` must be registered. The builder can be created via `CreativeModeTab#builder`. The tab can set the title, icon, default items, and a number of other properties. In addition, Forge provides additional methods to customize the tab's image, label and slot colors, where the tab should be ordered, etc.

```
// Assume we have a DeferredRegister<CreativeModeTab> called REGISTRAR
// Assume we have RegistryObject<Item> and RegistryObject<Block> called ITEM and BLOCK
public static final RegistryObject<CreativeModeTab> EXAMPLE_TAB = REGISTRAR.register("example", () ->
CreativeModeTab.builder()
    // Set name of tab to display
    .title(Component.translatable("item_group." + MOD_ID + ".example"))
    // Set icon of creative tab
    .icon(() -> new ItemStack(ITEM.get()))
    // Add default items to tab
    .displayItems((params, output) -> {
        output.accept(ITEM.get());
        output.accept(BLOCK.get());
    })
    .build()
);
```

Registering an Item

Items must be registered to function.

BlockEntityWithoutLevelRenderer

`BlockEntityWithoutLevelRenderer` is a method to handle dynamic rendering on items. This system is much simpler than the old `ItemStack` system, which required a `BlockEntity`, and did not allow access to the `ItemStack`.

Using BlockEntityWithoutLevelRenderer

`BlockEntityWithoutLevelRenderer` allows you to render your item using `public void renderByItem(ItemStack itemStack, ItemDisplayContext ctx, PoseStack poseStack, MultiBufferSource bufferSource, int combinedLight, int combinedOverlay)`.

In order to use an BEWLR, the `Item` must first satisfy the condition that its model returns true for `BakedModel#isCustomRenderer`. If it does not have one, it will use the default `ItemRenderer#getBlockEntityRenderer`. Once that returns true, the `Item`'s BEWLR will be accessed for rendering.

!!! note Blocks also render using a BEWLR if `Block#getRenderShape` is set to `RenderShape#ENTITYBLOCK_ANIMATED`.

To set the BEWLR for an `Item`, an anonymous instance of `IClientItemExtensions` must be consumed within `Item#initializeClient`. Within the anonymous instance, `IClientItemExtensions#getCustomRenderer` should be overridden to return the instance of your BEWLR:

```
// In your item class
@Override
public void initializeClient(Consumer<IClientItemExtensions> consumer) {
    consumer.accept(new IClientItemExtensions() {

        @Override
        public BlockEntityWithoutLevelRenderer getCustomRenderer() {
            return myBEWLRInstance;
        }
    });
}
```

!!! important Each mod should only have one instance of a custom BEWLR.

That is it, no additional setup is necessary to use a BEWLR.

Networking

Communication between servers and clients is the backbone of a successful mod implementation.

There are two primary goals in network communication:

1. Making sure the client view is “in sync” with the server view
 - The flower at coordinates (X, Y, Z) just grew
2. Giving the client a way to tell the server that something has changed about the player
 - the player pressed a key

The most common way to accomplish these goals is to pass messages between the client and the server. These messages will usually be structured, containing data in a particular arrangement, for easy sending and receiving.

There are a variety of techniques provided by Forge to facilitate communication mostly built on top of netty.

The simplest, for a new mod, would be `SimpleImpl`, where most of the complexity of the netty system is abstracted away. It uses a message and handler style system.

SimpleImpl

`SimpleImpl` is the name given to the packet system that revolves around the `SimpleChannel` class. Using this system is by far the easiest way to send custom data between clients and the server.

Getting Started

First you need to create your `SimpleChannel` object. We recommend that you do this in a separate class, possibly something like `ModidPacketHandler`. Create your `SimpleChannel` as a static field in this class, like so:

```
private static final String PROTOCOL_VERSION = "1";
public static final SimpleChannel INSTANCE = NetworkRegistry.newSimpleChannel(
    new ResourceLocation("mymodid", "main"),
    () -> PROTOCOL_VERSION,
    PROTOCOL_VERSION::equals,
    PROTOCOL_VERSION::equals
);
```

The first argument is a name for the channel. The second argument is a `Supplier<String>` returning the current network protocol version. The third and fourth arguments respectively are `Predicate<String>` checking whether an incoming connection protocol version is network-compatible with the client or server, respectively. Here, we simply compare with the `PROTOCOL_VERSION` field directly, meaning that the client and server `PROTOCOL_VERSION`s must always match or FML will deny login.

The Version Checker

If your mod does not require the other side to have a specific network channel, or to be a Forge instance at all, you should take care that you properly define your version compatibility checkers (the `Predicate<String>` parameters) to handle additional “meta-versions” (defined in `NetworkRegistry`) that can be received by the version checker. These are:

- **ABSENT** - if this channel is missing on the other endpoint. Note that in this case, the endpoint is still a Forge endpoint, and may have other mods.
- **ACCEPTVANILLA** - if the endpoint is a vanilla (or non-Forge) endpoint.

Returning `false` for both means that this channel must be present on the other endpoint. If you just copy the code above, this is what it does. Note that these values are also used during the list ping compatibility check, which is responsible for showing the green check / red cross in the multiplayer server select screen.

Registering Packets

Next, we must declare the types of messages that we would like to send and receive. This is done using `INSTANCE#registerMessage`, which takes 5 parameters:

- The first parameter is the discriminator for the packet. This is a per-channel unique ID for the packet. We recommend you use a local variable to hold the ID, and then call `registerMessage` using `id++`. This will guarantee 100% unique IDs.
- The second parameter is the actual packet class `MSG`.
- The third parameter is a `BiConsumer<MSG, FriendlyByteBuf>` responsible for encoding the message into the provided `FriendlyByteBuf`.
- The fourth parameter is a `Function<FriendlyByteBuf, MSG>` responsible for decoding the message from the provided `FriendlyByteBuf`.
- The final parameter is a `BiConsumer<MSG, Supplier<NetworkEvent.Context>>` responsible for handling the message itself.

The last three parameters can be method references to either static or instance methods in Java. Remember that an instance method `MSG#encode(FriendlyByteBuf)` still satisfies `BiConsumer<MSG, FriendlyByteBuf>`; the `MSG` simply becomes the implicit first argument.

Handling Packets

There are a couple things to highlight in a packet handler. A packet handler has both the message object and the network context available to it. The context allows access to the player that sent the packet (if on the server), and a way to enqueue thread-safe work.

```
public static void handle(MyMessage msg, Supplier<NetworkEvent.Context> ctx) {
    ctx.get().enqueueWork(() -> {
        // Work that needs to be thread-safe (most work)
        ServerPlayer sender = ctx.get().getSender(); // the client that sent this packet
        // Do stuff
    });
    ctx.get().setPacketHandled(true);
}
```

Packets sent from the server to the client should be handled in another class and wrapped via `DistExecutor#unsafeRunWhenOn`.

```
// In Packet class
public static void handle(MyClientMessage msg, Supplier<NetworkEvent.Context> ctx) {
    ctx.get().enqueueWork(() -> {
        // Make sure it's only executed on the physical client
        DistExecutor.unsafeRunWhenOn(Dist.CLIENT, () -> () -> ClientPacketHandlerClass.handlePacket(msg, ctx))
    });
    ctx.get().setPacketHandled(true);
}

// In ClientPacketHandlerClass
public static void handlePacket(MyClientMessage msg, Supplier<NetworkEvent.Context> ctx) {
    // Do stuff
}
```

Note the presence of `#setPacketHandled`, which is used to tell the network system that the packet has successfully completed handling.

!!! warning As of Minecraft 1.8 packets are by default handled on the network thread.

That means that your handler can `_not_` interact with most game objects directly. Forge provides a convenience

!!! warning Be defensive when handling packets on the server. A client could attempt to exploit the packet handling by sending unexpected data.

A common problem is vulnerability to **arbitrary chunk generation**. This typically happens when the server

To avoid this problem, a general rule of thumb is to only access blocks and block entities if `Level#hasChunk`

Sending Packets

Sending to the Server

There is but one way to send a packet to the server. This is because there is only ever *one* server the client can be connected to at once. To do so, we must again use that `SimpleChannel` that was defined earlier. Simply call `INSTANCE.sendToServer(new MyMessage())`. The message will be sent to the handler for its type, if one exists.

Sending to Clients

Packets can be sent directly to a client using the `SimpleChannel`: `HANDLER.sendTo(new MyClientMessage(), serverPlayer.connection.getConnection(), NetworkDirection.PLAY_TO_CLIENT)`. However, this can be quite inconvenient. Forge has some convenience functions that can be used:

```
// Send to one player
INSTANCE.send(PacketDistributor.PLAYER.with(serverPlayer), new MyMessage());

// Send to all players tracking this level chunk
INSTANCE.send(PacketDistributor.TRACKING_CHUNK.with(levelChunk), new MyMessage());

// Send to all connected players
INSTANCE.send(PacketDistributor.ALL.noArg(), new MyMessage());
```

There are additional `PacketDistributor` types available; check the documentation on the `PacketDistributor` class for more details.

Entities

In addition to regular network messages, there are various other systems provided to handle synchronizing entity data.

Spawn Data

In general, the spawning of modded entities is handled separately, by Forge.

!!! note This means that simply extending a vanilla entity class may not inherit all its behavior. You may need to implement certain vanilla behaviors yourself.

You can add extra data to the spawn packet Forge sends by implementing the following interface.

IEntityAdditionalSpawnData

If your entity has data that is needed on the client, but does not change over time, then it can be added to the entity spawn packet using this interface. `#writeSpawnData` and `#readSpawnData` control how the data should be encoded to/decoded from the network buffer.

Dynamic Data

Data Parameters

This is the main vanilla system for synchronizing entity data from the server to the client. As such, a number of vanilla examples are available to refer to.

Firstly, you need a `EntityDataAccessor<T>` for the data you wish to keep synchronized. This should be stored as a `static final` field in your entity class, obtained by calling `SynchedEntityData#defineId` and passing the entity class and a serializer for that type of data. The available serializer implementations can be found as static constants within the `EntityDataSerializers` class.

!!! warning You should **only** create data parameters for your own entities, *within that entity's class*. Adding parameters to entities you do not control can cause the IDs used to send that data over the network to become desynchronized, causing difficult to debug crashes.

Then, override `Entity#defineSynchedData` and call `this.entityData.define(...)` for each of your data parameters, passing the parameter and an initial value to use. Remember to always call the `super` method first!

You can then get and set these values via your entity's `entityData` instance. Changes made will be synchronized to the client automatically.

Particles

Particles are an effect within the game used as polish to better improve immersion. Their usefulness also requires great caution because of their methods of creation and reference.

Creating a Particle

Particles are broken up between its **client only** implementation to display the particle and its common implementation to reference the particle or sync data from the server.

Class	Side	Description
ParticleType	BOTH	The registry object of a particle's type definition used to reference the particle on either side
ParticleOptions	BOTH	A data holder used to sync information from the network or a command to the associated client(s)
ParticleProvider	CLIENT	A factory registered by the ParticleType used to construct a Particle from the associated ParticleOptions .
Particle	CLIENT	The renderable logic to display on the associated client(s)

ParticleType

A **ParticleType** is the registry object defining what a particular particle type is and provides an available reference to the specific particle on both sides. As such, every **ParticleType** must be registered.

Each **ParticleType** takes in two parameters: an `overrideLimiter` which determines whether the particle renders regardless of distance, and a `ParticleOptions$Deserializer` which is used to read the sent **ParticleOptions** on the client. As the base **ParticleType** is abstract, a single method needs to be implemented: `#codec`. This represents how to encode and decode the associated **ParticleOptions** of the type.

!!! note **ParticleType#codec** is only used within the biome codec for vanilla implementations.

In most cases, there is no need to have any particle data sent to the client. For these instances, it is easier to create a new instance of **SimpleParticleType**: an implementation of **ParticleType** and **ParticleOptions** which does not send any custom data to the client besides the type. Most vanilla implementations use **SimpleParticleType** besides redstone dust for coloring and block/item dependent particles.

!!! important A **ParticleType** is not needed to make a particle spawn if only referenced on the client. However, it is necessary to use any of the prebuilt logic within **ParticleEngine** or spawn a particle from the server.

ParticleOptions

An **ParticleOptions** represents the data that each particle takes in. It is also used to send data from particles spawned via the server. All particle spawning methods take in a **ParticleOptions** such that it knows the type of the particle and the data associated with spawning one.

ParticleOptions is broken down into three methods:

Method	Description
<code>getType</code>	Gets the type definition of the particle, or the ParticleType

Method	Description
writeToNetwork	Writes the particle data to a buffer on the server to send to the client
writeToString	Writes the particle data to a string

These objects are either constructed on the fly as needed, or they are singletons as a result of being a `SimpleParticleType`.

ParticleOptions\$Deserializer To receive the `ParticleOptions` on the client, or to reference the data within a command, the particle data must be deserialized via `ParticleOptions$Deserializer`. Each method within `ParticleOptions$Deserializer` has a parity encoding method within `ParticleOptions`:

Method	ParticleOptions Encoder	Description
fromCommand	writeToString	Decodes a particle data from a string, usually from a command.
fromNetwork	writeToNetwork	Decodes a particle data from a buffer on the client.

This object, when needing to send custom particle data, is passed into the constructor of the `ParticleType`.

Particle

A `Particle` provides the rendering logic needed to draw said data onto the screen. To create any `Particle`, two methods must be implemented:

Method	Description
render	Renders the particle onto the screen.
getRenderType	Gets the render type of the particle.

A common subclass of `Particle` to render textures is `TextureSheetParticle`. While `#getRenderType` needs to be implemented, whatever the texture sprite is set will be rendered at the particle's location.

ParticleRenderType `ParticleRenderType` is a variation on `RenderType` which constructs the startup and teardown phase for every particle of that type and then renders them all at once via the `Tesselator`. There are six different render types a particle can be in.

Render Type	Description
TERRAIN_SHEET	Renders a particle whose texture is located within the available blocks.
PARTICLE_SHEET_OPAQUE	Renders a particle whose texture is opaque and located within the available particles.
PARTICLE_SHEET_TRANSLUCENT	Renders a particle whose texture is translucent and located within the available particles.
PARTICLE_SHEET_LIT	Same as <code>PARTICLE_SHEET_OPAQUE</code> except without using the particle shader.
CUSTOM	Provides setup for blending and depth mask but provides no rendering functionality as that would be implemented within <code>Particle#render</code> .
NO_RENDER	The particle will never render.

Implementing a custom render type will be left as an exercise to the reader.

ParticleProvider

Finally, a particle is usually created via an **ParticleProvider**. A factory has a single method **#createParticle** which is used to create a particle given the particle data, client level, position, and movement delta. Since a **Particle** is not beholden to any particular **ParticleType**, it can be reused in different factories as necessary.

An **ParticleProvider** must be registered by subscribing to the **RegisterParticleProvidersEvent** on the **mod event bus**. Within the event, the factory can be registered via **#registerSpecial** by supplying an instance of the factory to the method.

!!! important **RegisterParticleProvidersEvent** should only be called on the client and thus sided off in some isolated client class, referenced by either **DistExecutor** or **@EventBusSubscriber**.

ParticleDescription, SpriteSet, and SpriteParticleRegistration There are three particle render types that cannot use the above method of registration: **PARTICLE_SHEET_OPAQUE**, **PARTICLE_SHEET_TRANSLUCENT**, and **PARTICLE_SHEET_LIT**. This is because all three of these particle render types use a sprite set that is loaded by the **ParticleEngine** directly. As such, the textures supplied must be obtained and registered through a different method. This will assume your particle is a subtype of **TextureSheetParticle** as that is the only vanilla implementation for this logic.

To add a texture to a particle, a new JSON file must be added to **assets/<modid>/particles**. This is known as the **ParticleDescription**. The name of this file will represent the registry name of the **ParticleType** the factory is being attached to. Each particle JSON is an object. The object stores a single key **textures** which holds an array of **ResourceLocations**. Any **<modid>:<path>** texture represented here will point to a texture at **assets/<modid>/textures/particle/<path>.png**.

```
{
  "textures": [
    // Will point to a texture located in
    // assets/mymod/textures/particle/particle_texture.png
    "mymod:particle_texture",
    // Textures should be ordered by drawing order
    // e.g. particle_texture will render first, then particle_texture2
    // after some time
    "mymod:particle_texture2"
  ]
}
```

To reference a particle texture, the subtype of **TextureSheetParticle** should either take in an **SpriteSet** or a **TextureAtlasSprite** obtained from **SpriteSet**. **SpriteSet** holds a list of textures which refer to the sprites as defined by our **ParticleDescription**. **SpriteSet** has two methods, both of which grab a **TextureAtlasSprite** in different methods. The first method takes in two integers. The backing implementation allows the sprite to have a texture change as it ages. The second method takes in a **Random** instance to get a random texture from the sprite set. The sprite can be set within **TextureSheetParticle** by using one of the helper methods that takes in the **SpriteSet**: **#pickSprite** which uses the random method of picking a texture, and **#setSpriteFromAge** which uses the percentage method of two integers to pick the texture.

To register these particle textures, a **SpriteParticleRegistration** needs to be supplied to the **RegisterParticleProvidersEvent#registerSpriteSet** method. This method takes in an **SpriteSet** holding the associated sprite set for the particle and creates an **ParticleProvider** to create the particle. The simplest method of implementation can be done by implementing **ParticleProvider** on some class and having the constructor take in an **SpriteSet**. Then the **SpriteSet** can be passed to the particle as normal.

!!! note If you are registering a **TextureSheetParticle** subtype which only contains one texture, then you can supply a **ParticleProvider\$Sprite** instead to the **#registerSprite** method, which has essentially the same functional interface method as **ParticleProvider**.

Spawning a Particle

Particles can be spawned from either level instance. However, each side has a specific way to spawn a particle. If on the **ClientLevel**, **#addParticle** can be called to spawn a particle or **#addAlwaysVisibleParticle** can be called to spawn a particle that is visible from any distance. If on the **ServerLevel**, **#sendParticles** can be called to

send a packet to the client to spawn the particle. Calling the two `ClientLevel` methods on the server will result in nothing.

Sounds

Terminology

Term	Description
Sound Events	Something that triggers a sound effect. Examples include <code>minecraft:block.anvil.hit</code> or <code>botania:spreader_fire</code> .
Sound Category	The category of the sound, for example <code>player</code> , <code>block</code> or simply <code>master</code> . The sliders in the sound settings GUI represent these categories.
Sound File	The literal file on disk that is played: an <code>.ogg</code> file.

`sounds.json`

This JSON defines sound events, and defines which sound files they play, the subtitle, etc. Sound events are identified with `ResourceLocations`. `sounds.json` should be located at the root of a resource namespace (`assets/<namespace>/sounds.json`), and it defines sound events in that namespace (`assets/<namespace>/sounds.json` defines sound events in the namespace `namespace`.).

A full specification is available on the vanilla wiki, but this example highlights the important parts:

```
{
  "open_chest": {
    "subtitle": "mymod.subtitle.open_chest",
    "sounds": [ "mymod:open_chest_sound_file" ]
  },
  "epic_music": {
    "sounds": [
      {
        "name": "mymod:music/epic_music",
        "stream": true
      }
    ]
  }
}
```

Underneath the top-level object, each key corresponds to a sound event. Note that the namespace is not given, as it is taken from the namespace of the JSON itself. Each event specifies a localization key to be shown when subtitles are enabled. Finally, the actual sound files to be played are specified. Note that the value is an array; if multiple sound files are specified, the game will randomly choose one to play whenever the sound event is triggered.

The two examples represent two different ways to specify a sound file. The wiki has precise details, but generally, long sound files such as background music or music discs should use the second form, because the “stream” argument tells Minecraft to not load the entire sound file into memory but to stream it from disk. The second form can also specify the volume, pitch, and weight of a sound file.

In all cases, the path to a sound file for namespace `namespace` and path `path` is `assets/<namespace>/sounds/<path>.ogg`. Therefore `mymod:open_chest_sound_file` points to `assets/mymod/sounds/open_chest_sound_file.ogg`, and `mymod:music/epic_music` points to `assets/mymod/sounds/music/epic_music.ogg`.

A `sounds.json` can be data generated.

Creating Sound Events

In order to reference sounds on the server, a `SoundEvent` holding a corresponding entry in `sounds.json` must be created. This `SoundEvent` must then be registered. Normally, the location used to create a sound event should be set as its registry name.

The `SoundEvent` acts as a reference to the sound and is passed around to play them. If a mod has an API, it should expose its `SoundEvents` in the API.

!!! note As long as a sound is registered within the `sounds.json`, it can still be referenced on the logical client regardless of whether there is a referencing `SoundEvent`.

Playing Sounds

Vanilla has lots of methods for playing sounds, and it is unclear which to use at times.

Note that each takes a `SoundEvent`, the ones registered above. Additionally, the terms “*Server Behavior*” and “*Client Behavior*” refer to the respective **logical** side.

Level

1. `playSound(Player, BlockPos, SoundEvent, SoundSource, volume, pitch)`
 - Simply forwards to overload (2), adding 0.5 to each coordinate of the `BlockPos` given.`playSound(Player, double x, double y, double z, SoundEvent, SoundSource, volume, pitch)`
2.
 - **Client Behavior:** If the passed in player is *the* client player, plays the sound event to the client player.
 - **Server Behavior:** Plays the sound event to everyone nearby **except** the passed in player. Player can be null.
 - **Usage:** The correspondence between the behaviors implies that these two methods are to be called from some player-initiated code that will be run on both logical sides at the same time: the logical client handles playing it to the user, and the logical server handles everyone else hearing it without re-playing it to the original user. They can also be used to play any sound in general at any position server-side by calling it on the logical server and passing in a null player, thus letting everyone hear it.
3. `playLocalSound(double x, double y, double z, SoundEvent, SoundSource, volume, pitch, distanceDelay)`
 - **Client Behavior:** Just plays the sound event in the client level. If `distanceDelay` is `true`, then delays the sound based on how far it is from the player.
 - **Server Behavior:** Does nothing.
 - **Usage:** This method only works client-side, and thus is useful for sounds sent in custom packets, or other client-only effect-type sounds. Used for thunder.

ClientLevel

1. `playLocalSound(BlockPos, SoundEvent, SoundSource, volume, pitch, distanceDelay)`
 - Simply forwards to `Level`'s overload (3), adding 0.5 to each coordinate of the `BlockPos` given.

Entity

1. `playSound(SoundEvent, volume, pitch)`
 - Forwards to `Level`'s overload (2), passing in `null` as the player.
 - **Client Behavior:** Does nothing.
 - **Server Behavior:** Plays the sound event to everyone at this entity's position.
 - **Usage:** Emitting any sound from any non-player entity server-side.

Player

1. `playSound(SoundEvent, volume, pitch)` (overriding the one in `Entity`)
 - Forwards to `Level`'s overload (2), passing in `this` as the player.
 - **Client Behavior:** Does nothing, see override in `LocalPlayer`.
 - **Server Behavior:** Plays the sound to everyone nearby *except* this player.
 - **Usage:** See `LocalPlayer`.

LocalPlayer

1. `playSound(SoundEvent, volume, pitch)` (overriding the one in `Player`)
 - Forwards to `Level`'s overload (2), passing in `this` as the player.

- **Client Behavior:** Just plays the Sound Event.
- **Server Behavior:** Method is client-only.
- **Usage:** Just like the ones in `Level`, these two overrides in the player classes seem to be for code that runs together on both sides. The client handles playing the sound to the user, while the server handles everyone else hearing it without re-playing to the original user.

The Capability System

Capabilities allow exposing features in a dynamic and flexible way without having to resort to directly implementing many interfaces.

In general terms, each capability provides a feature in the form of an interface.

Forge adds capability support to `BlockEntities`, `Entities`, `ItemStacks`, `Levels`, and `LevelChunks`, which can be exposed either by attaching them through an event or by overriding the capability methods in your own implementations of the objects. This will be explained in more detail in the following sections.

Forge-provided Capabilities

Forge provides three capabilities: `IItemHandler`, `IFluidHandler` and `IEnergyStorage`

`IItemHandler` exposes an interface for handling inventory slots. It can be applied to `BlockEntities` (chests, machines, etc.), `Entities` (extra player slots, mob/creature inventories/bags), or `ItemStacks` (portable backpacks and such). It replaces the old `Container` and `WorldlyContainer` with an automation-friendly system.

`IFluidHandler` exposes an interface for handling fluid inventories. It can also be applied to `BlockEntities`, `Entities`, or `ItemStacks`.

`IEnergyStorage` exposes an interface for handling energy containers. It can be applied to `BlockEntities`, `Entities`, or `ItemStacks`. It is based on the RedstoneFlux API by TeamCoFH.

Using an Existing Capability

As mentioned earlier, `BlockEntities`, `Entities`, and `ItemStacks` implement the capability provider feature through the `ICapabilityProvider` interface. This interface adds the method `#getCapability`, which can be used to query the capabilities present in the associated provider objects.

In order to obtain a capability, you will need to refer it by its unique instance. In the case of the `IItemHandler`, this capability is primarily stored in `ForgeCapabilities#ITEM_HANDLER`, but it is possible to get other instance references by using `CapabilityManager#get`

```
public static final Capability<IItemHandler> ITEM_HANDLER = CapabilityManager.get(new CapabilityToken<>());
```

When called, `CapabilityManager#get` provides a non-null capability for your associated type. The anonymous `CapabilityToken` allows Forge to keep a soft dependency system while still having the necessary generic information to get the correct capability.

!!! important Even if you have a non-null capability available to you at all times, it does not mean the capability itself is usable or registered yet. This can be checked via `Capability#isRegistered`.

The `#getCapability` method has a second parameter, of type `Direction`, which can be used to request the specific instance for that one face. If passed `null`, it can be assumed that the request comes either from within the block or from some place where the side has no meaning, such as a different dimension. In this case a general capability instance that does not care about sides will be requested instead. The return type of `#getCapability` will correspond to a `LazyOptional` of the type declared in the capability passed to the method. For the Item Handler capability, this is `LazyOptional<IItemHandler>`. If the capability is not available for a particular provider, it will return an empty `LazyOptional` instead.

Exposing a Capability

In order to expose a capability, you will first need an instance of the underlying capability type. Note that you should assign a separate instance to each object that keeps the capability, since the capability will most probably be tied to the containing object.

In the case of `IItemHandler`, the default implementation uses the `ItemStackHandler` class, which has an optional argument in the constructor, to specify a number of slots. However, relying on the existence of these default implementations should be avoided, as the purpose of the capability system is to prevent loading errors in contexts where the capability is not present, so instantiation should be protected behind a check testing if the capability has been registered (see the remarks about `CapabilityManager#get` in the previous section).

Once you have your own instance of the capability interface, you will want to notify users of the capability system that you expose this capability and provide a `LazyOptional` of the interface reference. This is done by overriding the `#getCapability` method, and comparing the capability instance with the capability you are exposing. If your machine has different slots based on which side is being queried, you can test this with the `side` parameter. For Entities and ItemStacks, this parameter can be ignored, but it is still possible to have side as a context, such as different armor slots on a player (`Direction#UP` exposing the player's helmet slot), or about the surrounding blocks in the inventory (`Direction#WEST` exposing the input slot of a furnace). Do not forget to fall back to `super`, otherwise existing attached capabilities will stop working.

Capabilities must be invalidated at the end of the provider's lifecycle via `LazyOptional#invalidate`. For owned BlockEntities and Entities, the `LazyOptional` can be invalidated within `#invalidateCaps`. For non-owned providers, a runnable supplying the invalidation should be passed into `AttachCapabilitiesEvent#addListener`.

```
// Somewhere in your BlockEntity subclass
LazyOptional<IItemHandler> inventoryHandlerLazyOptional;

// Supplied instance (e.g. () -> inventoryHandler)
// Ensure laziness as initialization should only happen when needed
inventoryHandlerLazyOptional = LazyOptional.of(inventoryHandlerSupplier);

@Override
public <T> LazyOptional<T> getCapability(Capability<T> cap, Direction side) {
    if (cap == ForgeCapabilities.ITEM_HANDLER) {
        return inventoryHandlerLazyOptional.cast();
    }
    return super.getCapability(cap, side);
}

@Override
public void invalidateCaps() {
    super.invalidateCaps();
    inventoryHandlerLazyOptional.invalidate();
}
```

!!! tip If only one capability is exposed on a given object, you can use `Capability#orEmpty` as an alternative to the if/else statement.

```
```java
@Override
public <T> LazyOptional<T> getCapability(Capability<T> cap, Direction side) {
 return ForgeCapabilities.ITEM_HANDLER.orEmpty(cap, inventoryHandlerLazyOptional);
}
```
```

Items are a special case since their capability providers are stored on an `ItemStack`. Instead, a provider should be attached through `Item#initCapabilities`. This should hold your capabilities for the lifecycle of the stack.

It is strongly suggested that direct checks in code are used to test for capabilities instead of attempting to rely on maps or other data structures, since capability tests can be done by many objects every tick, and they need to be as fast as possible in order to avoid slowing down the game.

Attaching Capabilities

As mentioned, attaching capabilities to existing providers, `Levels`, and `LevelChunks` can be done using `AttachCapabilitiesEvent`. The same event is used for all objects that can provide capabilities. `AttachCapabilitiesEvent` has 5 valid generic types providing the following events:

- `AttachCapabilitiesEvent<Entity>`: Fires only for entities.
- `AttachCapabilitiesEvent<BlockEntity>`: Fires only for block entities.
- `AttachCapabilitiesEvent<ItemStack>`: Fires only for item stacks.
- `AttachCapabilitiesEvent<Level>`: Fires only for levels.
- `AttachCapabilitiesEvent<LevelChunk>`: Fires only for level chunks.

The generic type cannot be more specific than the above types. For example: If you want to attach capabilities to `Player`, you have to subscribe to the `AttachCapabilitiesEvent<Entity>`, and then determine that the provided object is an `Player` before attaching the capability.

In all cases, the event has a method `#addCapability` which can be used to attach capabilities to the target object. Instead of adding capabilities themselves to the list, you add capability providers, which have the chance to return capabilities only from certain sides. While the provider only needs to implement `ICapabilityProvider`, if the capability needs to store data persistently, it is possible to implement `ICapabilitySerializable<T extends Tag>` which, on top of returning the capabilities, will provide tag save/load functions.

For information on how to implement `ICapabilityProvider`, refer to the `Exposing a Capability` section.

Creating Your Own Capability

A capability can be registered using one of two ways: `RegisterCapabilitiesEvent` or `@AutoRegisterCapability`.

RegisterCapabilitiesEvent

A capability can be registered using `RegisterCapabilitiesEvent` by supplying the class of the capability type to the `#register` method. The event is handled on the mod event bus.

```
@SubscribeEvent
public void registerCaps(RegisterCapabilitiesEvent event) {
    event.register(IExampleCapability.class);
}
```

@AutoRegisterCapability

A capability is registered using `@AutoRegisterCapability` by annotating the capability type.

```
@AutoRegisterCapability
public interface IExampleCapability {
    // ...
}
```

Persisting LevelChunk and BlockEntity capabilities

Unlike `Levels`, `Entities`, and `ItemStacks`, `LevelChunks` and `BlockEntities` are only written to disk when they have been marked as dirty. A capability implementation with persistent state for a `LevelChunk` or a `BlockEntity` should therefore ensure that whenever its state changes, its owner is marked as dirty.

`ItemStackHandler`, commonly used for inventories in `BlockEntities`, has an overridable method `void onContentsChanged(int slot)` designed to be used to mark the `BlockEntity` as dirty.

```
public class MyBlockEntity extends BlockEntity {

    private final IItemHandler inventory = new ItemStackHandler(...) {
        @Override
        protected void onContentsChanged(int slot) {
            super.onContentsChanged(slot);
            setChanged();
        }
    }
}
```

```
}  
  
// ...  
}
```

Synchronizing Data with Clients

By default, capability data is not sent to clients. In order to change this, the mods have to manage their own synchronization code using packets.

There are three different situations in which you may want to send synchronization packets, all of them optional:

1. When the entity spawns in the level, or the block is placed, you may want to share the initialization-assigned values with the clients.
2. When the stored data changes, you may want to notify some or all of the watching clients.
3. When a new client starts viewing the entity or block, you may want to notify it of the existing data.

Refer to the Networking page for more information on implementing network packets.

Persisting across Player Deaths

By default, the capability data does not persist on death. In order to change this, the data has to be manually copied when the player entity is cloned during the respawn process.

This can be done via `PlayerEvent$Clone` by reading the data from the original entity and assigning it to the new entity. In this event, the `#isWasDeath` method can be used to distinguish between respawning after death and returning from the End. This is important because the data will already exist when returning from the End, so care has to be taken to not duplicate values in this case.

Saved Data

The Saved Data (SD) system is an alternative to level capabilities that can attach data per level.

Declaration

Each SD implementation must subtype the `SavedData` class. There are two important methods to be aware of:

- **save**: Allows the implementation to write NBT data to the level.
- **setDirty**: A method that must be called after changing the data, to notify the game that there are changes that need to be written. If not called, **#save** will not get called and the existing data will persist.

Attaching to a Level

Any `SavedData` is loaded and/or attached to a level dynamically. As such, if one is never created on a level, then it will not exist.

`SavedData`s are created and loaded from the `DimensionDataStorage`, which can be accessed by either `ServerChunkCache#getDataStorage` or `ServerLevel#getDataStorage`. From there, you can get or create an instance of your SD by calling `DimensionDataStorage#computeIfAbsent`. This will attempt to get the current instance of the SD if present or create a new one and load all available data.

`DimensionDataStorage#computeIfAbsent` takes in three arguments: a function to load NBT data into a SD and return it, a supplier to construct a new instance of the SD, and the name of the `.dat` file stored within the `data` folder for the implemented level.

For example, if a SD was named “example” within the Nether, then a file would be created at `./<level_folder>/DIM-1/data/example.dat` and would be implemented like so:

```
// In some class  
public ExampleSavedData create() {  
    return new ExampleSavedData();  
}
```



```

}

public ExampleSavedData load(CompoundTag tag) {
    ExampleSavedData data = this.create();
    // Load saved data
    return data;
}

// In some method within the class
netherDataStorage.computeIfAbsent(this::load, this::create, "example");

```

To persist a SD across levels, a SD should be attached to the Overworld, which can be obtained from `MinecraftServer#overworld`. The Overworld is the only dimension that is never fully unloaded and as such makes it perfect to store multi-level data on.

Codecs

Codecs are a serialization tool from Mojang's DataFixerUpper used to describe how objects can be transformed between different formats, such as `JsonElements` for JSON and `Tags` for NBT.

Using Codecs

Codecs are primarily used to encode, or serialize, Java objects to some data format type and decode, or deserialize, formatted data objects back to its associated Java type. This is typically accomplished using `Codec#encodeStart` and `Codec#parse`, respectively.

DynamicOps

To determine what intermediate file format to encode and decode to, both `#encodeStart` and `#parse` require a `DynamicOps` instance to define the data within that format.

The DataFixerUpper library contains `JsonOps` to codec JSON data stored in Gson's `JsonElement` instances. `JsonOps` supports two versions of `JsonElement` serialization: `JsonOps#INSTANCE` which defines a standard JSON file, and `JsonOps#COMPRESSED` which allows data to be compressed into a single string.

```

// Let exampleCodec represent a Codec<ExampleJavaObject>
// Let exampleObject be a ExampleJavaObject
// Let exampleJson be a JsonElement

// Encode Java object to regular JsonElement
exampleCodec.encodeStart(JsonOps.INSTANCE, exampleObject);

// Encode Java object to compressed JsonElement
exampleCodec.encodeStart(JsonOps.COMPRESSED, exampleObject);

// Decode JsonElement into Java object
// Assume JsonElement was parsed normally
exampleCodec.parse(JsonOps.INSTANCE, exampleJson);

```

Minecraft also provides `NbtOps` to codec NBT data stored in `Tag` instances. This can be referenced using `NbtOps#INSTANCE`.

```

// Let exampleCodec represent a Codec<ExampleJavaObject>
// Let exampleObject be a ExampleJavaObject
// Let exampleNbt be a Tag

// Encode Java object to Tag
exampleCodec.encodeStart(JsonOps.INSTANCE, exampleObject);

// Decode Tag into Java object
exampleCodec.parse(JsonOps.INSTANCE, exampleNbt);

```

Format Conversion `DynamicOps` can also be used separately to convert between two different encoded formats. This can be done using `#convertTo` and supplying the `DynamicOps` format and the encoded object to convert.

```
// Convert Tag to JsonElement
// Let exampleTag be a Tag
JsonElement convertedJson = NbtOps.INSTANCE.convertTo(JsonOps.INSTANCE, exampleTag);
```

DataResult

Encoded or decoded data using codecs return a **DataResult** which holds the converted instance or some error data depending on whether the conversion was successful. When the conversion is successful, the **Optional** supplied by **#result** will contain the successfully converted object. If the conversion fails, the **Optional** supplied by **#error** will contain the **PartialResult**, which holds the error message and a partially converted object depending on the codec.

Additionally, there are many methods on **DataResult** that can be used to transform the result or error into the desired format. For example, **#resultOrPartial** will return an **Optional** containing the result on success, and the partially converted object on failure. The method takes in a string consumer to determine how to report the error message if present.

```
// Let exampleCodec represent a Codec<ExampleJavaObject>
// Let exampleJson be a JsonElement

// Decode JsonElement into Java object
DataResult<ExampleJavaObject> result = exampleCodec.parse(JsonOps.INSTANCE, exampleJson);

result
    // Get result or partial on error, report error message
    .resultOrPartial(errorMessage -> /* Do something with error message */)
    // If result or partial is present, do something
    .ifPresent(decodedObject -> /* Do something with decoded object */);
```

Existing Codecs

Primitives

The **Codec** class contains static instances of codecs for certain defined primitives.

| Codec | Java Type |
|-------------|-------------|
| BOOL | Boolean |
| BYTE | Byte |
| SHORT | Short |
| INT | Integer |
| LONG | Long |
| FLOAT | Float |
| DOUBLE | Double |
| STRING | String |
| BYTE_BUFFER | ByteBuffer |
| INT_STREAM | IntStream |
| LONG_STREAM | LongStream |
| PASSTHROUGH | Dynamic<?>* |
| EMPTY | Unit** |

* **Dynamic** is an object which holds a value encoded in a supported **DynamicOps** format. These are typically used to convert encoded object formats into other encoded object formats.

** **Unit** is an object used to represent null objects.

Vanilla and Forge

Minecraft and Forge define many codecs for objects that are frequently encoded and decoded. Some examples include **ResourceLocation#CODEC** for **ResourceLocations**, **ExtraCodecs#INSTANT_ISO8601** for **Instants** in the **DateTimeFormatter#ISO_INSTANT** format, and **CompoundTag#CODEC** for **CompoundTags**.

!!! warning **CompoundTags** cannot decode lists of numbers from JSON using **JsonOps**. **JsonOps**, when converting, sets a number to its most narrow type. **ListTags** force a specific type for its data, so numbers with different types (e.g. 64 would be **byte**, 384 would be **short**) will throw an error on conversion.

Vanilla and Forge registries also have codecs for the type of object the registry contains (e.g. `Registry#BLOCK` or `ForgeRegistries#BLOCKS` have a `Codec<Block>`). `Registry#byNameCodec` and `IForgeRegistry#getCodec` will encode the registry object to their registry name, or an integer identifier if compressed. Vanilla registries also have a `Registry#holderByNameCodec` which encodes to a registry name and decodes to the registry object wrapped in a `Holder`.

Creating Codecs

Codecs can be created for encoding and decoding any object. For understanding purposes, the equivalent encoded JSON will be shown.

Records

Codecs can define objects through the use of records. Each record codec defines any object with explicit named fields. There are many ways to create a record codec, but the simplest is via `RecordCodecBuilder#create`.

`RecordCodecBuilder#create` takes in a function which defines an `Instance` and returns an application (`App`) of the object. A correlation can be drawn to creating a class *instance* and the constructors used to *apply* the class to the constructed object.

```
// Some object to create a codec for
public class SomeObject {

    public SomeObject(String s, int i, boolean b) { /* ... */ }

    public String s() { /* ... */ }

    public int i() { /* ... */ }

    public boolean b() { /* ... */ }
}
```

Fields An `Instance` can define up to 16 fields using `#group`. Each field must be an application defining the instance the object is being made for and the type of the object. The simplest way to meet this requirement is by taking a `Codec`, setting the name of the field to decode from, and setting the getter used to encode the field.

A field can be created from a `Codec` using `#fieldOf`, if the field is required, or `#optionalFieldOf`, if the field is wrapped in an `Optional` or defaulted. Either method requires a string containing the name of the field in the encoded object. The getter used to encode the field can then be set using `#forGetter`, taking in a function which given the object, returns the field data.

From there, the resulting product can be applied via `#apply` to define how the instance should construct the object for the application. For ease of convenience, the grouped fields should be listed in the same order they appear in the constructor such that the function can simply be a constructor method reference.

```
public static final Codec<SomeObject> RECORD_CODEC = RecordCodecBuilder.create(instance -> // Given an instance
    instance.group( // Define the fields within the instance
        Codec.STRING.fieldOf("s").forGetter(SomeObject::s), // String
        Codec.INT.optionalFieldOf("i", 0).forGetter(SomeObject::i), // Integer, defaults to 0 if field not present
        Codec.BOOL.fieldOf("b").forGetter(SomeObject::b) // Boolean
    ).apply(instance, SomeObject::new) // Define how to create the object
);
```

```
// Encoded SomeObject
{
    "s": "value",
    "i": 5,
    "b": false
}
```

```
// Another encoded SomeObject
{
    "s": "value2",
    // i is omitted, defaults to 0
    "b": true
}
```

Transformers

Codecs can be transformed into equivalent, or partially equivalent, representations through mapping methods. Each mapping method takes in two functions: one to transform the current type into the new type, and one to transform the new type back to the current type. This is done through the `#xmap` function.

```
// A class
public class ClassA {

    public ClassB toB() { /* ... */ }
}

// Another equivalent class
public class ClassB {

    public ClassA toA() { /* ... */ }
}

// Assume there is some codec A_CODEC
public static final Codec<ClassB> B_CODEC = A_CODEC.xmap(ClassA::toB, ClassB::toA);
```

If a type is partially equivalent, meaning that there are some restrictions during conversion, there are mapping functions which return a `DataResult` which can be used to return an error state whenever an exception or invalid state is reached.

| Is A Fully Equivalent to B | Is B Fully Equivalent to A | Transform Method |
|----------------------------|----------------------------|----------------------------|
| Yes | Yes | <code>#xmap</code> |
| Yes | No | <code>#flatComapMap</code> |
| No | Yes | <code>#comapFlatMap</code> |
| No | No | <code>#flatXMap</code> |

```
// Given an string codec to convert to a integer
// Not all strings can become integers (A is not fully equivalent to B)
// All integers can become strings (B is fully equivalent to A)
public static final Codec<Integer> INT_CODEC = Codec.STRING.comapFlatMap(
    s -> { // Return data result containing error on failure
        try {
            return DataResult.success(Integer.valueOf(s));
        } catch (NumberFormatException e) {
            return DataResult.error(s + " is not an integer.");
        }
    },
    Integer::toString // Regular function
);
```

```
// Will return 5
"5"
```

```
// Will error, not an integer
"value"
```

Range Codecs Range codecs are an implementation of `#flatXMap` which returns an error `DataResult` if the value is not inclusively between the set minimum and maximum. The value is still provided as a partial result if outside the bounds. There are implementations for integers, floats, and doubles via `#intRange`, `#floatRange`, and `#doubleRange` respectively.

```
public static final Codec<Integer> RANGE_CODEC = Codec.intRange(0, 4);
```

```
// Will be valid, inside [0, 4]
4
```

```
// Will error, outside [0, 4]
5
```

Defaults

If the result of encoding or decoding fails, a default value can be supplied instead via `Codec#orElse` or `Codec#orElseGet`.

```
public static final Codec<Integer> DEFAULT_CODEC = Codec.INT.orElse(0); // Can also be a supplied value via #orElseGet

// Not an integer, defaults to 0
"value"
```

Unit

A codec which supplies an in-code value and encodes to nothing can be represented using `Codec#unit`. This is useful if a codec uses a non-encodable entry within the data object.

```
public static final Codec<IForgeRegistry<Block>> UNIT_CODEC = Codec.unit(
    () -> ForgeRegistries.BLOCKS // Can also be a raw value
);

// Nothing here, will return block registry codec
```

List

A codec for a list of objects can be generated from an object codec via `Codec#listOf`.

```
// BlockPos#CODEC is a Codec<BlockPos>
public static final Codec<List<BlockPos>> LIST_CODEC = BlockPos.CODEC.listOf();

// Encoded List<BlockPos>
[
  [1, 2, 3], // BlockPos(1, 2, 3)
  [4, 5, 6], // BlockPos(4, 5, 6)
  [7, 8, 9]  // BlockPos(7, 8, 9)
]
```

List objects decoded using a list codec are stored in an **immutable** list. If a mutable list is needed, a transformer should be applied to the list codec.

Map

A codec for a map of keys and value objects can be generated from two codecs via `Codec#unboundedMap`. Unbounded maps can specify any string-based or string-transformed value to be a key.

```
// BlockPos#CODEC is a Codec<BlockPos>
public static final Codec<Map<String, BlockPos>> MAP_CODEC = Codec.unboundedMap(Codec.STRING, BlockPos.CODEC);

// Encoded Map<String, BlockPos>
{
  "key1": [1, 2, 3], // key1 -> BlockPos(1, 2, 3)
  "key2": [4, 5, 6], // key2 -> BlockPos(4, 5, 6)
  "key3": [7, 8, 9]  // key3 -> BlockPos(7, 8, 9)
}
```

Map objects decoded using an unbounded map codec are stored in an **immutable** map. If a mutable map is needed, a transformer should be applied to the map codec.

!!! warning Unbounded maps only support keys that encode/decode to/from strings. A key-value pair list codec can be used to get around this restriction.

Pair

A codec for pairs of objects can be generated from two codecs via `Codec#pair`.

A pair codec decodes objects by first decoding the left object in the pair, then taking the remaining part of the encoded object and decodes the right object from that. As such, the codecs must either express something about

the encoded object after decoding (such as records), or they have to be augmented into a `MapCodec` and transformed into a regular codec via `#codec`. This can typically be done by making the codec a field of some object.

```
public static final Codec<Pair<Integer, String>> PAIR_CODEC = Codec.pair(  
    Codec.INT.fieldOf("left").codec(),  
    Codec.STRING.fieldOf("right").codec()  
);
```

```
// Encoded Pair<Integer, String>  
{  
    "left": 5,          // fieldOf looks up 'left' key for left object  
    "right": "value"    // fieldOf looks up 'right' key for right object  
}
```

!!! tip A map codec with a non-string key can be encoded/decoded using a list of key-value pairs applied with a transformer.

Either

A codec for two different methods of encoding/decoding some object data can be generated from two codecs via `Codec#either`.

An either codec attempts to decode the object using the first codec. If it fails, it attempts to decode using the second codec. If that also fails, then the `DataResult` will only contain the error from the second codec failure.

```
public static final Codec<Either<Integer, String>> EITHER_CODEC = Codec.either(  
    Codec.INT,  
    Codec.STRING  
);
```

```
// Encoded Either$Left<Integer, String>  
5  
  
// Encoded Either$Right<Integer, String>  
"value"
```

!!! tip This can be used in conjunction with a transformer to get a specific object from two different methods of encoding.

Dispatch

Codecs can have subcodecs which can decode a particular object based upon some specified type via `Codec#dispatch`. This is typically used in registries which contain codecs, such as rule tests or block placers.

A dispatch codec first attempts to get the encoded type from some string key (usually `type`). From there, the type is decoded, calling a getter for the specific codec used to decode the actual object. If the `DynamicOps` used to decode the object compresses its maps, or the object codec itself is not augmented into a `MapCodec` (such as records or fielded primitives), then the object needs to be stored within a `value` key. Otherwise, the object is decoded at the same level as the rest of the data.

```
// Define our object  
public abstract class ExampleObject {  
  
    // Define the method used to specify the object type for encoding  
    public abstract Codec<? extends ExampleObject> type();  
}  
  
// Create simple object which stores a string  
public class StringObject extends ExampleObject {  
  
    public StringObject(String s) { /* ... */ }  
  
    public String s() { /* ... */ }  
  
    public Codec<? extends ExampleObject> type() {  
        // A registered registry object  
        // "string":  
        //     Codec.STRING.xmap(StringObject::new, StringObject::s)  
    }
```

```

        return STRING_OBJECT_CODEC.get();
    }
}

// Create complex object which stores a string and integer
public class ComplexObject extends ExampleObject {

    public ComplexObject(String s, int i) { /* ... */ }

    public String s() { /* ... */ }

    public int i() { /* ... */ }

    public Codec<? extends ExampleObject> type() {
        // A registered registry object
        // "complex":
        // RecordCodecBuilder.create(instance ->
        //     instance.group(
        //         Codec.STRING.fieldOf("s").forGetter(ComplexObject::s),
        //         Codec.INT.fieldOf("i").forGetter(ComplexObject::i)
        //     ).apply(instance, ComplexObject::new)
        // )
        return COMPLEX_OBJECT_CODEC.get();
    }
}

// Assume there is an IForgeRegistry<Codec<? extends ExampleObject>> DISPATCH
public static final Codec<ExampleObject> = DISPATCH.getCodec() // Gets Codec<Codec<? extends ExampleObject>>
    .dispatch(
        ExampleObject::type, // Get the codec from the specific object
        Function.identity() // Get the codec from the registry
    );

// Simple object
{
    "type": "string", // For StringObject
    "value": "value" // Codec type is not augmented from MapCodec, needs field
}

// Complex object
{
    "type": "complex", // For ComplexObject

    // Codec type is augmented from MapCodec, can be inlined
    "s": "value",
    "i": 0
}

```

Menus

Menus are one type of backend for Graphical User Interfaces, or GUIs; they handle the logic involved in interacting with some represented data holder. Menus themselves are not data holders. They are views which allow to user to indirectly modify the internal data holder state. As such, a data holder should not be directly coupled to any menu, instead passing in the data references to invoke and modify.

MenuType

Menus are created and removed dynamically and as such are not registry objects. As such, another factory object is registered instead to easily create and refer to the *type* of the menu. For a menu, these are **MenuTypes**.

MenuTypes must be registered.

MenuSupplier

A **MenuType** is created by passing in a **MenuSupplier** and a **FeatureFlagSet** to its constructor. A **MenuSupplier** represents a function which takes in the id of the container and the inventory of the player viewing the menu, and returns a newly created **AbstractContainerMenu**.

```
// For some DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenu>> MY_MENU = REGISTER.register("my_menu", () -> new
MenuType(MyMenu::new, FeatureFlags.DEFAULT_FLAGS));

// In MyMenu, an AbstractContainerMenu subclass
public MyMenu(int containerId, Inventory playerInv) {
    super(MY_MENU.get(), containerId);
    // ...
}
```

!!! note The container identifier is unique for an individual player. This means that the same container id on two different players will represent two different menus, even if they are viewing the same data holder.

The `MenuSupplier` is usually responsible for creating a menu on the client with dummy data references used to store and interact with the synced information from the server data holder.

IContainerFactory

If additional information is needed on the client (e.g. the position of the data holder in the world), then the subclass `IContainerFactory` can be used instead. In addition to the container id and the player inventory, this also provides a `FriendlyByteBuf` which can store additional information that was sent from the server. A `MenuType` can be created using an `IContainerFactory` via `IForgeMenuType#create`.

```
// For some DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenuExtra>> MY_MENU_EXTRA = REGISTER.register("my_menu_extra", () ->
IForgeMenuType.create(MyMenu::new));

// In MyMenuExtra, an AbstractContainerMenu subclass
public MyMenuExtra(int containerId, Inventory playerInv, FriendlyByteBuf extraData) {
    super(MY_MENU_EXTRA.get(), containerId);
    // Store extra data from buffer
    // ...
}
```

AbstractContainerMenu

All menus are extended from `AbstractContainerMenu`. A menu takes in two parameters, the `MenuType`, which represents the type of the menu itself, and the container id, which represents the unique identifier of the menu for the current accessor.

!!! important The player can only have 100 unique menus open at once.

Each menu should contain two constructors: one used to initialize the menu on the server and one used to initialize the menu on the client. The constructor used to initialize the menu on the client is the one supplied to the `MenuType`. Any fields that the server menu constructor contains should have some default for the client menu constructor.

```
// Client menu constructor
public MyMenu(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory);
}

// Server menu constructor
public MyMenu(int containerId, Inventory playerInventory) {
    // ...
}
```

Each menu implementation must implement two methods: `#stillValid` and `#quickMoveStack`.

#stillValid and ContainerLevelAccess

`#stillValid` determines whether the menu should remain open for a given player. This is typically directed to the static `#stillValid` which takes in a `ContainerLevelAccess`, the player, and the `Block` this menu is attached to. The client menu must always return `true` for this method, which the static `#stillValid` does default to. This implementation checks whether the player is within eight blocks of where the data storage object is located.

A `ContainerLevelAccess` supplies the current level and location of the block within an enclosed scope. When constructing the menu on the server, a new access can be created by calling `ContainerLevelAccess#create`. The client menu constructor can pass in `ContainerLevelAccess#NULL`, which will do nothing.

```
// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, ContainerLevelAccess.NULL);
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerLevelAccess access) {
    // ...
}

// Assume this menu is attached to RegistryObject<Block> MY_BLOCK
@Override
public boolean stillValid(Player player) {
    return AbstractContainerMenu.stillValid(this.access, player, MY_BLOCK.get());
}
```

Data Synchronization

Some data needs to be present on both the server and the client to display to the player. To do this, the menu implements a basic layer of data synchronization such that whenever the current data does not match the data last synced to the client. For players, this is checked every tick.

Minecraft supports two forms of data synchronization by default: `ItemStacks` via `Slots` and integers via `DataSlots`. `Slots` and `DataSlots` are views which hold references to data storages that can be modified by the player in a screen, assuming the action is valid. These can be added to a menu within the constructor through `#addSlot` and `#addDataSlot`.

!!! note Since `Containers` used by `Slots` are deprecated by Forge in favor of using the `IItemHandler` capability, the rest of the explanation will revolve around using the capability variant: `SlotItemHandler`.

A `SlotItemHandler` contains four parameters: the `IItemHandler` representing the inventory the stacks are within, the index of the stack this slot is specifically representing, and the x and y position of where the top-left position of the slot will render on the screen relative to `AbstractContainerScreen#leftPos` and `#topPos`. The client menu constructor should always supply an empty instance of an inventory of the same size.

In most cases, any slots the menu contains is first added, followed by the player's inventory, and finally concluded with the player's hotbar. To access any individual `Slot` from the menu, the index must be calculated based upon the order of which slots were added.

A `DataSlot` is an abstract class which should implement a getter and setter to reference the data stored in the data storage object. The client menu constructor should always supply a new instance via `DataSlot#standalone`.

These, along with slots, should be recreated every time a new menu is initialized.

!!! warning Although a `DataSlot` stores an integer, it is effectively limited to a **short** (-32768 to 32767) because of how it sends the value across the network. The 16 high-order bits of the integer are ignored.

```
// Assume we have an inventory from a data object of size 5
// Assume we have a DataSlot constructed on each initialization of the server menu

// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, new ItemStackHandler(5), DataSlot.standalone());
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, IItemHandler dataInventory, DataSlot dataSingle) {
    // Check if the data inventory size is some fixed value
    // Then, add slots for data inventory
    this.addSlot(new SlotItemHandler(dataInventory, /*...*/));

    // Add slots for player inventory
    this.addSlot(new Slot(playerInventory, /*...*/));
}
```

```

// Add data slots for handled integers
this.addDataSlot(dataSingle);

// ...
}

```

ContainerData If multiple integers need to be synced to the client, a `ContainerData` can be used to reference the integers instead. This interface functions as an index lookup such that each index represents a different integer. `ContainerData`s can also be constructed in the data object itself if the `ContainerData` is added to the menu through `#addDataSlots`. The method creates a new `DataSlot` for the amount of data specified by the interface. The client menu constructor should always supply a new instance via `SimpleContainerData`.

```

// Assume we have a ContainerData of size 3

// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, new SimpleContainerData(3));
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerData dataMultiple) {
    // Check if the ContainerData size is some fixed value
    checkContainerDataCount(dataMultiple, 3);

    // Add data slots for handled integers
    this.addDataSlots(dataMultiple);

    // ...
}

```

!!! warning As `ContainerData` delegates to `DataSlots`, these are also limited to a **short** (-32768 to 32767).

#quickMoveStack `#quickMoveStack` is the second method that must be implemented by any menu. This method is called whenever a stack has been shift-clicked, or quick moved, out of its current slot until the stack has been fully moved out of its previous slot or there is no other place for the stack to go. The method returns a copy of the stack in the slot being quick moved.

Stacks are typically moved between slots using `#moveItemStackTo`, which moves the stack into the first available slot. It takes in the stack to be moved, the first slot index (inclusive) to try and move the stack to, the last slot index (exclusive), and whether to check the slots from first to last (when **false**) or from last to first (when **true**).

Across Minecraft implementations, this method is fairly consistent in its logic:

```

// Assume we have a data inventory of size 5
// The inventory has 4 inputs (index 1 - 4) which outputs to a result slot (index 0)
// We also have the 27 player inventory slots and the 9 hotbar slots
// As such, the actual slots are indexed like so:
//   - Data Inventory: Result (0), Inputs (1 - 4)
//   - Player Inventory (5 - 31)
//   - Player Hotbar (32 - 40)
@Override
public ItemStack quickMoveStack(Player player, int quickMovedSlotIndex) {
    // The quick moved slot stack
    ItemStack quickMovedStack = ItemStack.EMPTY;
    // The quick moved slot
    Slot quickMovedSlot = this.slots.get(quickMovedSlotIndex)

    // If the slot is in the valid range and the slot is not empty
    if (quickMovedSlot != null && quickMovedSlot.hasItem()) {
        // Get the raw stack to move
        ItemStack rawStack = quickMovedSlot.getItem();
        // Set the slot stack to a copy of the raw stack
        quickMovedStack = rawStack.copy();

        /*
        The following quick move logic can be simplified to if in data inventory,
        try to move to player inventory/hotbar and vice versa for containers
        that cannot transform data (e.g. chests).
        */
    }
}

```

```

*/

// If the quick move was performed on the data inventory result slot
if (quickMovedSlotIndex == 0) {
    // Try to move the result slot into the player inventory/hotbar
    if (!this.moveItemStackTo(rawStack, 5, 41, true)) {
        // If cannot move, no longer quick move
        return ItemStack.EMPTY;
    }

    // Perform logic on result slot quick move
    slot.onQuickCraft(rawStack, quickMovedStack);
}
// Else if the quick move was performed on the player inventory or hotbar slot
else if (quickMovedSlotIndex >= 5 && quickMovedSlotIndex < 41) {
    // Try to move the inventory/hotbar slot into the data inventory input slots
    if (!this.moveItemStackTo(rawStack, 1, 5, false)) {
        // If cannot move and in player inventory slot, try to move to hotbar
        if (quickMovedSlotIndex < 32) {
            if (!this.moveItemStackTo(rawStack, 32, 41, false)) {
                // If cannot move, no longer quick move
                return ItemStack.EMPTY;
            }
        }
        // Else try to move hotbar into player inventory slot
        else if (!this.moveItemStackTo(rawStack, 5, 32, false)) {
            // If cannot move, no longer quick move
            return ItemStack.EMPTY;
        }
    }
}
// Else if the quick move was performed on the data inventory input slots, try to move to player inventory/hotbar
else if (!this.moveItemStackTo(rawStack, 5, 41, false)) {
    // If cannot move, no longer quick move
    return ItemStack.EMPTY;
}

if (rawStack.isEmpty()) {
    // If the raw stack has completely moved out of the slot, set the slot to the empty stack
    quickMovedSlot.set(ItemStack.EMPTY);
} else {
    // Otherwise, notify the slot that that the stack count has changed
    quickMovedSlot.setChanged();
}

/*
The following if statement and Slot#onTake call can be removed if the
menu does not represent a container that can transform stacks (e.g.
chests).
*/
if (rawStack.getCount() == quickMovedStack.getCount()) {
    // If the raw stack was not able to be moved to another slot, no longer quick move
    return ItemStack.EMPTY;
}
// Execute logic on what to do post move with the remaining stack
quickMovedSlot.onTake(player, rawStack);
}

return quickMovedStack; // Return the slot stack
}

```

Opening a Menu

Once a menu type has been registered, the menu itself has been finished, and a screen has been attached, a menu can then be opened by the player. Menus can be opened by calling `NetworkHooks#openScreen` on the logical server. The method takes in the player opening the menu, the `MenuProvider` of the server side menu, and optionally a `FriendlyByteBuf` if extra data needs to be synced to the client.

!!! note `NetworkHooks#openScreen` with the `FriendlyByteBuf` parameter should only be used if a menu type was created using an `IContainerFactory`.

MenuProvider A MenuProvider is an interface that contains two methods: `#createMenu`, which creates the server instance of the menu, and `#getDisplayname`, which returns a component containing the title of the menu to pass to the screen. The `#createMenu` method contains three parameter: the container id of the menu, the inventory of the player who opened the menu, and the player who opened the menu.

A MenuProvider can easily be created using SimpleMenuProvider, which takes in a method reference to create the server menu and the title of the menu.

```
// In some implementation
NetworkHooks.openScreen(serverPlayer, new SimpleMenuProvider(
    (containerId, playerInventory, player) -> new MyMenu(containerId, playerInventory),
    Component.translatable("menu.title.examplemod.mymenu")
));
```

Common Implementations

Menus are typically opened on a player interaction of some kind (e.g. when a block or entity is right-clicked).

Block Implementation Blocks typically implement a menu by overriding `BlockBehaviour#use`. If on the logical client, the interaction returns `InteractionResult#SUCCESS`. Otherwise, it opens the menu and returns `InteractionResult#CONSUME`.

The MenuProvider should be implemented by overriding `BlockBehaviour#getMenuProvider`. Vanilla methods use this to view the menu in spectator mode.

```
// In some Block subclass
@Override
public MenuProvider getMenuProvider(BlockState state, Level level, BlockPos pos) {
    return new SimpleMenuProvider(/* ... */);
}

@Override
public InteractionResult use(BlockState state, Level level, BlockPos pos, Player player, InteractionHand hand,
BlockHitResult result) {
    if (!level.isClientSide && player instanceof ServerPlayer serverPlayer) {
        NetworkHooks.openScreen(serverPlayer, state.getMenuProvider(level, pos));
    }
    return InteractionResult.sidedSuccess(level.isClientSide);
}
```

!!! note This is the simplest way to implement the logic, not the only way. If you want the block to only open the menu under certain conditions, then some data will need to be synced to the client beforehand to return `InteractionResult#PASS` or `#FAIL` if the conditions are not met.

Mob Implementation Mobs typically implement a menu by overriding `Mob#mobInteract`. This is done similarly to the block implementation with the only difference being that the Mob itself should implement `MenuProvider` to support spectator mode viewing.

```
public class MyMob extends Mob implements MenuProvider {
    // ...

    @Override
    public InteractionResult mobInteract(Player player, InteractionHand hand) {
        if (!this.level.isClientSide && player instanceof ServerPlayer serverPlayer) {
            NetworkHooks.openScreen(serverPlayer, this);
        }
        return InteractionResult.sidedSuccess(this.level.isClientSide);
    }
}
```

!!! note Once again, this is the simplest way to implement the logic, not the only way.

Screens

Screens are typically the base of all Graphical User Interfaces (GUIs) in Minecraft: taking in user input, verifying it on the server, and syncing the resulting action back to the client. They can be combined with menus to create an communication network for inventory-like views, or they can be standalone which modders can handle through their own network implementations.

Screens are made up of numerous parts, making it difficult to fully understand what a ‘screen’ actually is in Minecraft. As such, this document will go over each of the screen’s components and how it is applied before discussing the screen itself.

Relative Coordinates

Whenever anything is rendered, there needs to be some identifier which specifies where it will appear. With numerous abstractions, most of Minecraft’s rendering calls takes in an x, y, and z value in a coordinate plane. X values increase from left to right, y from top to bottom, and z from far to near. However, the coordinates are not fixed to a specified range. They can change depending on the size of the screen and the scale at which is specified within the options. As such, extra care must be taken to make sure the values of the coordinates while rendering scale properly to the changeable screen size.

Information on how to relativize your coordinates will be within the screen section.

!!! important If you choose to use fixed coordinates or incorrectly scale the screen, the rendered objects may look strange or misplaced. An easy way to check if you relativized your coordinates correctly is to click the ‘Gui Scale’ button in your video settings. This value is used as the divisor to the width and height of your display when determining the scale at which a GUI should render.

Gui Graphics

Any GUI rendered by Minecraft is typically done using `GuiGraphics`. `GuiGraphics` is the first parameter to almost all rendering methods; it contains basic methods to render commonly used objects. These fall into five categories: colored rectangles, strings, and textures, items, and tooltips. There is also an additional method for rendering a snippet of a component (`#enableScissor` / `#disableScissor`). `GuiGraphics` also exposes the `PoseStack` which applies the transformations necessary to properly render where the component should be rendered. Additionally, colors are in the ARGB format.

Colored Rectangles

Colored rectangles are drawn through a position color shader. There are three types of colored rectangles that can be drawn.

First, there is a colored horizontal and vertical one-pixel wide line, `#hLine` and `#vLine` respectively. `#hLine` takes in two x coordinates defining the left and right (inclusively), the top y coordinate, and the color. `#vLine` takes in the left x coordinate, two y coordinates defining the top and bottom (inclusively), and the color.

Second, there is the `#fill` method, which draws a rectangle to the screen. The line methods internally call this method. This takes in the left x coordinate, the top y coordinate, the right x coordinate, the bottom y coordinate, and the color.

Finally, there is the `#fillGradient` method, which draws a rectangle with a vertical gradient. This takes in the right x coordinate, the bottom y coordinate, the left x coordinate, the top y coordinate, the z coordinate, and the bottom and top colors.

Strings

Strings are drawn through its `Font`, typically consisting of their own shaders for normal, see through, and offset mode. There are two alignment of strings that can be rendered, each with a back shadow: a left-aligned string (`#drawString`) and a center-aligned string (`#drawCenteredString`). These both take in the font the string will be rendered in, the string to draw, the x coordinate representing the left or center of the string respectively, the top y coordinate, and the color.

!!! note Strings should typically be passed in as **Components** as they handle a variety of usecases, including the two other overloads of the method.

Textures

Textures are drawn through blitting, hence the method name **#blit**, which, for this purpose, copies the bits of an image and draws them directly to the screen. These are drawn through a position texture shader. While there are many different **#blit** overloads, we will only discuss two static **#blits**.

The first static **#blit** takes in six integers and assumes the texture being rendered is on a 256 x 256 PNG file. It takes in the left x and top y screen coordinate, the left x and top y coordinate within the PNG, and the width and height of the image to render.

!!! note The size of the PNG file must be specified so that the coordinates can be normalized to obtain the associated UV values.

The static **#blit** which the first calls expands this to nine integers, only assuming the image is on a PNG file. It takes in the left x and top y screen coordinate, the z coordinate (referred to as the blit offset), the left x and top y coordinate within the PNG, the width and height of the image to render, and the width and height of the PNG file.

Blit Offset The z coordinate when rendering a texture is typically set to the blit offset. The offset is responsible for properly layering renders when viewing a screen. Renders with a smaller z coordinate are rendered in the background and vice versa where renders with a larger z coordinate are rendered in the foreground. The z offset can be set directly on the **PoseStack** itself via **#translate**. Some basic offset logic is applied internally in some methods of **GuiGraphics** (e.g. item rendering).

!!! important When setting the blit offset, you must reset it after rendering your object. Otherwise, other objects within the screen may be rendered in an incorrect layer causing graphical issues. It is recommended to push the current pose before translating and then popping after all rendering at the offset is completed.

Renderable

Renderables are essentially objects that are rendered. These include screens, buttons, chat boxes, lists, etc. **Renderables** only have one method: **#render**. This takes in the **GuiGraphics** used to render things to the screen, the x and y positions of the mouse scaled to the relative screen size, and the tick delta (how many ticks have passed since the last frame).

Some common renderables are screens and ‘widgets’: interactable elements which typically render on the screen such as **Button**, its subtype **ImageButton**, and **EditBox** which is used to input text on the screen.

GuiEventListener

Any screen rendered in Minecraft implements **GuiEventListener**. **GuiEventListeners** are responsible for handling user interaction with the screen. These include inputs from the mouse (movement, clicked, released, dragged, scrolled, mouseover) and keyboard (pressed, released, typed). Each method returns whether the associated action affected the screen successfully. Widgets like buttons, chat boxes, lists, etc. also implement this interface.

ContainerEventHandler

Almost synonymous with **GuiEventListeners** are their subtype: **ContainerEventHandlers**. These are responsible for handling user interaction on screens which contain widgets, managing which is currently focused and how the associated interactions are applied. **ContainerEventHandlers** add three additional features: interactable children, dragging, and focusing.

Event handlers hold children which are used to determine the interaction order of elements. During the mouse event handlers (excluding dragging), the first child in the list that the mouse hovers over has their logic executed.

Dragging an element with the mouse, implemented via **#mouseClicked** and **#mouseReleased**, provides more precisely executed logic.

Focusing allows for a specific child to be checked first and handled during an event's execution, such as during keyboard events or dragging the mouse. Focus is typically set through `#setFocused`. In addition, interactable children can be cycled using `#nextFocusPath`, selecting the child based upon the `FocusNavigationEvent` passed in.

!!! note Screens implement `ContainerEventHandler` through `AbstractContainerEventHandler`, which adds in the setter and getter logic for dragging and focusing children.

NarratableEntry

`NarratableEntry`s are elements which can be spoken about through Minecraft's accessibility narration feature. Each element can provide different narration depending on what is hovered or selected, prioritized typically by focus, hovering, and then all other cases.

`NarratableEntry`s have three methods: one which determines the priority of the element (`#narrationPriority`), one which determines whether to speak the narration (`#isActive`), and finally one which supplies the narration to its associated output, spoken or read (`#updateNarration`).

!!! note All widgets from Minecraft are `NarratableEntry`s, so it typically does not need to be manually implemented if using an available subtype.

The Screen Subtype

With all of the above knowledge, a basic screen can be constructed. To make it easier to understand, the components of a screen will be mentioned in the order they are typically encountered.

First, all screens take in a `Component` which represents the title of the screen. This component is typically drawn to the screen by one of its subtypes. It is only used in the base screen for the narration message.

```
// In some Screen subclass
public MyScreen(Component title) {
    super(title);
}
```

Initialization

Once a screen has been initialized, the `#init` method is called. The `#init` method sets the initial settings inside the screen from the `ItemRenderer` and `Minecraft` instance to the relative width and height as scaled by the game. Any setup such as adding widgets or precomputing relative coordinates should be done in this method. If the game window is resized, the screen will be reinitialized by calling the `#init` method.

There are three ways to add a widget to a screen, each serving a separate purpose:

| Method | Description |
|-----------------------------------|---|
| <code>#addWidget</code> | Adds a widget that is interactable and narrated, but not rendered. |
| <code>#addRenderableOnly</code> | Adds a widget that will only be rendered; it is not interactable or narrated. |
| <code>#addRenderableWidget</code> | Adds a widget that is interactable, narrated, and rendered. |

Typically, `#addRenderableWidget` will be used most often.

```
// In some Screen subclass
@Override
protected void init() {
    super.init();

    // Add widgets and precomputed values
    this.addRenderableWidget(new EditText(/* ... */));
}
```

Ticking Screens

Screens also tick using the `#tick` method to perform some level of client side logic for rendering purposes. The most common example is the `EditText` for the blinking cursor.

```
// In some Screen subclass
@Override
public void tick() {
    super.tick();

    // Add ticking logic for EditText in editText
    this.editBox.tick();
}
```

Input Handling

Since screens are subtypes of `GuiEventListeners`, the input handlers can also be overridden, such as for handling logic on a specific key press.

Rendering the Screen

Finally, screens are rendered through the `#render` method provided by being a `Renderable` subtype. As mentioned, the `#render` method draws the everything the screen has to render every frame, such as the background, widgets, tooltips, etc. By default, the `#render` method only renders the widgets to the screen.

The two most common things rendered within a screen that is typically not handled by a subtype is the background and the tooltips.

The background can be rendered using `#renderBackground`, with one method taking in a v Offset for the options background whenever a screen is rendered when the level behind it cannot be.

Tooltips are rendered through `GuiGraphics#renderTooltip` or `GuiGraphics#renderComponentTooltip` which can take in the text components being rendered, an optional custom tooltip component, and the x / y relative coordinates on where the tooltip should be rendered on the screen.

```
// In some Screen subclass

// mouseX and mouseY indicate the scaled coordinates of where the cursor is in on the screen
@Override
public void render(GuiGraphics graphics, int mouseX, int mouseY, float partialTick) {
    // Background is typically rendered first
    this.renderBackground(graphics);

    // Render things here before widgets (background textures)

    // Then the widgets if this is a direct child of the Screen
    super.render(graphics, mouseX, mouseY, partialTick);

    // Render things after widgets (tooltips)
}
```

Closing the Screen

When a screen is closed, two methods handle the teardown: `#onClose` and `#removed`.

`#onClose` is called whenever the user makes an input to close the current screen. This method is typically used as a callback to destroy and save any internal processes in the screen itself. This includes sending packets to the server.

`#removed` is called just before the screen changes and is released to the garbage collector. This handles anything that hasn't been reset back to its initial state before the screen was opened.

```
// In some Screen subclass

@Override
public void onClose() {
    // Stop any handlers here
}
```



```

    // Call last in case it interferes with the override
    super.onClose();
}

@Override
public void removed() {
    // Reset initial states here

    // Call last in case it interferes with the override
    super.removed();
};

```

AbstractContainerScreen

If a screen is directly attached to a menu, then an **AbstractContainerScreen** should be subclassed instead. An **AbstractContainerScreen** acts as the renderer and input handler of a menu and contains logic for syncing and interacting with slots. As such, only two methods typically need to be overridden or implemented to have a working container screen. Once again, to make it easier to understand, the components of a container screen will be mentioned in the order they are typically encountered.

An **AbstractContainerScreen** typically requires three parameters: the container menu being opened (represented by the generic T), the player inventory (only for the display name), and the title of the screen itself. Within here, a number of positioning fields can be set:

| Field | Description |
|------------------------------|--|
| <code>imageWidth</code> | The width of the texture used for the background. This is typically inside a PNG of 256 x 256 and defaults to 176. |
| <code>imageHeight</code> | The width of the texture used for the background. This is typically inside a PNG of 256 x 256 and defaults to 166. |
| <code>titleLabelX</code> | The relative x coordinate of where the screen title will be rendered. |
| <code>titleLabelY</code> | The relative y coordinate of where the screen title will be rendered. |
| <code>inventoryLabelX</code> | The relative x coordinate of where the player inventory name will be rendered. |
| <code>inventoryLabelY</code> | The relative y coordinate of where the player inventory name will be rendered. |

!!! important In a previous section, it mentioned that precomputed relative coordinates should be set in the `#init` method. This still remains true, as the values mentioned here are not precomputed coordinates but static values and relativized coordinates.

The image values are static and non changing as they represent the background texture size. To make things

The ``leftPos`` and ``topPos`` is also used as a convenient way to render the background as they already repre

```

// In some AbstractContainerScreen subclass
public MyContainerScreen(MyMenu menu, Inventory playerInventory, Component title) {
    super(menu, playerInventory, title);

    this.titleLabelX = 10;
    this.inventoryLabelX = 10;

    /*
     * If the 'imageHeight' is changed, 'inventoryLabelY' must also be
     * changed as the value depends on the 'imageHeight' value.
     */
}

```

Menu Access

As the menu is passed into the screen, any values that were within the menu and synced (either through slots, data slots, or a custom system) can now be accessed through the `menu` field.

Container Tick

Container screens tick within the `#tick` method when the player is alive and looking at the screen via `#containerTick`. This essentially takes the place of `#tick` within container screens, with its most common usage being to tick the recipe book.

```
// In some AbstractContainerScreen subclass
@Override
protected void containerTick() {
    super.containerTick();

    // Tick things here
}
```

Rendering the Container Screen

The container screen is rendered across three methods: `#renderBg`, which renders the background textures, `#renderLabels`, which renders any text on top of the background, and `#render` which encompass the previous two methods in addition to providing a grayed out background and tooltips.

Starting with `#render`, the most common override (and typically the only case) adds the background, calls the super to render the container screen, and finally renders the tooltips on top of it.

```
// In some AbstractContainerScreen subclass
@Override
public void render(GuiGraphics graphics, int mouseX, int mouseY, float partialTick) {
    this.renderBackground(graphics);
    super.render(graphics, mouseX, mouseY, partialTick);

    /*
     * This method is added by the container screen to render
     * the tooltip of the hovered slot.
     */
    this.renderTooltip(graphics, mouseX, mouseY);
}
```

Within the super, `#renderBg` is called to render the background of the screen. The most standard representation uses three method calls: two for setup and one to draw the background texture.

```
// In some AbstractContainerScreen subclass

// The location of the background texture (assets/<namespace>/<path>)
private static final ResourceLocation BACKGROUND_LOCATION = new ResourceLocation(MOD_ID,
"textures/gui/container/my_container_screen.png");

@Override
protected void renderBg(GuiGraphics graphics, float partialTick, int mouseX, int mouseY) {
    /*
     * Renders the background texture to the screen. 'leftPos' and
     * 'topPos' should already represent the top left corner of where
     * the texture should be rendered as it was precomputed from the
     * 'imageWidth' and 'imageHeight'. The two zeros represent the
     * integer u/v coordinates inside the 256 x 256 PNG file.
     */
    graphics.blit(BACKGROUND_LOCATION, this.leftPos, this.topPos, 0, 0, this.imageWidth, this.imageHeight);
}
```

Finally, `#renderLabels` is called to render any text above the background, but below the tooltips. This simply calls uses the font to draw the associated components.

```
// In some AbstractContainerScreen subclass
@Override
protected void renderLabels(GuiGraphics graphics, int mouseX, int mouseY) {
    super.renderLabels(graphics, mouseX, mouseY);

    // Assume we have some Component 'label'
    // 'label' is drawn at 'labelX' and 'labelY'
    graphics.drawString(this.font, this.label, this.labelX, this.labelY, 0x404040);
}
```

!!! note When rendering the label, you do **not** need to specify the `leftPos` and `topPos` offset. Those have already been translated within the `PoseStack` so everything within this method is drawn relative to those coordinates.

Registering an AbstractContainerScreen

To use an `AbstractContainerScreen` with a menu, it needs to be registered. This can be done by calling `MenuScreens#register` within the `FMLClientSetupEvent` on the **mod event bus**.

```
// Event is listened to on the mod event bus
private void clientSetup(FMLClientSetupEvent event) {
    event.enqueueWork(
        // Assume RegistryObject<MenuType<MyMenu>> MY_MENU
        // Assume MyContainerScreen<MyMenu> which takes in three parameters
        () -> MenuScreens.register(MY_MENU.get(), MyContainerScreen::new)
    );
}
```

!!! warning `MenuScreens#register` is not thread-safe, so it needs to be called inside `#enqueueWork` provided by the parallel dispatch event.

Root Transforms

Adding the `transform` entry at the top level of a model JSON suggests to the loader that a transformation should be applied to all geometry right before the rotations in the blockstate file in the case of a block model, and before the display transforms in the case of an item model. The transformation is available through `IGeometryBakingContext#getRootTransform()` in `IUnbakedGeometry#bake()`.

Custom model loaders may ignore this field entirely.

The root transforms can be specified in two formats:

1. A JSON object containing a singular `matrix` entry containing a raw transformation matrix in the form of a nested JSON array with the last row omitted (3*4 matrix, row major order). The matrix is the composition of the translation, left rotation, scale, right rotation and the transformation origin in that order. Example demonstrating the structure: `js "transform": { "matrix": [ [ 0, 0, 0, 0 ], [ 0, 0, 0, 0 ], [ 0, 0, 0, 0 ] ] }`
2. A JSON object containing any combination of the following optional entries:
 - `origin`: origin point used for the rotations and scaling
 - `translation`: relative translation
 - `rotation` or `left_rotation`: rotation around the translated origin to be applied before scaling
 - `scale`: scale relative to the translated origin
 - `right_rotation` or `post_rotation`: rotation around the translated origin to be applied after scaling

Element-wise specification

If the transformation is specified as a combination of the entries mentioned in option 4, these entries will be applied in the order of `translation`, `left_rotation`, `scale`, `right_rotation`.

The transformation is moved to the specified origin as a last step.

```
{
  "transform": {
    "origin": "center",
    "translation": [ 0, 0.5, 0 ],
    "rotation": { "y": 45 }
  },
  // ...
}
```

The elements are expected to be defined as follows:

Origin

The origin can be specified either as an array of 3 floating point values representing a three-dimensional vector: [*x*, *y*, *z*] or as one of the three default values:

- "corner" (0, 0, 0)
- "center" (.5, .5, .5)
- "opposing-corner" (1, 1, 1)

If the origin is not specified, it defaults to "opposing-corner".

Translation

The translation must be specified as an array of 3 floating point values representing a three-dimensional vector: [*x*, *y*, *z*] and defaults to (0, 0, 0) if not present.

Left and Right Rotation

The rotations can be specified in any one of the following four ways:

- Single JSON object with a single axis => rotation degree mapping: { "x": 90 }
- Array of an arbitrary amount of JSON objects with the above format (applied in the order they are specified in): [{ "x": 90 }, { "y": 45 }, { "x": -22.5 }]
- Array of 3 floating point values specifying the rotation in degrees around each axis: [90, 180, 45]
- Array of 4 floating point values specifying a quaternion directly: [0.38268346, 0, 0, 0.9238795] (example equals 45 degrees around the X axis)

If the respective rotation is not specified, it will default to no rotation.

Scale

The scale must be specified as an array of 3 floating point values representing a three-dimensional vector: [*x*, *y*, *z*] and defaults to (1, 1, 1) if not present.

Render Types

Adding the `render_type` entry at the top level of the JSON suggests to the loader what render type the model should use. If not specified, the loader gets to pick the render type(s) used, often falling back to the render types returned by `ItemBlockRenderTypes#getRenderLayers()`.

Custom model loaders may ignore this field entirely.

!!! note Since 1.19 this is preferred over the deprecated method of setting the applicable render type(s) via `ItemBlockRenderTypes#setRenderLayer()` for blocks.

Example of a model for a cutout block with the glass texture

```
{
  "render_type": "minecraft:cutout",
  "parent": "block/cube_all",
  "textures": {
    "all": "block/glass"
  }
}
```

Vanilla Values

The following options with the respective chunk and entity render type are supplied by Forge (`NamedRenderTypeManager#preRegisterVanillaRenderTypes()`):

- `minecraft:solid`
 - Chunk render type: `RenderType#solid()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_SOLID`

- Used for fully solid blocks (i.e. Stone)
- `minecraft:cutout`
 - Chunk render type: `RenderType#cutout()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_CUTOUT`
 - Used for blocks where any given pixel is either fully transparent or fully opaque (i.e. Glass Block)
- `minecraft:cutout_mipped`
 - Chunk render type: `RenderType#cutoutMipped()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_CUTOUT`
 - Chunk and entity render type differ due to mipmapping on the entity render type making items look weird
 - Used for blocks where any given pixel is either fully transparent or fully opaque and the texture should be scaled down at larger distances (mipmapping) to avoid visual artifacts (i.e. Leaves)
- `minecraft:cutout_mipped_all`
 - Chunk render type: `RenderType#cutoutMipped()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_CUTOUT_MIPPED`
 - Used in similar cases as `minecraft:cutout_mipped` when the item representation should also have mipmapping applied
- `minecraft:translucent`
 - Chunk render type: `RenderType#translucent()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_TRANSLUCENT`
 - Used for blocks where any given pixel may be partially transparent (i.e. Stained Glass)
- `minecraft:tripwire`
 - Chunk render type: `RenderType#tripwire()`
 - Entity render type: `ForgeRenderTypes#ITEM_LAYERED_TRANSLUCENT`
 - Chunk and entity render type differ due to the tripwire render type not being feasible as an entity render type
 - Used for blocks with the special requirement of being rendered to the weather render target (i.e. Tripwire)

Custom Values

Custom named render types to be specified in a model can be registered in the `RegisterNamedRenderTypesEvent`. This event is fired on the mod event bus.

A custom named render type consists of two or three components:

- A chunk render type - any of the types in the list returned by `RenderType.chunkBufferLayers()` can be used
- A render type with the `DefaultVertexFormat.NEW_ENTITY` vertex format (“entity render type”)
- A render type with the `DefaultVertexFormat.NEW_ENTITY` vertex format for use when the *Fabulous!* graphics mode is selected (optional)

The chunk render type is used when a block using this named render type is rendered as part of the chunk geometry. The required entity render type is used when an item using this named render type is rendered in the Fast and Fancy graphics modes (inventory, ground, item frame, etc.).

The optional entity render type is used the same way as the required entity render type when the *Fabulous!* graphics mode is selected. This render type is needed in cases where the required entity render type does not work in the *Fabulous!* graphics mode (typically only applies to translucent render types).

```
public static void onRegisterNamedRenderTypes(RegisterNamedRenderTypesEvent event)
{
    event.register("special_cutout", RenderType.cutout(), Sheets.cutoutBlockSheet());
    event.register("special_translucent", RenderType.translucent(), Sheets.translucentCullBlockSheet(),
    Sheets.translucentItemSheet());
}
```

These can then be addressed in JSON as `<your_mod_id>:special_cutout` and `<your_mod_id>:special_translucent`.

Part Visibility

Adding the `visibility` entry at the top level of a model JSON allows control over the visibility of different parts of the model to decide whether they should be baked into the final `BakedModel`. The definition of a “part” is dependent on the model loader loading this model and custom model loaders are free to ignore this entry completely. Out of the model loaders provided by Forge only the composite model loader and the OBJ model loader make use of this functionality. The visibility entries are specified as `"part name": boolean` entries.

Example of a composite model with two parts, the second of which will not be baked into the final model, and two child models overriding this visibility to have only the first part and both parts visible respectively:

```
// mycompositemodel.json
{
  "loader": "forge:composite",
  "children": {
    "part_one": {
      "parent": "mymod:mypartmodel_one"
    },
    "part_two": {
      "parent": "mymod:mypartmodel_two"
    }
  },
  "visibility": {
    "part_two": false
  }
}

// mycompositechild_one.json
{
  "parent": "mymod:mycompositemodel",
  "visibility": {
    "part_one": false,
    "part_two": true
  }
}

// mycompositechild_two.json
{
  "parent": "mymod:mycompositemodel",
  "visibility": {
    "part_two": true
  }
}
```

The visibility of a given part is determined by checking whether the model specifies a visibility for this part and, if not present, recursively checking the model’s parent until either an entry is found or there is no further parent to check, in which case it defaults to true.

This allows setups like the following where multiple models use different parts of a single composite model:

1. A composite model specifies multiple components
2. Multiple models specify this composite model as their parent
3. These child models individually specify different visibilities for the parts

Face Data

In a vanilla “elements” model, additional data about an element’s faces can be specified at either the element level or the face level. Faces which do not specify their own face data will fall back to the element’s face data or a default if no face data is specified at the element level.

To use this extension for a generated item model, the model must be loaded through the `forge:item_layers` model loader due to the vanilla item model generator not being extended to read this additional data.

All values of the face data are optional.

Elements Model

In vanilla “elements” models, the face data applies to the face it is specified in or all faces of the element it is specified in which don’t have their own face data.

!!!note If **forge_data** is specified on a face, it will not inherit any parameters from the element-level **forge_data** declaration.

The additional data can be specified in the two ways shown in this example:

```
{
  "elements": [
    {
      "forge_data": {
        "color": "0xFFFF0000",
        "block_light": 15,
        "sky_light": 15,
        "ambient_occlusion": false
      },
      "faces": {
        "north": {
          "forge_data": {
            "color": "0xFFFF0000",
            "block_light": 15,
            "sky_light": 15,
            "ambient_occlusion": false
          },
          // ...
        },
        // ...
      },
      // ...
    },
    // ...
  ]
}
```

Generated Item Model

In item models generated using the **forge:item_layers** loader, face data is specified for each texture layer and applies to all of the geometry (front/back facing quads and edge quads).

The **forge_data** field must be located at the top level of the model JSON, with each key-value pair associating a face data object to a layer index.

In the following example, layer 1 will be tinted red and glow at full brightness:

```
{
  "textures": {
    "layer0": "minecraft:item/stick",
    "layer1": "minecraft:item/glowstone_dust"
  },
  "forge_data": {
    "1": {
      "color": "0xFFFF0000",
      "block_light": 15,
      "sky_light": 15,
      "ambient_occlusion": false
    }
  }
}
```

Parameters

Color

Specifying a color value with the **color** entry will apply that color as a tint to the quads. Defaults to 0xFFFFFFFF (white, fully opaque). The color must be in the **ARGB** format packed into a 32-bit integer and can be specified as either a hexadecimal string ("0xAARRGGBB") or as a decimal integer literal (JSON does not support hexadecimal integer literals).

!!! warning The four color components are multiplied with the texture's pixels. Omitting the alpha component is equivalent to making it 0, which will make the geometry fully transparent.

This can be used as a replacement for tinting with `BlockColor` and `ItemColor` if the color values are constant.

Block and Sky Light

Specifying a block and/or sky light value with the `block_light` and `sky_light` entry respectively will override the respective light value of the quads. Both values default to 0. The values must be in the range 0-15 (inclusive) and are treated as a minimum value for the respective light type when the face is rendered, meaning that a higher in-world value of the respective light type will override the specified value.

The specified light values are purely client-side and affect neither the server's light level nor the brightness of surrounding blocks.

Ambient Occlusion

Specifying the `ambient_occlusion` flag will configure AO for the quads. Defaults to `true`. The behaviour of this flag is equivalent to the top-level `ambientocclusion` flag of the vanilla format.

Ambient occlusion in action

Ambient occlusion enabled on the left and disabled on the right, demonstrated with the Smooth Lighting graphics setting

!!! note If the top-level AO flag is set to false, specifying this flag as true on an element or face won't be able to override the top-level flag.

```
js      {      "ambientocclusion": false,      "elements": [      {  
"forge_data": {      "ambient_occlusion": true // Has no effect      }      }  
]      }
```

Custom Model Loaders

A “model” is simply a shape. It can be a simple cube, it can be several cubes, it can be a truncated icosidodecahedron, or anything in between. Most models you'll see will be in the vanilla JSON format. Models in other formats are loaded into `IUnbakedGeometries` by an `IGeometryLoader` at runtime. Forge provides default implementations for WaveFront OBJ files, buckets, composite models, models in different render layers, and a reimplement of Vanilla's `builtin/generated` item model. Most things do not care about what loaded the model or what format it's in as they are all eventually represented by an `BakedModel` in code.

!!! warning Specifying a custom model loader through the top-level `loader` entry in a model JSON will cause the `elements` entry to be ignored unless it is consumed by the custom loader. All other vanilla entries will still be loaded and available in the unbaked `BlockModel` representation and may be consumed outside of the custom loader.

WaveFront OBJ Models

Forge adds a loader for the `.obj` file format. To use these models, the JSON must reference the `forge:obj` loader. This loader accepts any model location that is in a registered namespace and whose path ends in `.obj`. The `.mtl` file should be placed in the same location with the same name as the `.obj` to be used automatically. The `.mtl` file will probably have to be manually edited to change the paths pointing to textures defined within the JSON. Additionally, the V axis for textures may be flipped depending on the external program that created the model (i.e. V = 0 may be the bottom edge, not the top). This may be rectified in the modelling program itself or done in the model JSON like so:

```
{  
  // Add the following line on the same level as a 'model' declaration  
  "loader": "forge:obj",  
  "flip_v": true,  
  "model": "examplemod:models/block/model.obj",  
  "textures": {  
    // Can refer to in .mtl using #texture0  
    "texture0": "minecraft:block/dirt",  
    "particle": "minecraft:block/dirt"  
  }  
}
```



```
}
```

BakedModel

BakedModel is the result of calling **UnbakedModel#bake** for the vanilla model loader or **IUnbakedGeometry#bake** for custom model loaders. Unlike **UnbakedModel** or **IUnbakedGeometry**, which purely represents a shape without any concept of items or blocks, **BakedModel** is not as abstract. It represents geometry that has been optimized and reduced to a form where it is (almost) ready to go to the GPU. It can also process the state of an item or block to change the model.

In a majority of cases, it is not really necessary to implement this interface manually. One can instead use one of the existing implementations.

getOverrides

Returns the **ItemOverrides** to use for this model. This is only used if this model is being rendered as an item.

useAmbientOcclusion

If the model is rendered as a block in the level, the block in question does not emit any light, and ambient occlusion is enabled. This causes the model to be rendered with ambient occlusion.

isGui3d

If the model is rendered as an item in an inventory, on the ground as an entity, on an item frame, etc., this makes the model look “flat.” In GUIs, this also disables the lighting.

isCustomRenderer

!!! important Unless you know what you’re doing, just **return false** from this and continue on.

When rendering this as an item, returning **true** causes the model to not be rendered, instead falling back to **BlockEntityWithoutLevelRenderer#renderByItem**. For certain vanilla items such as chests and banners, this method is hardcoded to copy data from the item into a **BlockEntity**, before using a **BlockEntityRenderer** to render that BE in place of the item. For all other items, it will use the **BlockEntityWithoutLevelRenderer** instance provided by **IClientItemExtensions#getCustomRenderer**. Refer to **BlockEntityWithoutLevelRenderer** page for more information.

getParticleIcon

Whatever texture should be used for the particles. For blocks, this shows when an entity falls on it, when it breaks, etc. For items, this shows when it breaks or when it’s eaten.

!!! important The vanilla method with no parameters has been deprecated in favor of **#getParticleIcon(ModelData)** since model data can have an effect on how a particular model might be rendered.

getTransforms

Deprecated in favor of implementing **#applyTransform**. The default implementation is fine if **#applyTransform** is implemented. See **Transform**.

applyTransform

See **Transform**.

getQuads

This is the main method of **BakedModel**. It returns a list of **BakedQuads**: objects which contain the low-level vertex data that will be used to render the model. If the model is being rendered as a block, then the **BlockState** passed in is non-null. If the model is being rendered as an item, the **ItemOverrides** returned from **#getOverrides** is responsible for handling the state of the item, and the **BlockState** parameter will be null.

!!! note The origin point for the vertices in a **BakedQuad** is the bottom, northwest corner. Vertex coordinate values less than 0 or greater than 1 will position the vertex outside of the block. To avoid lighting issues, provide the vertices in counterclockwise order.

The **Direction** passed in is used for face culling. If the block against the given side of another block being rendered is opaque, then the faces associated with that side are not rendered. If that parameter is null, all faces not associated with a side are returned (that will never be culled).

The **rand** parameter is an instance of **Random**.

It also takes in a non null **ModelData** instance. This can be used to define extra data when rendering the specific model via **ModelProperty**s. For example, one such property is **CompositeModel\$Data**, which is used to store any additional submodel data for a model using the **forge:composite** model loader.

Note that this method is called very often: once for every combination of non-culled face and supported block render layer (anywhere between 0 to 28 times) *per block in a level*. This method should be as fast as possible, and should probably cache heavily.

Transform

When an **BakedModel** is being rendered as an item, it can apply special handling depending on which transform it is being rendered in. “Transform” means in what context the model is being rendered. The possible transforms are represented in code by the **ItemDisplayContext** enum. There are two systems for handling transform: the deprecated vanilla system, constituted by **BakedModel#getTransforms**, **ItemTransforms**, and **ItemTransform**, and the Forge system, embodied by the method **IForgeBakedModel#applyTransform**. The vanilla code is patched to favor using **applyTransform** over the vanilla system whenever possible.

ItemDisplayContext

NONE - Used for the display entity by default when no context is set and by Forge when a **Block**’s **RenderShape** is set to **#ENTITYBLOCK_ANIMATED**.

THIRD_PERSON_LEFT_HAND/THIRD_PERSON_RIGHT_HAND/FIRST_PERSON_LEFT_HAND/FIRST_PERSON_RIGHT_HAND - The first person values represent when the player is holding the item in their own hand. The third person values represent when another player is holding the item and the client is looking at them in the 3rd person. Hands are self-explanatory.

HEAD - Represents when any player is wearing the item in the helmet slot (e.g. pumpkins).

GUI - Represents when the item is being rendered in a **Screen**.

GROUND - Represents when the item is being rendered in the level as an **ItemEntity**.

FIXED - Used for item frames.

The Vanilla Way

The vanilla way of handling transform is through **BakedModel#getTransforms**. This method returns an **ItemTransforms**, which is a simple object that contains various **ItemTransforms** as public final fields. An **ItemTransform** represents a rotation, a translation, and a scale to be applied to the model. The **ItemTransforms** is a container for these, holding one for each of the **ItemDisplayContexts** except **NONE**. In the vanilla implementation, calling **#getTransform** for **NONE** results in the default transform, **ItemTransform#NO_TRANSFORM**.

The entire vanilla system for handling transforms is deprecated by Forge, and most implementations of `BakedModel` should simply return `ItemTransforms#NO_TRANSFORMS` (which is the default implementation) from `BakedModel#getTransforms`. Instead, they should implement `#applyTransform`.

The Forge Way

The Forge way of handling transforms is `#applyTransform`, a method patched into `BakedModel`. It supersedes the `#getTransforms` method.

BakedModel#applyTransform Given a `ItemDisplayContext`, `PoseStack`, and a boolean to determine whether to apply the transform for the left hand, this method produces an `BakedModel` to be rendered. Because the returned `BakedModel` can be a totally new model, this method is more flexible than the vanilla method (e.g. a piece of paper that looks flat in hand but crumpled on the ground).

ItemOverrides

`ItemOverrides` provides a way for an `BakedModel` to process the state of an `ItemStack` and return a new `BakedModel`; thereafter, the returned model replaces the old one. `ItemOverrides` represents an arbitrary function `(BakedModel, ItemStack, ClientLevel, LivingEntity, int) → BakedModel`, making it useful for dynamic models. In vanilla, it is used to implement item property overrides.

ItemOverrides()

Given a list of `ItemOverrides`, the constructor copies and bakes the list. The baked overrides may be accessed with `#getOverrides`.

resolve

This takes an `BakedModel`, an `ItemStack`, a `ClientLevel`, a `LivingEntity`, and an `int` to produce another `BakedModel` to use for rendering. This is where models can handle the state of their items.

This should not mutate the level.

getOverrides

Returns an immutable list containing all the `BakedOverrides` used by this `ItemOverrides`. If none are applicable, this returns the empty list.

BakedOverride

This class represents a vanilla item override, which holds several `ItemOverrides$PropertyMatcher` for the properties on an item and a model to use in case those matchers are satisfied. They are the objects in the `overrides` array of a vanilla item JSON model:

```
{
  // Inside a vanilla JSON item model
  "overrides": [
    {
      // This is an ItemOverride
      "predicate": {
        // This is the Map<ResourceLocation, Float>, containing the names of properties and their minimum values
        "example1:prop": 0.5
      },
      // This is the 'location', or target model, of the override, which is used if the predicate above matches
      "model": "example1:item/model"
    },
    {
      // This is another ItemOverride
      "predicate": {
        "example2:prop": 1
      },
      "model": "example2:item/model"
    }
  ]
}
```

```
}  
]  
}
```

Resource Packs

Resource Packs allow for the customization of client resources through the **assets** directory. This includes textures, models, sounds, localizations, and others. Your mod (as well as Forge itself) can also have resource packs. Any user can therefore modify all the textures, models, and other assets defined within this directory.

Creating a Resource Pack

Resource Packs are stored within your project's resources. The **assets** directory contains the contents of the pack, while the pack itself is defined by the **pack.mcmeta** alongside the **assets** folder. Your mod can have multiple asset domains, since you can add or modify already existing resource packs, like vanilla's, Forge's, or another mod's. You can then follow the steps found at the Minecraft Wiki to create any resource pack.

Additional reading: Resource Locations

Models

The model system is Minecraft's way of giving blocks and items their shapes. Through the model system, blocks and items are mapped to their models, which define how they look. One of the main goals of the model system is to allow not only textures but the entire shape of a block/item to be changed by resource packs. Indeed, any mod that adds items or blocks also contains a mini-resource pack for their blocks and items.

Model Files

Models and textures are linked through **ResourceLocations** but are stored in the **ModelManager** using **ModelResourceLocations**. Models are referenced in different locations through the block or item's registry name depending on whether they are referencing block states or item models. Blocks will have their **ModelResourceLocation** represent their registry name along with a stringified version of its current **BlockState** while items will use their registry name followed by **inventory**.

!!! note JSON models only support cuboid elements; there is no way to express a triangular wedge or anything like it. To have more complicated models, another format must be used.

Textures

Textures, like models, are contained within resource packs and are referred to with **ResourceLocations**. In Minecraft, the UV coordinates (0,0) are taken to mean the **top-left** corner. UVs are *always* from 0 to 16. If a texture is larger or smaller, the coordinates are scaled to fit. A texture should also be square, and the side length of a texture should be a power of two, as doing otherwise breaks mipmapping (e.g. 1x1, 2x2, 8x8, 16x16, and 128x128 are good. 5x5 and 30x30 are not recommended because they are not powers of 2. 5x10 and 4x8 are completely broken as they are not square.). Textures should only ever be not a square if it is animated.

Coloring Textures

Many blocks and items in vanilla change their texture color depending on where they are or what properties they have, such as grass. Models support specifying "tint indices" on faces, which are integers that can then be handled by **BlockColors** and **ItemColors**. See the wiki for information on how tint indices are defined in vanilla models.

BlockColor/ItemColor

Both of these are single-method interfaces. **BlockColor** takes a **BlockState**, a (nullable) **BlockAndTintGetter**, and a (nullable) **BlockPos**. **ItemColor** takes an **ItemStack**. Both of them take an **int** parameter **tintIndex**, which is the tint index of the face being colored. Both of them return an **int**, a color multiplier. This **int** is treated as 4

unsigned bytes, alpha, red, green, and blue, in that order, from most significant byte to least. For each pixel in the tinted face, the value of each color channel is $(\text{int})(\text{float } \text{base} * \text{multiplier} / 255.0)$, where **base** is the original value for the channel, and **multiplier** is the associated byte from the color multiplier. Note that blocks do not use the alpha channel. For example, the grass texture, untinted, looks white and gray. The **BlockColor** and **ItemColor** for grass return color multipliers with low red and blue components, but high alpha and green components, (at least in warm biomes) so when the multiplication is performed, the green is brought out and the red/blue diminished.

If an item inherits from the **builtin/generated** model, each layer (“layer0”, “layer1”, etc.) has a tint index corresponding to its layer index.

Creating Color Handlers

BlockColors need to be registered to the **BlockColors** instance of the game. **BlockColors** can be acquired through **RegisterColorHandlersEvent\$Block**, and an **BlockColor** can be registered by **#register**. Note that this does not cause the **BlockItem** for the given block to be colored. **BlockItems** are items and need to be colored with an **ItemColor**.

```
@SubscribeEvent
public void registerBlockColors(RegisterColorHandlersEvent.Block event){
    event.register(myBlockColor, coloredBlock1, coloredBlock2, ...);
}
```

ItemColors need to be registered to the **ItemColors** instance of the game. **ItemColors** can be acquired through **RegisterColorHandlersEvent\$Item**, and an **ItemColor** can be registered by **#register**. This method is overloaded to also take **Blocks**, which simply registers the color handler for the item **Block#asItem** (i.e. the block’s **BlockItem**).

```
@SubscribeEvent
public void registerItemColors(RegisterColorHandlersEvent.Item event){
    event.register(myItemColor, coloredItem1, coloredItem2, ...);
}
```

Item Properties

Item properties are a way for the “properties” of items to be exposed to the model system. An example is the bow, where the most important property is how far the bow has been pulled. This information is then used to choose a model for the bow, creating an animation for pulling it.

An item property assigns a certain **float** value to every **ItemStack** it is registered for, and vanilla item model definitions can use these values to define “overrides”, where an item defaults to a certain model, but if an override matches, it overrides the model and uses another. They are useful mainly because they are continuous. For example, bows use item properties to define their pull animation. The item models are decided by the ‘float’ number predicates, it is not limited but generally between 0.0F and 1.0F. This allows resource packs to add as many models as they want for the bow pulling animation along that spectrum, instead of being stuck with four “slots” for their models in the animation. The same is true of the compass and clock.

Adding Properties to Items

ItemProperties#register is used to add a property to a certain item. The **Item** parameter is the item the property is being attached to (e.g. **ExampleItems#APPLE**). The **ResourceLocation** parameter is the name given to the property (e.g. **new ResourceLocation("pull")**). The **ItemPropertyFunction** is a functional interface that takes the **ItemStack**, the **ClientLevel** it is in (may be null), the **LivingEntity** that holds it (may be null), and the **int** containing the id of the holding entity (may be 0), returning the **float** value for the property. For modded item properties, it is recommended that the mod id of the mod is used as the namespace (e.g. **examplemod:property** and not just **property**, as that really means **minecraft:property**). These should be done in **FMLClientSetupEvent**. There’s also another method **ItemProperties#registerGeneric** that is used to add properties to all items, and it does not take **Item** as its parameter since all items will apply this property.

!!! important Use `FMLClientSetupEvent#enqueueWork` to proceed with the tasks, since the data structures in `ItemProperties` are not thread-safe.

!!! note `ItemPropertyFunction` is deprecated by Mojang in favor of using the subinterface `ClampedItemPropertyFunction` which clamps the result between 0 and 1.

Using Overrides

The format of an override can be seen on the wiki, and a good example can be found in `model/item/bow.json`. For reference, here is a hypothetical example of an item with an `examplemod:power` property. If the values have no match, the default is the current model, but if there are multiple matches, the last match in the list will be selected.

!!! important A predicate applies to all values *greater than or equal to* the given value.

```
{
  "parent": "item/generated",
  "textures": {
    // Default
    "layer0": "examplemod:items/example_partial"
  },
  "overrides": [
    {
      // power >= .75
      "predicate": {
        "examplemod:power": 0.75
      },
      "model": "examplemod:item/example_powered"
    }
  ]
}
```

And here is a hypothetical snippet from the supporting code. Unlike the older versions (lower than 1.16.x), this needs to be done on client side only because `ItemProperties` does not exist on server.

```
private void setup(final FMLClientSetupEvent event)
{
  event.enqueueWork(() ->
  {
    ItemProperties.register(ExampleItems.APPLE,
      new ResourceLocation(ExampleMod.MODID, "pulling"), (stack, level, living, id) -> {
        return living != null && living.isUsingItem() && living.getItem() == stack ? 1.0F : 0.0F;
      });
  });
}
```

Datapacks

In 1.13, Mojang added datapacks to the base game. They allow for the modification of the files for logical servers through the `data` directory. This includes advancements, loot_tables, structures, recipes, tags, etc. Forge, and your mod, can also have datapacks. Any user can therefore modify all the recipes, loot tables, and other data defined within this directory.

Creating a Datapack

Datapacks are stored within the `data` directory within your project's resources. Your mod can have multiple data domains, since you can add or modify already existing datapacks, like vanilla's, forge's, or another mod's. You can then follow the steps found here to create any datapack.

Additional reading: Resource Locations

Recipes

Recipes are a way to transform some number of objects into other objects within a Minecraft world. Although the vanilla system deals purely with item transformations, the system as a whole can be expanded to use any object

the programmer creates.

Data-Driven Recipes

Most recipe implementations within vanilla are data driven via JSON. This means that a mod is not necessary to create a new recipe, only a Data pack. A full list on how to create and put these recipes within the mod's **resources** folder can be found on the Minecraft Wiki.

A recipe can be obtained within the Recipe Book as a reward for completing an advancement. Recipe advancements always have `minecraft:recipes/root` as their parent, to not to appear on the advancement screen. The default criteria to gain the recipe advancement is a check if the user has unlocked the recipe from using it once or receiving it through a command like `/recipe`:

```
// Within some recipe advancement json
"has_the_recipe": { // Criteria label
  // Succeeds if examplemod:example_recipe is used
  "trigger": "minecraft:recipe_unlocked",
  "conditions": {
    "recipe": "examplemod:example_recipe"
  }
}
//...
"requirements": [
  [
    "has_the_recipe"
    // ... Other criteria labels to be ORed against to unlock recipe
  ]
]
```

Data-driven recipes and their unlocking advancement can be generated via `RecipeProvider`.

Recipe Manager

Recipes are loaded and stored via the `RecipeManager`. Any operations relating to getting available recipe(s) are handled by this manager. There are two important methods to know of:

| Method | Description |
|----------------------------|---|
| <code>getRecipeFor</code> | Gets the first recipe that matches the current input. |
| <code>getRecipesFor</code> | Gets all recipes that match the current input. |

Each method takes in a `RecipeType`, which denotes what method is being applied to use the recipe (crafting, smelting, etc.), a `Container` which holds the configuration of the inputs, and the current level which is passed to `Recipe#matches` along with the container.

!!! important Forge provides the `RecipeWrapper` utility class which extends `Container` for wrapping around `IItemHandlers` and passing them to methods which requires a `Container` parameter.

```
```java
// Within some method with IItemHandlerModifiable handler
recipeManger.getRecipeFor(RecipeType.CRAFTING, new RecipeWrapper(handler), level);
```
```

Additional Features

Forge provides some additional behavior to the recipe schema and its implementations for greater control of the system.

Recipe ItemStack Result

Except for `minecraft:stonecutting` recipes, all vanilla recipe serializers expand the **result** tag to take in a full `ItemStack` as a `JsonObject` instead of just the item name and amount in some cases.

```
// In some recipe JSON
"result": {
  // The name of the registry item to give as a result
  "item": "examplemod:example_item",
  // The number of items to return
  "count": 4,
  // The tag data of the stack, can also be a string
  "nbt": {
    // Add tag data here
  }
}
```

!!! note The **nbt** tag can alternatively be a string containing a stringified NBT (or SNBT) for data which cannot be properly represented as a JSON object (such as `IntArrayTags`).

Conditional Recipes

Recipes and their unlocking advancement can be loaded conditionally and defaulted depending on what information is present (mod loaded, item exists, etc.).

Larger Crafting Grids

By default, vanilla declares a maximum width and height for a crafting grid to be a 3x3 square. This can be expanded by calling `ShapedRecipe#setCraftingSize` with the new width and height in `FMLCommonSetupEvent`.

!!! warning `ShapedRecipe#setCraftingSize` is **NOT** thread-safe. As such, it should be enqueued to the synchronous work queue via `FMLCommonSetupEvent#enqueueWork`.

Larger crafting grids in recipes can be data generated.

Ingredient Types

A few additional ingredient types are added to allow recipes to have inputs which check tag data or combine multiple ingredients into a single input checker.

Custom Recipes

Every recipe definition is made up of three components: the **Recipe** implementation which holds the data and handles the execution logic with the provided inputs, the **RecipeType** which represents the category or context the recipe will be used in, and the **RecipeSerializer** which handles decoding and network communication of the recipe data. How one chooses to use the recipe is up to the implementor.

Recipe

The **Recipe** interface describes the recipe data and the execution logic. This includes matching the inputs and providing the associated result. As the recipe subsystem performs item transformations by default, the inputs are supplied through a **Container** subtype.

!!! important The **Containers** passed into the recipe should be treated as if its contents were immutable. Any mutable operations should be performed on a copy of the input through `ItemStack#copy`.

To be able to obtain a recipe instance from the manager, `#matches` must return true. This method checks against the provided container to see whether the associated inputs are valid. **Ingredients** can be used for validation by calling `Ingredient#test`.

If the recipe has been chosen, it is then built using `#assemble` which may use data from the inputs to create the result.

!!! tip `#assemble` should always produce a unique `ItemStack`. If unsure whether `#assemble` does so, call `ItemStack#copy` on the result before returning.

Most of the other methods are purely for integration with the recipe book.


```
public record ExampleRecipe(Ingredient input, int data, ItemStack output) implements Recipe<Container> {
    // Implement methods here
}
```

!!! note While a record is used in the above example, it is not required to do so in your own implementation.

RecipeType

RecipeType is responsible for defining the category or context the recipe will be used within. For example, if a recipe was going to be smelted in a furnace, it would have a type of **RecipeType#SMELTING**. Being blasted in a blast furnace would have a type of **RecipeType#BLASTING**.

If none of the existing types match what context the recipe will be used within, then a new **RecipeType** must be registered.

The **RecipeType** instance must then be returned by **Recipe#getType** in the new recipe subtype.

```
// For some RegistryObject<RecipeType> EXAMPLE_TYPE
// In ExampleRecipe
@Override
public RecipeType<?> getType() {
    return EXAMPLE_TYPE.get();
}
```

RecipeSerializer

A **RecipeSerializer** is responsible for decoding JSONs and communicating across the network for an associated **Recipe** subtype. Each recipe decoded by the serializer is saved as a unique instance within the **RecipeManager**. A **RecipeSerializer** must be registered.

Only three methods need to be implemented for a **RecipeSerializer**:

| Method | Description |
|-------------|--|
| fromJson | Decodes a JSON into the Recipe subtype. |
| toNetwork | Encodes a Recipe to the buffer to send to the client. The recipe identifier does not need to be encoded. |
| fromNetwork | Decodes a Recipe from the buffer sent from the server. The recipe identifier does not need to be decoded. |

The **RecipeSerializer** instance must then be returned by **Recipe#getSerializer** in the new recipe subtype.

```
// For some RegistryObject<RecipeSerializer> EXAMPLE_SERIALIZER
// In ExampleRecipe
@Override
public RecipeSerializer<?> getSerializer() {
    return EXAMPLE_SERIALIZER.get();
}
```

!!! tip There are some useful methods to make reading and writing data for recipes easier. **Ingredients** can use **#fromJson**, **#toNetwork**, and **#fromNetwork** while **ItemStacks** can use **CraftingHelper#getItemStack**, **FriendlyByteBuf#writeItem**, and **FriendlyByteBuf#readItem**.

Building the JSON

Custom Recipe JSONs are stored in the same place as other recipes. The specified **type** should represent the registry name of the **recipe serializer**. Any additional data is specified by the serializer during decoding.

```
{
    // The custom serializer registry name
    "type": "examplemod:example_serializer",
    "input": {
        // Some ingredient input
    },
}
```

```

"data": 0, // Some data wanted for the recipe
"output": {
  // Some stack output
}
}

```

Non-Item Logic

If items are not used as part of the input or result of a recipe, then the normal methods provided in `RecipeManager` will not be useful. Instead, an additional method for testing a recipe's validity and/or supplying the result should be added to the custom `Recipe` instance. From there, all the recipes for that specific `RecipeType` can be obtained via `RecipeManager#getAllRecipesFor` and then checked and/or supplied the result using the newly implemented methods.

```

// In some Recipe subimplementation ExampleRecipe

// Checks the block at the position to see if it matches the stored data
boolean matches(Level level, BlockPos pos);

// Creates the block state to set the block at the specified position to
BlockState assemble(RegistryAccess access);

// In some manager class
public Optional<ExampleRecipe> getRecipeFor(Level level, BlockPos pos) {
    return level.getRecipeManager()
        .getAllRecipesFor(exampleRecipeType) // Gets all recipes
        .stream() // Looks through all recipes for types
        .filter(recipe -> recipe.matches(level, pos)) // Checks if the recipe inputs are valid
        .findFirst(); // Finds the first recipe whose inputs match
}

```

Data Generation

All custom recipes, regardless of input or output data, can be created into a `FinishedRecipe` for data generation using the `RecipeProvider`.

Ingredients

`Ingredients` are predicate handlers for item-based inputs which check whether a certain `ItemStack` meets the condition to be a valid input in a recipe. All vanilla recipes that take inputs use an `Ingredient` or a list of `Ingredients`, which is then merged into a single `Ingredient`.

Custom Ingredients

Custom ingredients can be specified by setting `type` to the name of the ingredient's serializer, with the exception of compound ingredients. When no type is specified, `type` defaults to the vanilla ingredient `minecraft:item`. Custom ingredients can also easily be used in data generation.

Forge Types

Forge provides a few additional `Ingredient` types for programmers to implement.

CompoundIngredient Though they are functionally identical, Compound ingredients replaces the way one would implement a list of ingredients would in a recipe. They work as a set OR where the passed in stack must be within at least one of the supplied ingredients. This change was made to allow custom ingredients to work correctly within lists. As such, **no type** needs to be specified.

```

// For some input
[
  // At least one of these ingredients must match to succeed
  {
    // Ingredient

```

```

},
{
  // Custom ingredient
  "type": "examplemod:example_ingredient"
}
]

```

StrictNBTPIngredient StrictNBTPIngredients compare the item, damage, and the share tags (as defined by `IForgeItem#getShareTag`) on an `ItemStack` for exact equivalency. This can be used by specifying the `type` as `forge:nbt`.

```

// For some input
{
  "type": "forge:nbt",
  "item": "examplemod:example_item",
  "nbt": {
    // Add nbt data (must match exactly what is on the stack)
  }
}

```

PartialNBTPIngredient

PartialNBTPIngredients are a looser version of **StrictNBTPIngredient** as they compare against a single or set of items and only keys specified within the share tag (as defined by `IForgeItem#getShareTag`). This can be used by specifying the `type` as `forge:partial_nbt`.

```

// For some input
{
  "type": "forge:partial_nbt",

  // Either 'item' or 'items' must be specified
  // If both are specified, only 'item' will be read
  "item": "examplemod:example_item",
  "items": [
    "examplemod:example_item",
    "examplemod:example_item2"
    // ...
  ],

  "nbt": {
    // Checks only for equivalency on 'key1' and 'key2'
    // All other keys in the stack will not be checked
    "key1": "data1",
    "key2": {
      // Data 2
    }
  }
}

```

IntersectionIngredient

IntersectionIngredients work as a set AND where the passed in stack must match all supplied ingredients. There must be at least two ingredients supplied to this. This can be used by specifying the `type` as `forge:intersection`.

```

// For some input
{
  "type": "forge:intersection",

  // All of these ingredients must return true to succeed
  "children": [
    {
      // Ingredient 1
    },
    {
      // Ingredient 2
    }
    // ...
  ]
}

```

```
}
```

DifferenceIngredient

DifferenceIngredients work as a set subtraction (SUB) where the passed in stack must match the first ingredient but must not match the second ingredient. This can be used by specifying the `type` as `forge:difference`.

```
// For some input
{
  "type": "forge:difference",
  "base": {
    // Ingredient the stack is in
  },
  "subtracted": {
    // Ingredient the stack is NOT in
  }
}
```

Creating Custom Ingredients

Custom ingredients can be created by implementing **IIngredientSerializer** for the created **Ingredient** subclass.

!!! tip Custom ingredients should subclass **AbstractIngredient** as it provides some useful abstractions for ease of implementation.

Ingredient Subclass

There are three important methods to implement for each ingredient subclass:

| Method | Description |
|----------------------------|---|
| <code>getSerializer</code> | Returns the serializer used to read and write the ingredient. |
| <code>test</code> | Returns true if the input is valid for this ingredient. |
| <code>isSimple</code> | Returns false if the ingredient matches on the stack's tag. AbstractIngredient subclasses will need to define this behavior, while Ingredient subclasses return <code>true</code> by default. |

All other defined methods are left as an exercise to the reader to use as required for the ingredient subclass.

IIngredientSerializer

IIngredientSerializer subtypes must implement three methods:

| Method | Description |
|------------------------------|---|
| <code>parse (JSON)</code> | Converts a JsonObject to an Ingredient . |
| <code>parse (Network)</code> | Reads the network buffer to decode an Ingredient . |
| <code>write</code> | Writes an Ingredient to the network buffer. |

Additionally, **Ingredient** subclasses should implement **Ingredient#toJson** for use with data generation. **AbstractIngredient** subclasses make **toJson** an abstract method requiring the method to be implemented.

Afterwards, a static instance should be declared to hold the initialized serializer and then registered using **CraftingHelper#register** either during the **RegisterEvent** for **RecipeSerializers** or during **FMLCommonSetupEvent**. The **Ingredient** subclass return the static instance of the serializer in **Ingredient#getSerializer**.

```
// In some serializer class
public static final ExampleIngredientSerializer INSTANCE = new ExampleIngredientSerializer();

// In some handler class
```

```

public void registerSerializers(RegisterEvent event) {
    event.register(ForgeRegistries.Keys.RECIPE_SERIALIZERS,
        helper -> CraftingHelper.register(registryName, INSTANCE)
    );
}

// In some ingredient subclass
@Override
public IIngredientSerializer<? extends Ingredient> getSerializer() {
    return INSTANCE;
}

```

!!! tip If using FMLCommonSetupEvent to register an ingredient serializer, it must be enqueued to the synchronous work queue via FMLCommonSetupEvent#enqueueWork as CraftingHelper#register is not thread-safe.

Non-Datapack Recipes

Not all recipes are simplistic enough or migrated to using data-driven recipes. Some subsystems still need to be patched within the codebase to provide support for adding new recipes.

Brewing Recipes

Brewing is one of the few recipes that still exist in code. Brewing recipes are added as part of a bootstrap within PotionBrewing for their containers, container recipes, and potion mixes. To expand upon the existing system, Forge allows brewing recipes to be added by calling BrewingRecipeRegistry#addRecipe in FMLCommonSetupEvent.

!!! warning BrewingRecipeRegistry#addRecipe must be called within the synchronous work queue via #enqueueWork as the method is not thread-safe.

The default implementation takes in an input ingredient, a catalyst ingredient, and a stack output for a standard implementation. Additionally, an IBrewingRecipe instance can be supplied instead to do the transformations.

IBrewingRecipe

IBrewingRecipe is a pseudo-Recipe interface that checks whether the input and catalyst is valid and provides the associated output if so. This is provided through #isInput, #isIngredient, and #getOutput respectively. The output method has access to the input and catalyst stacks to construct the result.

!!! important When copying data between ItemStacks or CompoundTags, make sure to use their respective #copy methods to create unique instances.

There is no wrapper for adding additional potion containers or potion mixes similar to vanilla. A new IBrewingRecipe implementation will need to be added to replicate this behavior.

Anvil Recipes

Anvils are responsible for taking a damaged input and given some material or a similar input, remove some of the damage on the input result. As such, its system is not easily data-driven. However, as anvil recipes are an input with some number of materials equals some output when the user has the required experience levels, it can be modified to create a pseudo-recipe system via AnvilUpdateEvent. This takes in the input and materials and allows the modder to specify the output, experience level cost, and number of materials to use for the output. The event can also prevent any output by canceling it.

```

// Checks whether the left and right items are correct
// When true, sets the output, level experience cost, and material amount
public void updateAnvil(AnvilUpdateEvent event) {
    if (event.getLeft().is(...) && event.getRight().is(...)) {
        event.setOutput(...);
        event.setCost(...);
        event.setMaterialCost(...);
    }
}

```

The update event must be attached to the Forge event bus.

Loom Recipes

Looms are responsible for applying a dye and pattern (either from the loom or from an item) to a banner. While the banner and the dye must be a `BannerItem` or `DyeItem` respectively, custom patterns can be created and applied in the loom. Banner Patterns can be created by registering a `BannerPattern`.

!!! important `BannerPatterns` which are in the `minecraft:no_item_required` tag appear as an option in the loom. Patterns not in this tag must have an accompanying `BannerPatternItem` to be used along with an associated tag.

```
private static final DeferredRegister<BannerPattern> REGISTER = DeferredRegister.create(Registries.BANNER_PATTERN,
"examplemod");

// Takes in the pattern name to send over the network
public static final BannerPattern EXAMPLE_PATTERN = REGISTER.register("example_pattern", () -> new
BannerPattern("examplemod:ep"));
```

Loot Tables

Loot tables are logic files which dictate what should happen when various actions or scenarios occur. Although the vanilla system deals purely with item generation, the system can be expanded to perform any number of defined actions.

Data-Driven Tables

Most loot tables within vanilla are data driven via JSON. This means that a mod is not necessary to create a new loot table, only a Data pack. A full list on how to create and put these loot tables within the mod's `resources` folder can be found on the Minecraft Wiki.

Using a Loot Table

A loot table is referenced by its `ResourceLocation` which points to `data/<namespace>/loot_tables/<path>.json`. The `LootTable` associated with the reference can be obtained using `LootDataResolver#getLootTable`, where `LootDataResolver` can be obtained via `MinecraftServer#getLootData`.

A loot table is always generated with given parameters. The `LootParams` contains the level the table is generated in, luck for better generation, the `LootContextParams` which define scenario context, and any dynamic information that should occur on activation. The `LootParams` can be created using the constructor of the `LootParams$Builder` builder, and built via `LootParams$Builder#create` by passing in the `LootContextParamSet`.

A loot table may also have some context. The `LootContext` takes in the built `LootParams` and can set some random seeded instance. The context is created via the builder `LootContext$Builder` and built using `LootContext$Builder#create` by passing in a nullable `ResourceLocation` representing the random instance to use.

A `LootTable` can be used to generate `ItemStacks` using one of the available methods which may take in a `LootParams` or a `LootContext`:

| Method | Description |
|--------------------------------|--|
| <code>getRandomItemsRaw</code> | Consumes the items generated by the loot table. |
| <code>getRandomItems</code> | Returns the items generated by the loot table. |
| <code>fill</code> | Fills a container with the generated loot table. |

!!! note Loot tables were built for generating items, so the methods expect some handling for the `ItemStacks`.

Additional Features

Forge provides some additional behavior to loot tables for greater control of the system.

LootTableLoadEvent

LootTableLoadEvent is an event fired on the Forge event bus which is fired whenever a loot table is loaded. If the event is canceled, then an empty loot table will be loaded instead.

!!! important Do **not** modify a loot table's drops through this event. Those modifications should be done using global loot modifiers.

Loot Pool Names

Loot pools can be named using the **name** key. Any non-named loot pool will be the hash code of the pool prefixed by **custom#**.

```
// For some loot pool
{
  "name": "example_pool", // Pool will be named 'example_pool'
  "rolls": {
    // ...
  },
  "entries": {
    // ...
  }
}
```

Looting Modifiers

Loot tables are now affected by the **LootingLevelEvent**, on the Forge event bus, in addition to the looting enchantment.

Additional Context Parameters

Forge extends certain parameter sets to account for missing contexts which may be applicable. **LootContextParamSets#CHEST** now allows for a **LootContextParams#KILLER_ENTITY** as chest minecarts are entities which can be broken (or 'killed'). **LootContextParamSets#FISHING** also allows for a **LootContextParams#KILLER_ENTITY** since the fishing hook is also an entity which is retracted (or 'killed') when the player retrieves it.

Multiple Items on Smelting

When using the **SmeltItemFunction**, a smelted recipe will now return the actual number of items from the result instead of a single smelted item (e.g. if a smelting recipe returns 3 items and there are 3 drops, then the result would be 9 smelted items instead of 3).

Loot Table Id Condition

Forge adds an additional **LootItemCondition** which allows certain items to generate for a specific table. This is typically used within global loot modifiers.

```
// In some loot pool or pool entry
{
  "conditions": [
    {
      "condition": "forge:loot_table_id",
      // Will apply when the loot table is for dirt
      "loot_table_id": "minecraft:blocks/dirt"
    }
  ]
}
```

Can Tool Perform Action Condition

Forge adds an additional **LootItemCondition** which checks whether the given **LootContextParams#TOOL** can perform the specified **ToolAction**.

```
// In some loot pool or pool entry
{
  "conditions": [
    {
      "condition": "forge:can_tool_perform_action",
      // Will apply when the tool can strip a log like an axe
      "action": "axe_strip"
    }
  ]
}
```

Global Loot Modifiers

Global Loot Modifiers are a data-driven method of handling modification of harvested drops without the need to overwrite dozens to hundreds of vanilla loot tables or to handle effects that would require interactions with another mod's loot tables without knowing what mods may be loaded. Global Loot Modifiers are also stacking, rather than last-load-wins, similar to tags.

Registering a Global Loot Modifier

You will need 4 things:

1. Create a `global_loot_modifiers.json`.
 - This will tell Forge about your modifiers and works similar to tags.
2. A serialized json representing your modifier.
 - This will contain all of the data about your modification and allows data packs to tweak your effect.
3. A class that extends `IGlobalLootModifier`.
 - The operational code that makes your modifier work. Most modders can extend `LootModifier` as it supplies base functionality.
4. Finally, a codec to encode and decode your operational class.
 - This is registered as any other `IForgeRegistryEntry`.

The `global_loot_modifiers.json`

The `global_loot_modifiers.json` represents all loot modifiers to be loaded into the game. This file **MUST** be placed within `data/forge/loot_modifiers/global_loot_modifiers.json`.

!!! important `global_loot_modifiers.json` will only be read in the `forge` namespace. The file will be neglected if it is under the mod's namespace.

`entries` is an *ordered list* of the modifiers that will be loaded. The `ResourceLocations` specified points to their associated entry within `data/<namespace>/loot_modifiers/<path>.json`. This is primarily relevant to data pack makers for resolving conflicts between modifiers from separate mods.

`replace`, when `true`, changes the behavior from appending loot modifiers to the global list to replacing the global list entries entirely. Modders will want to use `false` for compatibility with other mod implementations. Datapack makers may want to specify their overrides with `true`.

```
{
  "replace": false, // Must be present
  "entries": [
    // Represents a loot modifier in 'data/examplemod/loot_modifiers/example_glm.json'
    "examplemod:example_glm",
    "examplemod:example_glm2"
    // ...
  ]
}
```

The Serialized JSON

This file contains all of the potential variables related to your modifier, including the conditions that must be met prior to modifying any loot. Avoid hard-coded values wherever possible so that data pack makers can adjust balance

if they wish to.

type represents the registry name of the codec used to read the associated JSON file. This must always be present. **conditions** should represent the loot table conditions for this modifier to activate. Conditions should avoid being hardcoded to allow datapack creators as much flexibility to adjust the criteria. This must also be always present.

!!! important Although **conditions** should represent what is needed for the modifier to activate, this is only the case if using the bundled Forge classes. If using **LootModifier** as a subclass, all conditions will be **ANDed** together and checked to see if the modifier should be applied.

Any additional properties read by the serializer and defined by the modifier can also be specified.

```
// Within data/examplemod/loot_modifiers/example_glm.json
{
  "type": "examplemod:example_loot_modifier",
  "conditions": [
    // Normal loot table conditions
    // ...
  ],
  "prop1": "val1",
  "prop2": 10,
  "prop3": "minecraft:dirt"
}
```

IGlobalLootModifier

To supply the functionality a global loot modifier specifies, a **IGlobalLootModifier** implementation must be specified. These are instances generated each time a serializer decodes the information from JSON and supplies it into this object.

There are two methods that needs to be defined in order to create a new modifier: **#apply** and **#codec**. **#apply** takes in the current loot that will be generated along with the context information such as the currently level or additional defined parameters. It returns the list of drops to generate.

!!! note The returned list of drops from any one modifier is fed into other modifiers in the order they are registered. As such, modified loot can be modified by another loot modifier.

#codec returns the registered codec used to encode and decode the modifier to/from JSON.

The LootModifier Subclass

LootModifier is an abstract implementation of **IGlobalLootModifier** to provide the base functionality which most modders can easily extend and implement. This expands upon the existing interface by defining the **#apply** method to check the conditions to determine whether or not to modify the generated loot.

There are two things of note within the subclass implementation: the constructor which must take in an array of **LootItemConditions** and the **#doApply** method.

The array of **LootItemConditions** define the list of conditions that must be true before the loot can be modified. The supplied conditions are **ANDed** together, meaning that all conditions must be true.

The **#doApply** method works the same as the **#apply** method except that it only executes once all conditions return true.

```
public class ExampleModifier extends LootModifier {

  public ExampleModifier(LootItemCondition[] conditionsIn, String prop1, int prop2, Item prop3) {
    super(conditionsIn);
    // Store the rest of the parameters
  }

  @NotNull
  @Override
  protected ObjectArrayList<ItemStack> doApply(ObjectArrayList<ItemStack> generatedLoot, LootContext context) {
    // Modify the loot and return the new drops
  }
}
```

```

@Override
public Codec<? extends IGlobalLootModifier> codec() {
    // Return the codec used to encode and decode this modifier
}
}

```

The Loot Modifier Codec

The connector between the JSON and the `IGlobalLootModifier` instance is a `Codec<T>`, where `T` represents the type of the `IGlobalLootModifier` to use.

For ease of convenience, a loot conditions codec has been provided for an easy addition to a record-like codec via `LootModifier#codecStart`. This is utilized for data generation of the associated loot modifier.

```

// For some DeferredRegister<Codec<? extends IGlobalLootModifier>> REGISTRAR
public static final RegistryObject<Codec<ExampleModifier>> = REGISTRAR.register("example_codec", () ->
    RecordCodecBuilder.create(
        inst -> LootModifier.codecStart(inst).and(
            inst.group(
                Codec.STRING.fieldOf("prop1").forGetter(m -> m.prop1),
                Codec.INT.fieldOf("prop2").forGetter(m -> m.prop2),
                ForgeRegistries.ITEMS.getCodec().fieldOf("prop3").forGetter(m -> m.prop3)
            )
        ).apply(inst, ExampleModifier::new)
    )
);

```

Examples can be found on the Forge Git repository, including silk touch and smelting effects.

Tags

Tags are generalized sets of objects in the game used for grouping related things together and providing fast membership checks.

Declaring Your Own Groupings

Tags are declared in your mod's datapack. For example, a `TagKey<Block>` with a given identifier of `modid:foo/tagname` will reference a tag at `/data/<modid>/tags/blocks/foo/tagname.json`. Tags for `Blocks`, `Items`, `EntityTypes`, `Fluids`, and `GameEvents` use the plural forms for their folder location while all other registries use the singular version (`EntityType` uses the folder `entity_types` while `Potion` would use the folder `potion`). Similarly, you may append to or override tags declared in other domains, such as Vanilla, by declaring your own JSONs. For example, to add your own mod's saplings to the Vanilla sapling tag, you would specify it in `/data/minecraft/tags/blocks/saplings.json`, and Vanilla will merge everything into one tag at reload, if the `replace` option is false. If `replace` is true, then all entries before the json specifying `replace` will be removed. Values listed that are not present will cause the tag to error unless the value is listed using an `id` string and `required` boolean set to false, as in the following example:

```

{
  "replace": false,
  "values": [
    "minecraft:gold_ingot",
    "mymod:my_ingot",
    {
      "id": "othermod:ingot_other",
      "required": false
    }
  ]
}

```

See the Vanilla wiki for a description of the base syntax.

There is also a Forge extension on the Vanilla syntax. You may declare a `remove` array of the same format as the `values` array. Any values listed here will be removed from the tag. This acts as a finer grained version of the Vanilla `replace` option.

Using Tags In Code

Tags for all registries are automatically sent from the server to any remote clients on login and reload. **Blocks**, **Items**, **EntityTypes**, **Fluids**, and **GameEvents** are special cased as they have **Holders** allowing for available tags to be accessible through the object itself.

!!! note Intrusive **Holders** may be removed in a future version of Minecraft. If they are, the below methods can be used instead to query the associated **Holders**.

ITagManager

Forge wrapped registries provide an additional helper for creating and managing tags through **ITagManager** which can be obtained via **IForgeRegistry#tags**. Tags can be created using **#createTagKey** or **#createOptionalTagKey**. Tags or registry objects can also be checked for either or using **#getTag** or **#getReverseTag** respectively.

Custom Registries Custom registries can create tags when constructing their **DeferredRegister** via **#createTagKey** or **#createOptionalTagKey** respectively. Their tags or registry objects can then checked for either using the **IForgeRegistry** obtained by calling **DeferredRegister#makeRegistry**.

Referencing Tags

There are four methods of creating a tag wrapper:

| Method | For |
|--------------------------------------|---|
| *Tags#create | BannerPattern , Biome , Block , CatVariant , DamageType , EntityType , FlatLevelGeneratorPreset , Fluid , GameEvent , Instrument , Item , PaintingVariant , PoiType , Structure , and WorldPreset where * represents one of these types. |
| ITagManager#createTagKey | Forge wrapped vanilla registries, registries can be obtained from ForgeRegistries . |
| DeferredRegister#createTagKey | Custom forge registries. |
| TagKey#create | Vanilla registries without forge wrappers, registries can be obtained from Registry . |

Registry objects can check their tags or registry objects either through their **Holder** or through **ITag/IReverseTag** for vanilla or forge registry objects respectively.

Vanilla registry objects can grab their associated holder using either **Registry#getHolder** or **Registry#getHolderOrThrow** and then compare if the registry object has a tag using **Holder#is**.

Forge registry objects can grab their tag definition using either **ITagManager#getTag** or **ITagManager#getReverseTag** and then compare if a registry object has a tag using **ITag#contains** or **IReverseTag#containsTag** respectively.

Tag-holding registry objects contain a method called **#is** in either their registry object or state-aware class to check whether the object belongs to a certain tag.

As an example:

```
public static final TagKey<Item> myItemTag = ItemTags.create(new ResourceLocation("mymod", "myitemgroup"));

public static final TagKey<Potion> myPotionTag = ForgeRegistries.POTIONS.tags().createTagKey(new
ResourceLocation("mymod", "mypotiongroup"));

public static final TagKey<VillagerType> myVillagerTypeTag = TagKey.create(Registries.VILLAGER_TYPE, new
ResourceLocation("mymod", "myvillagertypegroup"));

// In some method:

ItemStack stack = /*...*/;
boolean isInItemGroup = stack.is(myItemTag);
```

```
Potion potion = /*...*/;
boolean isInPotionGroup = ForgeRegistries.POTIONS.tags().getTag(myPotionTag).contains(potion);

ResourceKey<VillagerType> villagerTypeKey = /*...*/;
boolean isInVillagerTypeGroup = BuiltInRegistries.VILLAGER_TYPE.getHolder(villagerTypeKey).map(holder ->
holder.is(myVillagerTypeTag)).orElse(false);
```

Conventions

There are several conventions that will help facilitate compatibility in the ecosystem:

- If there is a Vanilla tag that fits your block or item, add it to that tag. See the list of Vanilla tags.
- If there is a Forge tag that fits your block or item, add it to that tag. The list of tags declared by Forge can be seen on GitHub.
- If there is a group of something you feel should be shared by the community, use the **forge** namespace instead of your mod id.
- Tag naming conventions should follow Vanilla conventions. In particular, item and block groupings are plural instead of singular (e.g. `minecraft:logs`, `minecraft:saplings`).
- Item tags should be sorted into subdirectories according to their type (e.g. `forge:ingots/iron`, `forge:nuggets/brass`, etc.).

Migration from OreDictionary

- For recipes, tags can be used directly in the vanilla recipe format (see below).
- For matching items in code, see the section above.
- If you are declaring a new type of item grouping, follow a couple naming conventions:
 - Use `domain:type/material`. When the name is a common one that all modders should adopt, use the **forge** domain.
 - For example, brass ingots should be registered under the `forge:ingots/brass` tag and cobalt nuggets under the `forge:nuggets/cobalt` tag.

Using Tags in Recipes and Advancements

Tags are directly supported by Vanilla. See the respective Vanilla wiki pages for recipes and advancements for usage details.

Advancements

Advancements are tasks that can be achieved by the player which may advance the progress of the game. Advancements can trigger based on any action the player may be directly involved in.

All advancement implementations within vanilla are data driven via JSON. This means that a mod is not necessary to create a new advancement, only a data pack. A full list on how to create and put these advancements within the mod's **resources** can be found on the Minecraft Wiki. Additionally, advancements can be loaded conditionally and defaulted depending on what information is present (mod loaded, item exists, etc.).

Advancement Criteria

To unlock an advancement, the specified criteria must be met. Criteria are tracked through triggers which execute when a certain action is performed: killing an entity, changing an inventory, breeding animals, etc. Any time an advancement is loaded into the game, the criteria defined are read and added as listeners to the trigger. Afterwards a trigger function is called (usually named **#trigger**) which checks all listeners as to whether the current state meets the conditions of the advancement criteria. The criteria listeners for the advancement are only removed once the advancement has been obtained by completing all requirements.

Requirements are defined as an array of string arrays representing the name of the criteria specified on the advancement. An advancement is completed once one string array of criteria has been met:

```
// In some advancement JSON

// List of defined criteria to meet
"criteria": {
  "example_criterion1": { /*...*/ },
  "example_criterion2": { /*...*/ },
  "example_criterion3": { /*...*/ },
  "example_criterion4": { /*...*/ }
},

// This advancement is only unlocked once
// - Criteria 1 AND 2 have been met
// OR
// - Criteria 3 and 4 have been met
"requirements": [
  [
    "example_criterion1",
    "example_criterion2"
  ],
  [
    "example_criterion3",
    "example_criterion4"
  ]
]
```

A list of criteria triggers defined by vanilla can be found in `CriteriaTriggers`. Additionally, the JSON formats are defined on the Minecraft Wiki.

Custom Criteria Triggers

Custom criteria triggers can be created by implementing `SimpleCriterionTrigger` for the created `AbstractCriterionTriggerInstance` subclass.

AbstractCriterionTriggerInstance Subclass

The `AbstractCriterionTriggerInstance` represents a single criteria defined in the `criteria` object. Trigger instances are responsible for holding the defined conditions, returning whether the inputs match the condition, and writing the instance to JSON for data generation.

Conditions are usually passed in through the constructor. The `AbstractCriterionTriggerInstance` super constructor requires the instance to define the registry name of the trigger and the conditions the player must meet as an `ContextAwarePredicate`. The registry name of the trigger should be supplied to the super directly while the conditions of the player should be a constructor parameter.

```
// Where ID is the registry name of the trigger
public ExampleTriggerInstance(ContextAwarePredicate player, ItemPredicate item) {
    super(ID, player);
    // Store the item condition that must be met
}
```

!!! note Typically, trigger instances have a static constructor which allow these instances to be easily created for data generation. These static factory methods can also be statically imported instead of the class itself.

```
```java
public static ExampleTriggerInstance instance(ContextAwarePredicate player, ItemPredicate item) {
 return new ExampleTriggerInstance(player, item);
}
```
```

Additionally, the `#serializeToJson` method should be overridden. The method should add the conditions of the instance to the other JSON data.

```
@Override
public JsonObject serializeToJson(SerializationContext context) {
    JsonObject obj = super.serializeToJson(context);
    // Write conditions to json
    return obj;
}
```

```
}
```

Finally, a method should be added which takes in the current data state and returns whether the user has met the necessary conditions. The conditions of the player are already checked through `SimpleCriterionTrigger#trigger(ServerPlayer, Predicate)`. Most trigger instances call this method `#matches`.

```
// This method is unique for each instance and is as such not overridden
public boolean matches(ItemStack stack) {
    // Since ItemPredicate matches a stack, a stack is the input
    return this.item.matches(stack);
}
```

SimpleCriterionTrigger

The `SimpleCriterionTrigger<T>` subclass, where `T` is the type of the trigger instance, is responsible for specifying the registry name of the trigger, creating a trigger instance, and a method to check trigger instances and run attached listeners on success.

The registry name of the trigger is supplied to `#getId`. This should match the registry name supplied to the trigger instance.

A trigger instance is created via `#createInstance`. This method reads a criteria from JSON.

```
@Override
public ExampleTriggerInstance createInstance(JsonObject json, ContextAwarePredicate player, DeserializationContext context) {
    // Read conditions from JSON: item
    return new ExampleTriggerInstance(player, item);
}
```

Finally, a method is defined to check all trigger instances and run the listeners if their condition is met. This method takes in the `ServerPlayer` and whatever other data defined by the matching method in the `AbstractCriterionTriggerInstance` subclass. This method should internally call `SimpleCriterionTrigger#trigger` to properly handle checking all listeners. Most trigger instances call this method `#trigger`.

```
// This method is unique for each trigger and is as such not overridden
public void trigger(ServerPlayer player, ItemStack stack) {
    this.trigger(player,
        // The condition checker method within the AbstractCriterionTriggerInstance subclass
        triggerInstance -> triggerInstance.matches(stack)
    );
}
```

Afterwards, an instance should be registered using `CriteriaTriggers#register` during `FMLCommonSetupEvent`.

!!! important `CriteriaTriggers#register` must be enqueued to the synchronous work queue via `FMLCommonSetupEvent#enqueueWork` as the method is not thread-safe.

Calling the Trigger

Whenever the action being checked is performed, the `#trigger` method defined by the `SimpleCriterionTrigger` subclass should be called.

```
// In some piece of code where the action is being performed
// Where EXAMPLE_CRITERIA_TRIGGER is the custom criteria trigger
public void performExampleAction(ServerPlayer player, ItemStack stack) {
    // Run code to perform action
    EXAMPLE_CRITERIA_TRIGGER.trigger(player, stack);
}
```

Advancement Rewards

When an advancement is completed, rewards may be given out. These can be a combination of experience points, loot tables, recipes for the recipe book, or a function executed as a creative player.

```
// In some advancement JSON
"rewards": {
  "experience": 10,
  "loot": [
    "minecraft:example_loot_table",
    "minecraft:example_loot_table2"
    // ...
  ],
  "recipes": [
    "minecraft:example_recipe",
    "minecraft:example_recipe2"
    // ...
  ],
  "function": "minecraft:example_function"
}
```

Conditionally-Loaded Data

There are times when modders may want to include data-driven objects using information from another mod without having to explicitly make that mod a dependency. Other cases may be to swap out certain objects with other modded entries when they are present. This can be done through the conditional subsystem.

Implementations

Currently, conditional loading is implemented for recipes and advancements. For any conditional recipe or advancement, a list of conditions to datum pair is loaded. If the conditions specified for a datum in the list is true, then that datum is returned. Otherwise, the datum is discarded.

```
{
  // The type needs to be specified for recipes as they can have custom serializers
  // Advancements do not need this type
  "type": "forge:conditional",

  "recipes": [ // Or 'advancements' for Advancements
    {
      // The conditions to check
      "conditions": [
        // Conditions in the list are ANDed together
        {
          // Condition 1
        },
        {
          // Condition 2
        }
      ],
      "recipe": { // Or 'advancement' for Advancements
        // The recipe to use if all conditions succeed
      }
    },
    {
      // Next condition to check if the previous fails
    },
  ]
}
```

Conditionally-loaded data additionally have wrappers for data generation through `ConditionalRecipe$Builder` and `ConditionalAdvancement$Builder`.

Conditions

Conditions are specified by setting `type` to the name of the condition as specified by `IConditionSerializer#getID`.

True and False

Boolean conditions consist of no data and return the expected value of the condition. They are represented by `forge:true` and `forge:false`.

```
// For some condition
{
  // Will always return true (or false for 'forge:false')
  "type": "forge:true"
}
```

Not, And, and Or

Boolean operator conditions consist of the condition(s) being operated upon and apply the following logic. They are represented by `forge:not`, `forge:and`, and `forge:or`.

```
// For some condition
{
  // Inverts the result of the stored condition
  "type": "forge:not",
  "value": {
    // A condition
  }
}
```

```
// For some condition
{
  // ANDs the stored conditions together (or ORs for 'forge:or')
  "type": "forge:and",
  "values": [
    {
      // First condition
    },
    {
      // Second condition to be ANDed (or ORed for 'forge:or')
    }
  ]
}
```

Mod Loaded

`ModLoadedCondition` returns true whenever the specified mod with the given id is loaded in the current application. This is represented by `forge:mod_loaded`.

```
// For some condition
{
  "type": "forge:mod_loaded",
  // Returns true if 'examplemod' is loaded
  "modid": "examplemod"
}
```

Item Exists

`ItemExistsCondition` returns true whenever the given item has been registered in the current application. This is represented by `forge:item_exists`.

```
// For some condition
{
  "type": "forge:item_exists",
  // Returns true if 'examplemod:example_item' has been registered
  "item": "examplemod:example_item"
}
```

Tag Empty

`TagEmptyCondition` returns true whenever the given item tag has no items within it. This is represented by `forge:tag_empty`.


```
// For some condition
{
    "type": "forge:tag_empty",
    // Returns true if 'examplemod:example_tag' is an item tag with no entries
    "tag": "examplemod:example_tag"
}
```

Creating Custom Conditions

Custom conditions can be created by implementing `ICondition` and its associated `IConditionSerializer`.

ICondition

Any condition only need to implement two methods:

| Method | Description |
|--------------------|--|
| <code>getID</code> | The registry name of the condition. Must be equivalent to <code>IConditionSerializer#getID</code> . Used only for data generation. |
| <code>test</code> | Returns true if the condition has been satisfied. |

!!! note Every `#test` has access to some `IContext` representing the state of the game. Currently, only tags can be obtained from a registry.

IConditionSerializer

Serializers need to implement three methods:

| Method | Description |
|--------------------|---|
| <code>getID</code> | The registry name of the condition. Must be equivalent to <code>ICondition#getID</code> . |
| <code>read</code> | Reads the condition data from JSON. |
| <code>write</code> | Writes the given condition data to JSON. |

!!! note Condition serializers are not responsible for writing or reading the type of the serializer, similar to other serializer implementations in Minecraft.

Afterwards, a static instance should be declared to hold the initialized serializer and then registered using `CraftingHelper#register` either during the `RegisterEvent` for `RecipeSerializers` or during `FMLCommonSetupEvent`.

```
// In some serializer class
public static final ExampleConditionSerializer INSTANCE = new ExampleConditionSerializer();

// In some handler class
public void registerSerializers(RegisterEvent event) {
    event.register(ForgeRegistries.Keys.RECIPE_SERIALIZERS,
        helper -> CraftingHelper.register(INSTANCE)
    );
}
```

!!! important If using `FMLCommonSetupEvent` to register a condition serializer, it must be enqueued to the synchronous work queue via `FMLCommonSetupEvent#enqueueWork` as `CraftingHelper#register` is not thread-safe.

Data Generators

Data generators are a way to programmatically generate the assets and data of mods. It allows the definition of the contents of these files in the code and their automatic generation, without worrying about the specifics.

The data generator system is loaded by the main class `net.minecraft.data.Main`. Different command-line arguments can be passed to customize which mods' data are gathered, what existing files are considered, etc. The class responsible for data generation is `net.minecraft.data.DataGenerator`.

The default configurations in the MDK `build.gradle` adds the `runData` task for running the data generators.

Existing Files

All references to textures or other data files not generated for data generation must reference existing files on the system. This is to ensure that all referenced textures are in the correct places, so typos can be found and corrected.

`ExistingFileHelper` is the class responsible for validating the existence of those data files. An instance can be retrieved from `GatherDataEvent#getExistingFileHelper`.

The `--existing <folderpath>` argument allows the specified folder and its subfolders to be used when validating the existence of files. Additionally, the `--existing-mod <modid>` argument allows the resources of a loaded mod to be used for validation. By default, only the vanilla datapack and resources are available to the `ExistingFileHelper`.

Generator Modes

The data generator can be configured to run 4 different data generations, which are configured from the command-line parameters, and can be checked from `GatherDataEvent#include***` methods.

- **Client Assets**
 - Generates client-only files in `assets`: block/item models, blockstate JSONs, language files, etc.
 - `--client, #includeClient`
- **Server Data**
 - Generates server-only files in `data`: recipes, advancements, tags, etc.
 - `--server, #includeServer`
- **Development Tools**
 - Runs some development tools: converting SNBT to NBT and vice-versa, etc.
 - `--dev, #includeDev`
- **Reports**
 - Dumps all registered blocks, items, commands, etc.
 - `--reports, #includeReports`

All of the generators can be included using `--all`.

Data Providers

Data providers are the classes that actually define what data will be generated and provided. All data providers implement `DataProvider`. Minecraft has abstract implementations for most assets and data, so modders need only to extend and override the specified method.

The `GatherDataEvent` is fired on the mod event bus when the data generator is being created, and the `DataGenerator` can be obtained from the event. Create and register data providers using `DataGenerator#addProvider`.

Client Assets

- `net.minecraftforge.common.data.LanguageProvider` - for language strings; implement `#addTranslations`
- `net.minecraftforge.common.data.SoundDefinitionsProvider` - for `sounds.json`; implement `#registerSounds`
- `net.minecraftforge.client.model.generators.ModelProvider<?>` - for models; implement `#registerModels`
 - `ItemModelProvider` - for item models
 - `BlockModelProvider` - for block models
- `net.minecraftforge.client.model.generators.BlockStateProvider` - for blockstate JSONs and their block and item models; implement `#registerStatesAndModels`

Server Data

These classes are under the `net.minecraftforge.common.data` package:

- `GlobalLootModifierProvider` - for global loot modifiers; implement `#start`
- `DatapackBuiltinEntriesProvider` for datapack registry objects; pass in `RegistrySetBuilder` to the constructor

These classes are under the `net.minecraftforge.data` package:

- `loot.LootTableProvider` - for loot tables; pass in `LootTableProvider$SubProviderEntrys` to the constructor
- `recipes.RecipeProvider` - for recipes and their unlocking advancements; implement `#buildRecipes`
- `tags.TagsProvider` - for tags; implement `#addTags`
- `advancements.AdvancementProvider` - for advancements; pass in `AdvancementSubProviders` to the constructor

Model Generation

Models can be generated for models or block states by default. Each provides a method of generating the necessary JSONs (`ModelBuilder#toJson` for models and `IGeneratedBlockState#toJson` for block states). After implementation, the associated providers must be added to the `DataGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    DataGenerator gen = event.getGenerator();
    ExistingFileHelper efh = event.getExistingFileHelper();

    gen.addProvider(
        // Tell generator to run only when client assets are generating
        event.includeClient(),
        output -> new MyItemModelProvider(output, MOD_ID, efh)
    );
    gen.addProvider(
        event.includeClient(),
        output -> new MyBlockStateProvider(output, MOD_ID, efh)
    );
}
```

Model Files

A `ModelFile` acts as the base for all models referenced or generated by a provider. Each model file stores the location relative to the `models` subdirectory and can assert whether the file exists.

Existing Model Files

`ExistingModelFile` is a subclass of `ModelFile` which checks via `ExistingFileHelper#exists` whether the model already exists within the `models` subdirectory. All non-generated models are usually referenced through `ExistingModelFiles`.

Unchecked Model Files

`UncheckedModelFile` is a subclass of `ModelFile` which assumes the specified model exists in some location.

!!! note There should be no cases where an `UncheckedModelFile` is used to reference a model. If there is, then the associated resources are not properly being tracked by `ExistingFileHelper`.

Model Builders

A `ModelBuilder` represents a to-be-generated `ModelFile`. It contains all the data about a model: its parent, faces, textures, transformations, lighting, and loader.

!!! tip While a complex model can be generated, it is recommended that those models be constructed using a modeling software beforehand. Then, the data provider can generate the children models with specific textures applied through the defined references in the parent complex model.

The parent (via `ModelBuilder#parent`) of the builder can be any `ModelFile`: generated or existing. Generated files are added to `ModelProviders` as soon as the builder is created. The builder itself can be passed in as a parent, or the `ResourceLocation` can supplied alternatively.

!!! warning If the parent is not generated before the child model when passing in a `ResourceLocation`, then an exception will be thrown.

Each element (via `ModelBuilder#element`) within a model is defined as cube using two three-dimensional points (`ElementBuilder#from` and `#to` respectively) where each axis is limited to the values `[-16,32]` (between -16 and 32 inclusive). Each face (`ElementBuilder#face`) of the cube can specify when the face is culled (`FaceBuilder#cullface`), tint index (`FaceBuilder#tintindex`), texture reference from the `textures` keys (`FaceBuilder#texture`), UV coordinate on the texture (`FaceBuilder#uvs`), and rotation in 90 degree intervals (`FaceBuilder#rotation`).

!!! note It recommended for block models which have elements that exceed a bound of `[0,16]` on any axis to separate into multiple blocks, such as for a multiblock structure, to avoid lighting and culling issues.

Each cube can additionally be rotated (`ElementBuilder#rotation`) around a specified point (`RotationBuilder#origin`) for a given axis (`RotationBuilder#axis`) in 22.5 degree intervals (`RotationBuilder#angle`). The cube can scale all faces in relation to the entire model as well (`RotationBuilder#rescale`). The cube can also determine whether its shadows should be rendered (`ElementBuilder#shade`).

Each model defines a list of texture keys (`ModelBuilder#texture`) which points to either a location or a reference. Each key can then be referenced in any element by prefixing using a `#` (a texture key of `example` can be referenced in an element using `#example`). A location specifies where a texture is in `assets/<namespace>/textures/<path>.png`. A reference is used by any models parenting the current model as keys to define textures for later.

The model can additionally be transformed (`ModelBuilder#transforms`) for any defined perspective (in the left hand in first person, in the gui, on the ground, etc.). For any perspective (`TransformsBuilder#transform`), the rotation (`TransformVecBuilder#rotation`), translation (`TransformVecBuilder#translation`), and scale (`TransformVecBuilder#scale`) can be set.

Finally, the model can set whether to use ambient occlusion in a level (`ModelBuilder#ao`) and from what location to light and shade the model from `ModelBuilder#guiLight`.

BlockModelBuilder

A `BlockModelBuilder` represents a block model to-be-generated. In addition to the `ModelBuilder`, a transform to the entire model (`BlockModelBuilder#rootTransform`) can be generated. The root can be translated (`RootTransformBuilder#transform`), rotated (`RootTransformBuilder#rotation`, `RootTransformBuilder#postRotation`), and scaled (`RootTransformBuilder#scale`) either individually or all in one transformation (`RootTransformBuilder#transform`) around some origin (`RootTransformBuilder#origin`).

ItemModelBuilder

An `ItemModelBuilder` represents an item model to-be-generated. In addition to the `ModelBuilder`, overrides (`OverrideBuilder#override`) can be generated. Each override applied to a model can apply conditions which represent for a given property that must be above the specified value (`OverrideBuilder#predicate`). If the conditions are met, then the specified model (`OverrideBuilder#model`) will be rendered instead of this model.

Model Providers

The `ModelProvider` subclasses are responsible for generating the constructed `ModelBuilders`. The provider takes in the generator, mod id, subdirectory in the `models` folder to generate within, a `ModelBuilder` factory, and the existing file helper. Each provider subclass must implement `#registerModels`.

The provider contains basic methods which either create the **ModelBuilder** or provides convenience for getting texture or model references:

| Method | Description |
|---------------------------|---|
| getBuilder | Creates a new ModelBuilder within the provider's subdirectory for the given mod id. |
| withExistingParent | Creates a new ModelBuilder for the given parent. Should be used when the parent is not generated by the builder. |
| mcLoc | Creates a ResourceLocation for the path in the minecraft namespace. |
| modLoc | Creates a ResourceLocation for the path in the given mod id's namespace. |

Additionally, there are several helpers for easily generating common models using vanilla templates. Most are for block models with only a few being universal.

!!! note Although the models are within a specific subdirectory, that does **not** mean that the model cannot be referenced by a model in another subdirectory. Usually, it is indicative of that model being used for that type of object.

BlockModelProvider

The **BlockModelProvider** is used for generating block models via **BlockModelBuilder** in the **block** folder. Block models should typically parent **minecraft:block/block** or one of its children models for use with item models.

!!! note Block models and its item model counterpart are typically not generated through a direct subclass of **BlockModelProvider** and **ItemModelProvider** but through **BlockstateProvider**.

ItemModelProvider

The **ItemModelProvider** is used for generating block models via **ItemModelBuilder** in the **item** folder. Most item models parent **item/generated** and use **layer0** to specify their texture, which can be done using **#singleTexture**.

!!! note **item/generated** can support five texture layers stacked on top of each other: **layer0**, **layer1**, **layer2**, **layer3**, and **layer4**.

```
// In some ItemModelProvider#registerModels
// Will generate 'assets/<modid>/models/item/example_item.json'
// Parent will be 'minecraft:item/generated'
// For the texture key 'layer0'
// It will be at 'assets/<modid>/textures/item/example_item.png'
this.basicItem(EXAMPLE_ITEM.get());
```

!!! note Item models for blocks should typically parent an existing block model instead of generating a separate model for an item.

Block State Provider

A **BlockstateProvider** is responsible for generating block state JSONs in **blockstates**, block models in **models/block**, and item models in **models/item** for said blocks. The provider takes in the data generator, mod id, and existing file helper. Each **BlockstateProvider** subclass must implement **#registerStatesAndModels**.

The provider contains basic methods for generating block state JSONs and block models. Item models must be generated separately as a block state JSON may define multiple models to use in different contexts. There are a number of common methods, however, that that the modder should be aware of when dealing with more complex tasks:

| Method | Description |
|---------------|--|
| models | Gets the BlockModelProvider used to generate the item block models. |

| Method | Description |
|----------------------------------|---|
| <code>itemModels</code> | Gets the <code>ItemModelProvider</code> used to generate the item block models. |
| <code>modLoc</code> | Creates a <code>ResourceLocation</code> for the path in the given mod id's namespace. |
| <code>mcLoc</code> | Creates a <code>ResourceLocation</code> for the path in the <code>minecraft</code> namespace. |
| <code>blockTexture</code> | References a texture within <code>textures/block</code> which has the same name as the block. |
| <code>simpleBlockItem</code> | Creates an item model for a block given the associated model file. |
| <code>simpleBlockWithItem</code> | Creates a single block state for a block model and an item model using the block model as its parent. |

A block state JSON is made up of variants or conditions. Each variant or condition references a `ConfiguredModelList`: a list of `ConfiguredModels`. Each configured model contains the model file (via `ConfiguredModel$Builder#modelFile`), the X and Y rotation in 90 degree intervals (via `#rotationX` and `rotationY` respectively), whether the texture can rotate when the model is rotated by the block state JSON (via `#uvLock`), and the weight of the model appearing compared to other models in the list (via `#weight`).

The builder (`ConfiguredModel#builder`) can also create an array of `ConfiguredModels` by creating the next model using `#nextModel` and repeating the settings until `#build` is called.

VariantBlockStateBuilder

Variants can be generated using `BlockStateProvider#getVariantBuilder`. Each variant specifies a list of properties (`PartialBlockstate`) which when matches a `BlockState` in a level, will display a model chosen from the corresponding model list. An exception is thrown if there is a `BlockState` which is not covered by any variant defined. Only one variant can be true for any `BlockState`.

A `PartialBlockstate` is typically defined using one of three methods:

| Method | Description |
|--|--|
| <code>partialState</code>
<code>forAllStates</code> | Creates a <code>PartialBlockstate</code> to be defined. Defines a function where a given <code>BlockState</code> can be represented by an array of <code>ConfiguredModels</code> . |
| <code>forAllStatesExcept</code> | Defines a function similar to <code>#forAllStates</code> ; however, it also specifies which properties do not affect the models rendered. |

For a `PartialBlockstate`, the properties defined can be specified (`#with`). The configured models can be set (`#setModels`), appended to the existing models (`#addModels`), or built (`#modelForState` and then `ConfiguredModel$Builder#addModel` once finished instead of `#ConfiguredModel$Builder#build`).

```
// In some BlockStateProvider#registerStatesAndModels

// EXAMPLE_BLOCK_1: Has Property BlockStateProperties#AXIS
this.getVariantBuilder(EXAMPLE_BLOCK_1) // Get variant builder
    .partialState() // Construct partial state
    .with(AXIS, Axis.Y) // When BlockState AXIS = Y
        .modelForState() // Set models when AXIS = Y
        .modelFile(yModelFile1) // Can show 'yModelFile1'
        .nextModel() // Adds another model when AXIS = Y
        .modelFile(yModelFile2) // Can show 'yModelFile2'
        .weight(2) // Will show 'yModelFile2' 2/3 of the time
        .addModel() // Finalizes models when AXIS = Y
    .with(AXIS, Axis.Z) // When BlockState AXIS = Z
        .modelForState() // Set models when AXIS = Z
        .modelFile(hModelFile) // Can show 'hModelFile'
        .addModel() // Finalizes models when AXIS = Z
    .with(AXIS, Axis.X) // When BlockState AXIS = X
        .modelForState() // Set models when AXIS = X
        .modelFile(hModelFile) // Can show 'hModelFile'
        .rotationY(90) // Rotates 'hModelFile' 90 degrees on the Y axis
```

```

.addModel(); // Finalizes models when AXIS = X

// EXAMPLE_BLOCK_2: Has Property BlockStateProperties#HORIZONTAL_FACING
this.getVariantBuilder(EXAMPLE_BLOCK_2) // Get variant builder
.forAllStates(state -> // For all possible states
    ConfiguredModel.builder() // Creates configured model builder
        .modelFile(modelFile) // Can show 'modelFile'
        .rotationY((int) state.getValue(HORIZONTAL_FACING).toYRot()) // Rotates 'modelFile' on the Y axis depending on
        the property
        .build() // Creates the array of configured models
);

// EXAMPLE_BLOCK_3: Has Properties BlockStateProperties#HORIZONTAL_FACING, BlockStateProperties#WATERLOGGED
this.getVariantBuilder(EXAMPLE_BLOCK_3) // Get variant builder
.forAllStatesExcept(state -> // For all HORIZONTAL_FACING states
    ConfiguredModel.builder() // Creates configured model builder
        .modelFile(modelFile) // Can show 'modelFile'
        .rotationY((int) state.getValue(HORIZONTAL_FACING).toYRot()) // Rotates 'modelFile' on the Y axis depending on
        the property
        .build(), // Creates the array of configured models
    WATERLOGGED); // Ignores WATERLOGGED property

```

MultiPartBlockStateBuilder

Multiparts can be generated using `BlockstateProvider#getMultipartBuilder`. Each part (`MultiPartBlockStateBuilder#part`) specifies a group of conditions of properties which when matches a `BlockState` in a level, will display a model from the model list. All condition groups that match the `BlockState` will display their chosen model overlaid on each other.

For any part (obtained via `ConfiguredModel$Builder#addModel`), a condition can be added (via `#condition`) when a property is one of the specified values. Conditions must all succeed or, when `#useOr` is set, at least one must succeed. Conditions can be grouped (via `#nestedGroup`) as long as the current grouping only contains other groups and no single conditions. Groups of conditions can be left using `#endNestedGroup` and a given part can be finished via `#end`.

```

// In some BlockstateProvider#registerStatesAndModels

// Redstone Wire
this.getMultipartBuilder(REDSTONE) // Get multipart builder
.part() // Create part
    .modelFile(redstoneDot) // Can show 'redstoneDot'
    .addModel() // 'redstoneDot' is displayed when...
    .useOr() // At least one of these conditions are true
    .nestedGroup() // true when all grouped conditions are true
        .condition(WEST_REDSTONE, NONE) // true when WEST_REDSTONE is NONE
        .condition(EAST_REDSTONE, NONE) // true when EAST_REDSTONE is NONE
        .condition(SOUTH_REDSTONE, NONE) // true when SOUTH_REDSTONE is NONE
        .condition(NORTH_REDSTONE, NONE) // true when NORTH_REDSTONE is NONE
    .endNestedGroup() // End group
    .nestedGroup() // true when all grouped conditions are true
        .condition(EAST_REDSTONE, SIDE, UP) // true when EAST_REDSTONE is SIDE or UP
        .condition(NORTH_REDSTONE, SIDE, UP) // true when NORTH_REDSTONE is SIDE or UP
    .endNestedGroup() // End group
    .nestedGroup() // true when all grouped conditions are true
        .condition(EAST_REDSTONE, SIDE, UP) // true when EAST_REDSTONE is SIDE or UP
        .condition(SOUTH_REDSTONE, SIDE, UP) // true when SOUTH_REDSTONE is SIDE or UP
    .endNestedGroup() // End group
    .nestedGroup() // true when all grouped conditions are true
        .condition(WEST_REDSTONE, SIDE, UP) // true when WEST_REDSTONE is SIDE or UP
        .condition(SOUTH_REDSTONE, SIDE, UP) // true when SOUTH_REDSTONE is SIDE or UP
    .endNestedGroup() // End group
    .nestedGroup() // true when all grouped conditions are true
        .condition(WEST_REDSTONE, SIDE, UP) // true when WEST_REDSTONE is SIDE or UP
        .condition(NORTH_REDSTONE, SIDE, UP) // true when NORTH_REDSTONE is SIDE or UP
    .endNestedGroup() // End group
    .end() // Finish part
.part() // Create part
    .modelFile(redstoneSide0) // Can show 'redstoneSide0'
    .addModel() // 'redstoneSide0' is displayed when...

```

```

.condition(NORTH_REDSTONE, SIDE, UP) // NORTH_REDSTONE is SIDE or UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneSideAlt0) // Can show 'redstoneSideAlt0'
.addModel() // 'redstoneSideAlt0' is displayed when...
.condition(SOUTH_REDSTONE, SIDE, UP) // SOUTH_REDSTONE is SIDE or UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneSideAlt1) // Can show 'redstoneSideAlt1'
.rotationY(270) // Rotates 'redstoneSideAlt1' 270 degrees on the Y axis
.addModel() // 'redstoneSideAlt1' is displayed when...
.condition(EAST_REDSTONE, SIDE, UP) // EAST_REDSTONE is SIDE or UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneSide1) // Can show 'redstoneSide1'
.rotationY(270) // Rotates 'redstoneSide1' 270 degrees on the Y axis
.addModel() // 'redstoneSide1' is displayed when...
.condition(WEST_REDSTONE, SIDE, UP) // WEST_REDSTONE is SIDE or UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneUp) // Can show 'redstoneUp'
.addModel() // 'redstoneUp' is displayed when...
.condition(NORTH_REDSTONE, UP) // NORTH_REDSTONE is UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneUp) // Can show 'redstoneUp'
.rotationY(90) // Rotates 'redstoneUp' 90 degrees on the Y axis
.addModel() // 'redstoneUp' is displayed when...
.condition(EAST_REDSTONE, UP) // EAST_REDSTONE is UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneUp) // Can show 'redstoneUp'
.rotationY(180) // Rotates 'redstoneUp' 180 degrees on the Y axis
.addModel() // 'redstoneUp' is displayed when...
.condition(SOUTH_REDSTONE, UP) // SOUTH_REDSTONE is UP
.end() // Finish part
.part() // Create part
.modelFile(redstoneUp) // Can show 'redstoneUp'
.rotationY(270) // Rotates 'redstoneUp' 270 degrees on the Y axis
.addModel() // 'redstoneUp' is displayed when...
.condition(WEST_REDSTONE, UP) // WEST_REDSTONE is UP
.end(); // Finish part

```

Model Loader Builders

Custom model loaders can also be generated for a given `ModelBuilder`. Custom model loaders subclass `CustomLoaderBuilder` and can be applied to a `ModelBuilder` via `#customLoader`. The factory method passed in creates a new loader builder to which configurations can be made. After all the changes have been finished, the custom loader can return back to the `ModelBuilder` via `CustomLoaderBuilder#end`.

| Model Builder | Factory Method | Description |
|--|---------------------|---|
| <code>DynamicFluidContainerModelBuilder</code> | <code>#begin</code> | Generates a bucket model for the specified fluid. |
| <code>CompositeModelBuilder</code> | <code>#begin</code> | Generates a model composed of models. |
| <code>ItemLayersModelBuilder</code> | <code>#begin</code> | Generates a Forge implementation of an <code>item/generated</code> model. |
| <code>SeparateTransformsModelBuilder</code> | <code>#begin</code> | Generates a model which changes based on the specified transform. |
| <code>ObjModelBuilder</code> | <code>#begin</code> | Generates an OBJ model. |

```

// For some BlockModelBuilder builder
builder.customLoader(ObjModelBuilder::begin) // Custom loader 'forge:obj'
.modelLocation(modLoc("models/block/model.obj")) // Set the OBJ model location
.flipV(true) // Flips the V coordinate in the supplied .mtl texture
.end() // Finish custom loader configuration
.texture("particle", mcLoc("block/dirt")) // Set particle texture to dirt
.texture("texture0", mcLoc("block/dirt")); // Set 'texture0' texture to dirt

```


Custom Model Loader Builders

Custom loader builders can be created by extending `CustomLoaderBuilder`. The constructor can still have a `protected` visibility with the `ResourceLocation` hardcoded to the loader id registered via `ModelEvent$RegisterGeometryLoaders#register`. The builder can then be initialized via a static factory method or the constructor if made public.

```
public class ExampleLoaderBuilder<T extends ModelBuilder<T>> extends CustomLoaderBuilder<T> {
    public static <T extends ModelBuilder<T>> ExampleLoaderBuilder<T> begin(T parent, ExistingFileHelper
existingFileHelper) {
        return new ExampleLoaderBuilder<>(parent, existingFileHelper);
    }

    protected ExampleLoaderBuilder(T parent, ExistingFileHelper existingFileHelper) {
        super(new ResourceLocation(MOD_ID, "example_loader"), parent, existingFileHelper);
    }
}
```

Afterwards, any configurations specified by the loader should be added as chainable methods.

```
// In ExampleLoaderBuilder
public ExampleLoaderBuilder<T> exampleInt(int example) {
    // Set int
    return this;
}

public ExampleLoaderBuilder<T> exampleString(String example) {
    // Set string
    return this;
}
```

If any additional configuration is specified, `#toJson` should be overridden to write the additional properties.

```
// In ExampleLoaderBuilder
@Override
public JsonObject toJson(JsonObject json) {
    json = super.toJson(json); // Handle base loader properties
    // Encode custom loader properties
    return json;
}
```

Custom Model Providers

Custom model providers require a `ModelBuilder` subclass, which defines the base of the model to generate, and a `ModelProvider` subclass, which generates the models.

The `ModelBuilder` subclass contains any special properties to which can be applied specifically to those types of models (item models can have overrides). If any additional properties are added, `#toJson` needs to be overridden to write the additional information.

```
public class ExampleModelBuilder extends ModelBuilder<ExampleModelBuilder> {
    // ...
}
```

The `ModelProvider` subclass requires no special logic. The constructor should hardcode the subdirectory within the `models` folder and the `ModelBuilder` to represent the to-be-generated models.

```
public class ExampleModelProvider extends ModelProvider<ExampleModelBuilder> {

    public ExampleModelProvider(PackOutput output, String modid, ExistingFileHelper existingFileHelper) {
        // Models will be generated to 'assets/<modid>/models/example' if no 'modid' is specified in '#getBuilder'
        super(output, modid, "example", ExampleModelBuilder::new, existingFileHelper);
    }
}
```

Custom Model Consumers

Custom model consumers like `BlockstateProvider` can be created by manually generating the models themselves. The `ModelProvider` subclass used to generate the models should be specified and made available.

```
public class ExampleModelConsumerProvider implements IDataProvider {  
  
    public ExampleModelConsumerProvider(PackOutput output, String modid, ExistingFileHelper existingFileHelper) {  
        this.example = new ExampleModelProvider(output, modid, existingFileHelper);  
    }  
}
```

Once the data provider is running, the models within the `ModelProvider` subclass can be generated using `ModelProvider#generateAll`.

```
// In ExampleModelConsumerProvider  
@Override  
public CompletableFuture<?> run(CachedOutput cache) {  
    // Populate the model provider  
    CompletableFuture<?> exampleFutures = this.example.generateAll(cache); // Generate the models  
  
    // Run logic and create CompletableFuture(s) for writing files  
    // ...  
  
    // Assume we have a new CompletableFuture providerFuture  
    return CompletableFuture.allOf(exampleFutures, providerFuture);  
}
```

Language Generation

Language files can be generated for a mod by subclassing `LanguageProvider` and implementing `#addTranslations`. Each `LanguageProvider` subclass created represents a separate locale (`en_us` represents American English, `es_es` represents Spanish, etc.). After implementation, the provider must be added to the `DataGenerator`.

```
// On the MOD event bus  
@SubscribeEvent  
public void gatherData(GatherDataEvent event) {  
    event.getGenerator().addProvider(  
        // Tell generator to run only when client assets are generating  
        event.includeClient(),  
        // Localizations for American English  
        output -> new MyLanguageProvider(output, MOD_ID, "en_us")  
    );  
}
```

LanguageProvider

Each language provider is simple a map of strings where each translation key is mapped to a localized name. A translation key mapping can be added using `#add`. Additionally, there are methods which use the translation key of a `Block`, `Item`, `ItemStack`, `Enchantment`, `MobEffect`, and `EntityType`.

```
// In LanguageProvider#addTranslations  
this.addBlock(EXAMPLE_BLOCK, "Example Block");  
this.add("object.examplemod.example_object", "Example Object");
```

!!! tip Localized names which contain alphanumeric values not in American English can be supplied as is. The provider automatically translates the characters into their unicode equivalents to be read by the game.

```
```java  
// Encoded as 'Example with a d\u00E9acritic'
this.addItem("example.diacritic", "Example with a diácrítico");
```
```

Sound Definition Generation

The `sounds.json` file can be generated for a mod by subclassing `SoundDefinitionsProvider` and implementing `#registerSounds`. After implementation, the provider must be added to the `DataGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when client assets are generating
        event.includeClient(),
        output -> new MySoundDefinitionsProvider(output, MOD_ID, event.getExistingFileHelper())
    );
}
```

Adding a Sound

A sound definition can be generated by specifying the sound name and definition via `#add`. The sound name can either be provided from a `SoundEvent`, `ResourceLocation`, or string.

!!! warning The sound name supplied will always assume the namespace is the mod id supplied to the constructor of the provider. There is no validation performed on the namespace of the sound name!

SoundDefinition

The `SoundDefinition` can be created using `#definition`. The definition contains the data to define a sound instance.

A definition specifies a few methods:

| Method | Description |
|-----------------------|---|
| <code>with</code> | Adds a sound(s) which may be played when the definition is selected. |
| <code>subtitle</code> | Sets the translation key of the definition. |
| <code>replace</code> | When <code>true</code> , removes the sounds already defined by other <code>sounds.json</code> for this definition instead of appending to it. |

SoundDefinition\$Sound

A sound supplied to the `SoundDefinition` can be specified using `SoundDefinitionsProvider#sound`. These methods take in the reference of the sound and a `SoundType` if specified.

The `SoundType` can be one of two values:

| Sound Type | Definition |
|--------------------|---|
| <code>SOUND</code> | Specifies a reference to the sound located at <code>assets/<namespace>/sounds/<path>.ogg</code> . |
| <code>EVENT</code> | Specifies a reference to the name of another sound defined by the <code>sounds.json</code> . |

Each `Sound` created from `SoundDefinitionsProvider#sound` can specify additional configurations on how to load and play the sound provided:

| Method | Description |
|---------------------|---|
| <code>volume</code> | Sets the volume scale of the sound, must be greater than 0. |
| <code>pitch</code> | Sets the pitch scale of the sound, must be greater than 0. |
| <code>weight</code> | Sets the likelihood of the sound getting played when the sound is selected. |
| <code>stream</code> | When <code>true</code> , reads the sound from file instead of loading the sound into memory. Recommended for long sounds: background music, music discs, etc. |

| Method | Description |
|----------------------------------|--|
| <code>attenuationDistance</code> | Sets the number of blocks the sound can be heard from. |
| <code>preload</code> | When <code>true</code> , immediately loads the sound into memory as soon as the resource pack is loaded. |

```
// In some SoundDefinitionsProvider#registerSounds
this.add(EXAMPLE_SOUND_EVENT, definition()
    .subtitle("sound.examplemod.example_sound") // Set translation key
    .with(
        sound(new ResourceLocation(MODID, "example_sound_1")) // Set first sound
        .weight(4) // Has a 4 / 5 = 80% chance of playing
        .volume(0.5), // Scales all volumes called on this sound by half
        sound(new ResourceLocation(MODID, "example_sound_2")) // Set second sound
        .stream() // Streams the sound
    )
);

this.add(EXAMPLE_SOUND_EVENT_2, definition()
    .subtitle("sound.examplemod.example_sound") // Set translation key
    .with(
        sound(EXAMPLE_SOUND_EVENT.getLocation(), SoundType.EVENT) // Adds sounds from 'EXAMPLE_SOUND_EVENT'
        .pitch(0.5) // Scales all pitches called on this sound by half
    )
);
```

Recipe Generation

Recipes can be generated for a mod by subclassing `RecipeProvider` and implementing `#buildRecipes`. A recipe is supplied for data generation once a `FinishedRecipe` view is accepted by the consumer. `FinishedRecipes` can either be created and supplied manually or, for convenience, created using a `RecipeBuilder`.

After implementation, the provider must be added to the `DataGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        MyRecipeProvider::new
    );
}
```

RecipeBuilder

`RecipeBuilder` is a convenience implementation for creating `FinishedRecipes` to generate. It provides basic definitions for unlocking, grouping, saving, and getting the result of a recipe. This is done through `#unlockedBy`, `#group`, `#save`, and `#getResult` respectively.

!!! important `ItemStack` outputs in recipes are not supported within vanilla recipe builders. A `FinishedRecipe` must be built in a different manner for existing vanilla recipe serializers to generate this data.

!!! warning The item results being generated must have a valid `RecipeCategory` specified; otherwise, a `NullPointerException` will be thrown.

All recipe builders except for `[SpecialRecipeBuilder]` require an advancement criteria to be specified. All recipes generate a criteria unlocking the recipe if the player has used the recipe previously. However, an additional criteria must be specified that allows the player to obtain the recipe without any prior knowledge. If any of the criteria specified is true, then the player will obtain the recipe for the recipe book.

!!! tip Recipe criteria commonly use `InventoryChangeTrigger` to unlock their recipe when certain items are present in the user's inventory.

ShapedRecipeBuilder

ShapedRecipeBuilder is used to generate shaped recipes. The builder can be initialized via `#shaped`. The recipe group, input symbol pattern, symbol definition of ingredients, and the recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
ShapedRecipeBuilder builder = ShapedRecipeBuilder.shaped(RecipeCategory.MISC, result)
    .pattern("a a") // Create recipe pattern
    .define('a', item) // Define what the symbol represents
    .unlockedBy("criteria", criteria) // How the recipe is unlocked
    .save(writer); // Add data to builder
```

Additional Validation Checks Shaped recipes have some additional validation checks performed before building:

- A pattern must be defined and take in more than one item.
- All pattern rows must be the same width.
- A symbol cannot be defined more than once.
- The space character (' ') is reserved for representing no item in a slot and, as such, cannot be defined.
- A pattern must use all symbols defined by the user.

ShapelessRecipeBuilder

ShapelessRecipeBuilder is used to generate shapeless recipes. The builder can be initialized via `#shapeless`. The recipe group, input ingredients, and the recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
ShapelessRecipeBuilder builder = ShapelessRecipeBuilder.shapeless(RecipeCategory.MISC, result)
    .requires(item) // Add item to the recipe
    .unlockedBy("criteria", criteria) // How the recipe is unlocked
    .save(writer); // Add data to builder
```

SimpleCookingRecipeBuilder

SimpleCookingRecipeBuilder is used to generate smelting, blasting, smoking, and campfire cooking recipes. Additionally, custom cooking recipes using the **SimpleCookingSerializer** can also be data generated using this builder. The builder can be initialized via `#smelting`, `#blasting`, `#smoking`, `#campfireCooking`, or `#cooking` respectively. The recipe group and the recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
SimpleCookingRecipeBuilder builder = SimpleCookingRecipeBuilder.smelting(input, RecipeCategory.MISC, result,
experience, cookingTime)
    .unlockedBy("criteria", criteria) // How the recipe is unlocked
    .save(writer); // Add data to builder
```

SingleItemRecipeBuilder

SingleItemRecipeBuilder is used to generate stonecutting recipes. Additionally, custom single item recipes using a serializer like **SingleItemRecipe\$Serializer** can also be data generated using this builder. The builder can be initialized via `#stonecutting` or through the constructor respectively. The recipe group and the recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
SingleItemRecipeBuilder builder = SingleItemRecipeBuilder.stonecutting(input, RecipeCategory.MISC, result)
    .unlockedBy("criteria", criteria) // How the recipe is unlocked
    .save(writer); // Add data to builder
```

Non-RecipeBuilder Builders

Some recipe builders do not implement **RecipeBuilder** due to lacking features used by all previously mentioned recipes.

SmithingTransformRecipeBuilder

SmithingTransformRecipeBuilder is used to generate smithing recipes which transform an item. Additionally, custom recipes using a serializer like SmithingTransformRecipe\$Serializer can also be data generated using this builder. The builder can be initialized via #smithing or through the constructor respectively. The recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
SmithingTransformRecipeBuilder builder = SmithingTransformRecipeBuilder.smithing(template, base, addition,
RecipeCategory.MISC, result)
    .unlocks("criteria", criteria) // How the recipe is unlocked
    .save(writer, name); // Add data to builder
```

SmithingTrimRecipeBuilder

SmithingTrimRecipeBuilder is used to generate smithing recipes for armor trims. Additionally, custom upgrade recipes using a serializer like SmithingTrimRecipe\$Serializer can also be data generated using this builder. The builder can be initialized via #smithingTrim or through the constructor respectively. The recipe unlock criteria can be specified before saving.

```
// In RecipeProvider#buildRecipes(writer)
SmithingTrimRecipe builder = SmithingTrimRecipe.smithingTrim(template, base, addition, RecipeCategory.MISC)
    .unlocks("criteria", criteria) // How the recipe is unlocked
    .save(writer, name); // Add data to builder
```

SpecialRecipeBuilder

SpecialRecipeBuilder is used to generate empty JSONs for dynamic recipes that cannot easily be constrained to the recipe JSON format (dyeing armor, firework, etc.). The builder can be initialized via #special.

```
// In RecipeProvider#buildRecipes(writer)
SpecialRecipeBuilder.special(dynamicRecipeSerializer)
    .save(writer, name); // Add data to builder
```

Conditional Recipes

Conditional recipes can also be data generated via ConditionalRecipe\$Builder. The builder can be obtained using #builder.

Conditions for each recipe can be specified by first calling #addCondition and then calling #addRecipe after all conditions have been specified. This process can be repeated as many times as the programmer would like.

After all recipes have been specified, advancements can be added for each recipe at the end using #generateAdvancement. Alternatively, the conditional advancement can be set using #setAdvancement.

```
// In RecipeProvider#buildRecipes(writer)
ConditionalRecipe.builder()
    // Add the conditions for the recipe
    .addCondition(...)
    // Add recipe to return when conditions are true
    .addRecipe(...)

    // Add the next conditions for the next recipe
    .addCondition(...)
    // Add next recipe to return when the next conditions are true
    .addRecipe(...)

    // Create conditional advancement which uses the conditions
    // and unlocking advancement in the recipes above
    .generateAdvancement()
    .build(writer, name);
```

IConditionBuilder

To simplify adding conditions to conditional recipes without having to construct the instances of each condition instance manually, the extended RecipeProvider can implement IConditionBuilder. The interface adds methods

to easily construct condition instances.

```
// In ConditionalRecipe$Builder#addCondition
(
    // If either 'examplemod:example_item'
    // OR 'examplemod:example_item2' exists
    // AND
    // NOT FALSE

    // Methods are defined by IConditionBuilder
    and(
        or(
            itemExists("examplemod", "example_item"),
            itemExists("examplemod", "example_item2")
        ),
        not(
            FALSE()
        )
    )
)
```

Custom Recipe Serializers

Custom recipe serializers can be data generated by creating a builder that can construct a **FinishedRecipe**. The finished recipe encodes the recipe data and its unlocking advancement, when present, to JSON. Additionally, the name and serializer of the recipe is also specified to know where to write to and what can decode the object when loading. Once a **FinishedRecipe** is constructed, it simply needs to be passed to the **Consumer** supplied by **RecipeProvider#buildRecipes**.

!!! tip **FinishedRecipes** are flexible enough that any object transformation can be data generated, not just items.

Loot Table Generation

Loot tables can be generated for a mod by constructing a new **LootTableProvider** and providing **LootTableProvider\$SubProviderEntrys**. The provider must be added to the **DataGenerator**.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        output -> new MyLootTableProvider(
            output,
            // Specify registry names of tables that are required to generate, or can leave empty
            Collections.emptySet(),
            // Sub providers which generate the loot
            List.of(subProvider1, subProvider2, /*...*/)
        )
    );
}
```

LootTableSubProvider

Each **LootTableProvider\$SubProviderEntry** takes in a supplied **LootTableSubProvider**, which generates the loot table, for a given **LootContextParamSet**. The **LootTableSubProvider** contains a method which takes in the writer (**BiConsumer<ResourceLocation, LootTable.Builder>**) to generate a table.

```
public class ExampleSubProvider implements LootTableSubProvider {

    // Used to create a factory method for the wrapping Supplier
    public ExampleSubProvider() {}

    // The method used to generate the loot tables
    @Override
    public void generate(BiConsumer<ResourceLocation, LootTable.Builder> writer) {
        // Generate loot tables here by calling writer#accept
    }
}
```

```
}
}
```

The table can then be added to `LootTableProvider#getTables` for any available `LootContextParamSet`:

```
// In the list passed into the LootTableProvider constructor
new LootTableProvider.SubProviderEntry(
    ExampleSubProvider::new,
    // Loot table generator for the 'empty' param set
    LootContextParamSets.EMPTY
)
```

BlockLootSubProvider and EntityLootSubProvider Subclasses

For `LootContextParamSets#BLOCK` and `#ENTITY`, there are special types (`BlockLootSubProvider` and `EntityLootSubProvider` respectively) which provide additional helper methods for creating and validating that there are loot tables.

The `BlockLootSubProvider`'s constructor takes in a list of items, which are explosion resistant to determine whether the loot table can be generated if a block is exploded, and a `FeatureFlagSet`, which determines whether the block is enabled so that a loot table is generated for it.

```
// In some BlockLootSubProvider subclass
public MyBlockLootSubProvider() {
    super(Collections.emptySet(), FeatureFlags.REGISTRY.allFlags());
}
```

The `EntityLootSubProvider`'s constructor takes in a `FeatureFlagSet`, which determines whether the entity type is enabled so that a loot table is generated for it.

```
// In some EntityLootSubProvider subclass
public MyEntityLootSubProvider() {
    super(FeatureFlags.REGISTRY.allFlags());
}
```

To use them, all registered objects must be supplied to either `BlockLootSubProvider#getKnownBlocks` and `EntityLootSubProvider#getKnownEntityTypes` respectively. These methods are to make sure all objects within the iterable has a loot table.

!!! tip If `DeferredRegister` is being used to register a mod's objects, then the `#getKnown*` methods can be supplied the entries via `DeferredRegister#getEntries`:

```
```java
// In some BlockLootSubProvider subclass for some DeferredRegister BLOCK_REGISTRAR
@Override
protected Iterable<Block> getKnownBlocks() {
 return BLOCK_REGISTRAR.getEntries() // Get all registered entries
 .stream() // Stream the wrapped objects
 .flatMap(RegistryObject::stream) // Get the object if available
 ::iterator; // Create the iterable
}
```
```

The loot tables themselves can be added by implementing the `#generate` method.

```
// In some BlockLootSubProvider subclass
@Override
public void generate() {
    // Add loot tables here
}
```


Loot Table Builders

To generate loot tables, they are accepted by the `LootTableSubProvider` as a `LootTable$Builder`. Afterwards, the specified `LootContextParamSet` is set in the `LootTableProvider$SubProviderEntry` and then built via `#build`. Before being built, the builder can specify entries, conditions, and modifiers which affect how the loot table functions.

!!! note The functionality of loot tables is so expansive that it will not be covered by this documentation in its entirety. Instead, a brief description of each component will be mentioned. The specific subtypes of each component can be found using an IDE. Their implementations will be left as an exercise to the reader.

LootTable

Loot tables are the base object and can be transformed into the required `LootTable$Builder` using `LootTable#lootTable`. The loot table can be built with a list of pools (via `#withPool`) applied in the order they are specified along with functions (via `#apply`) to modify the resulting items of those pools.

LootPool

Loot pools represents a group to perform operations and can generate a `LootPool$Builder` using `LootPool#lootPool`. Each loot pool can specify the entries (via `#add`) which define the operations in the pool, the conditions (via `#when`) which define if the operations in the pool should be performed, and functions (via `#apply`) to modify the resulting items of the entries. Each pool can be executed as many times as specified (via `#setRolls`). Additionally, bonus executions can be specified (via `#setBonusRolls`) which is modified by the luck of the executor.

LootPoolEntryContainer

Loot entries define the operations to occur when selected, typically generating items. Each entry has an associated, registered `LootPoolEntryType`. They also have their own associated builders which subtype `LootPoolEntryContainer$Builder`. Multiple entries can execute at the same time (via `#append`) or sequentially until one fails (via `#then`). Additionally, entries can default to another entry on failure (via `#otherwise`).

LootItemCondition

Loot conditions define requirements which need to be met for some operation to execute. Each condition has an associated, registered `LootItemConditionType`. They also have their own associated builders which subtype `LootItemCondition$Builder`. By default, all loot conditions specified must return true for an operation to execute. Loot conditions can also be specified such that only one must return true instead (via `#or`). Additionally, the resulting output of a condition can be inverted (via `#invert`).

LootItemFunction

Loot functions modify the result of an execution before passing it to the output. Each function has an associated, registered `LootItemFunctionType`. They also have their own associated builders which subtype `LootItemFunction$Builder`.

NbtProvider NBT providers are a special type of functions defined by `CopyNbtFunction`. They define where to pull tag information from. Each provider has an associated, registered `LootNbtProviderType`.

NumberProvider

Number providers determine how many times a loot pool executes. Each provider has an associated, registered `LootNumberProviderType`.

ScoreboardNameProvider Scoreboard providers are a special type of number providers defined by `ScoreboardValue`. They define the name of the scoreboard to pull the number of rolls to execute from. Each provider has an associated, registered `LootScoreProviderType`.

Tag Generation

Tags can be generated for a mod by subclassing `TagsProvider` and implementing `#addTags`. After implementation, the provider must be added to the `DataGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        // Extends net.minecraftforge.common.data.BlockTagsProvider
        output -> new MyBlockTagsProvider(
            output,
            event.getLookupProvider(),
            MOD_ID,
            event.getExistingFileHelper()
        )
    );
}
```

TagsProvider

The tags provider has two methods used for generating tags: creating a tag with objects and other tags via `#tag`, or using tags from other object types to generate the tag data via `#getOrCreateRawBuilder`.

!!! note Typically, a provider will not call `#getOrCreateRawBuilder` directly unless a registry contains a representation of objects from a different registry (blocks have item representations to obtain the blocks in the inventory).

When `#tag` is called, a `TagAppender` is created which acts as a chainable consumer of elements to add to the tag:

| Method | Description |
|-----------------------------|--|
| <code>add</code> | Adds an object to a tag through its resource key. |
| <code>addOptional</code> | Adds an object to a tag through its name. If the object is not present, then the object will be skipped when loading. |
| <code>addTag</code> | Adds a tag to a tag through its tag key. All elements within the inner tag are now a part of the outer tag. |
| <code>addOptionalTag</code> | Adds a tag to a tag through its name. If the tag is not present, then the tag will be skipped when loading. |
| <code>replace</code> | When <code>true</code> , all previously loaded entries added to this tag from other datapacks will be discarded. If a datapack is loaded after this one, then it will still append the entries to the tag. |
| <code>remove</code> | Removes an object or tag from a tag through its name or key. |

```
// In some TagProvider#addTags
this.tag(EXAMPLE_TAG)
    .add(EXAMPLE_OBJECT) // Adds an object to the tag
    .addOptional(new ResourceLocation("othermod", "other_object")) // Adds an object from another mod to the tag

this.tag(EXAMPLE_TAG_2)
    .addTag(EXAMPLE_TAG) // Adds a tag to the tag
    .remove(EXAMPLE_OBJECT) // Removes an object from this tag
```

!!! important If the mod's tags softly depends on another mod's tags (the other mod may or may not be present at runtime), the other mods' tags should be referenced using the optional methods.

Existing Providers

Minecraft contains a few tag providers for certain registries that can be subclassed instead. Additionally, some providers contain additional helper methods to more easily create tags.

| Registry Object Type | Tag Provider |
|--------------------------|--------------------------------------|
| Block | BlockTagsProvider* |
| Item | ItemTagsProvider |
| EntityType | EntityTypeTagsProvider |
| Fluid | FluidTagsProvider |
| GameEvent | GameEventTagsProvider |
| Biome | BiomeTagsProvider |
| FlatLevelGeneratorPreset | FlatLevelGeneratorPresetTagsProvider |
| WorldPreset | WorldPresetTagsProvider |
| Structure | StructureTagsProvider |
| PoiType | PoiTypeTagsProvider |
| BannerPattern | BannerPatternTagsProvider |
| CatVariant | CatVariantTagsProvider |
| PaintingVariant | PaintingVariantTagsProvider |
| Instrument | InstrumentTagsProvider |
| DamageType | DamageTypeTagsProvider |

* BlockTagsProvider is a Forge added TagsProvider.

ItemTagsProvider#copy Blocks have item representations to obtain them in the inventory. As such, many of the block tags can also be an item tag. To easily generate item tags to have the same entries as block tags, the **#copy** method can be used which takes in the block tag to copy from and the item tag to copy to.

```
//In ItemTagsProvider#addTags
this.copy(EXAMPLE_BLOCK_TAG, EXAMPLE_ITEM_TAG);
```

Custom Tag Providers

A custom tag provider can be created via a **TagsProvider** subclass which takes in the registry key to generate tags for.

```
public RecipeTypeTagsProvider(PackOutput output, CompletableFuture<HolderLookup.Provider> registries,
ExistingFileHelper fileHelper) {
    super(output, Registries.RECIPE_TYPE, registries, MOD_ID, fileHelper);
}
```

Intrinsic Holder Tags Providers

One special type of **TagProviders** are **IntrinsicHolderTagsProviders**. When creating a tag using this provider via **#tag**, the object itself can be used to add itself to the tag via **#add**. To do so, a function is provided within the constructor to turn an object into its **ResourceKey**.

```
// Subtype of `IntrinsicHolderTagsProvider`
public AttributeTagsProvider(PackOutput output, CompletableFuture<HolderLookup.Provider> registries,
ExistingFileHelper fileHelper) {
    super(
        output,
        ForgeRegistries.Keys.ATTRIBUTES,
        registries,
        attribute -> ForgeRegistries.ATTRIBUTES.getResourceKey(attribute).get(),
        MOD_ID,
        fileHelper
    );
}
```

Advancement Generation

Advancements can be generated for a mod by constructing a new **AdvancementProvider** and providing **AdvancementSubProviders**. Advancements can either be created and supplied manually or, for convenience, created using **Advancement\$Builder**. The provider must be added to the **DataGenerator**.

!!! note Forge provides an extension for the `AdvancementProvider` called `ForgeAdvancementProvider` which integrates better for generating advancements. So, this documentation will use `ForgeAdvancementProvider` along with the sub provider interface `ForgeAdvancementProvider$AdvancementGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        output -> new ForgeAdvancementProvider(
            output,
            event.getLookupProvider(),
            event.getExistingFileHelper(),
            // Sub providers which generate the advancements
            List.of(subProvider1, subProvider2, /*...*/)
        )
    );
}
```

ForgeAdvancementProvider\$AdvancementGenerator

A `ForgeAdvancementProvider$AdvancementGenerator` is responsible for generating advancements, containing a method which takes in a registry lookup, the writer (`Consumer<Advancement>`), and the existing file helper..

```
// In some subclass of ForgeAdvancementProvider$AdvancementGenerator or as a lambda reference
@Override
public void generate(HolderLookup.Provider registries, Consumer<Advancement> writer, ExistingFileHelper existingFileHelper) {
    // Build advancements here
}
```

Advancement\$Builder

`Advancement$Builder` is a convenience implementation for creating `Advancements` to generate. It allows the definition of the parent advancement, the display information, the rewards when the advancement has been completed, and the requirements to unlock the advancement. Only the requirements need to be specified to create an `Advancement`.

Although not required, there are a number of methods that are important to know of:

| Method | Description |
|---------------------------|---|
| <code>parent</code> | Sets the advancement which this advancement is directly linked to. Can either specify the name of the advancement or the advancement itself if its generated by the modder. |
| <code>display</code> | Sets the information to display to the chat, toast, and advancement screen. |
| <code>rewards</code> | Sets the rewards obtained when this advancement is completed. |
| <code>addCriterion</code> | Adds a condition to the advancement. |
| <code>requirements</code> | Specifies if the conditions must all return true or at least one must return true. An additional overload can be used to mix-and-match those operations. |

Once an `Advancement$Builder` is ready to be built, the `#save` method should be called which takes in the writer, the registry name of the advancement, and the file helper used to check whether the supplied parent exists.

```
// In some ForgeAdvancementProvider$AdvancementGenerator#generate(registries, writer, existingFileHelper)
Advancement example = Advancement.Builder.advancement()
    .addCriterion("example_criterion", triggerInstance) // How the advancement is unlocked
    .save(writer, name, existingFileHelper); // Add data to builder
```

Global Loot Modifier Generation

Global Loot Modifiers (GLMs) can be generated for a mod by subclassing `GlobalLootModifierProvider` and implementing `#start`. Each GLM can be added generated by calling `#add` and specifying the name of the modifier and the modifier instance to be serialized. After implementation, the provider must be added to the `DataGenerator`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        output -> new MyGlobalLootModifierProvider(output, MOD_ID)
    );
}

// In some GlobalLootModifierProvider#start
this.add("example_modifier", new ExampleModifier(
    new LootItemCondition[] {
        WeatherCheck.weather().setRaining(true).build() // Executes when raining
    },
    "vill",
    10,
    Items.DIRT
));
```

Datapack Registry Object Generation

Datapack registry objects can be generated for a mod by constructing a new `DatapackBuiltinEntriesProvider` and providing a `RegistrySetBuilder` with the new objects to register. The provider must be added to the `DataGenerator`.

!!! note `DatapackBuiltinEntriesProvider` is a Forge extension on top of `RegistriesDatapackGenerator` which properly handles referencing existing datapack registry objects without exploding the entry. So, this documentation will use `DatapackBuiltinEntriesProvider`.

```
// On the MOD event bus
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        // Tell generator to run only when server data are generating
        event.includeServer(),
        output -> new DatapackBuiltinEntriesProvider(
            output,
            event.getLookupProvider(),
            // The builder containing the datapack registry objects to generate
            new RegistrySetBuilder().add(/* ... */),
            // Set of mod ids to generate the datapack registry objects of
            Set.of(MOD_ID)
        )
    );
}
```

RegistrySetBuilder

A `RegistrySetBuilder` is responsible for building all datapack registry objects to be used within the game. The builder can add a new entry for a registry, which can then register objects to that registry.

First, a new instance of a `RegistrySetBuilder` can be initialized by calling the constructor. Then, the `#add` method (which takes in the `ResourceKey` of the registry, a `RegistryBootstrap` consumer containing the `BootstrapContext` to register the objects, and an optional `Lifecycle` argument to indicate the registry's current lifecycle status) can be called to handle a specific registry for registration.

```
new RegistrySetBuilder()
    // Create configured features
    .add(Registries.CONFIGURED_FEATURE, bootstrap -> {
```

```

    // Register configured features here
})
// Create placed features
.add(Registries.PLACED_FEATURE, bootstrap -> {
    // Register placed features here
});

```

!!! note Datapack registries created through Forge can also generate their objects using this builder by also passing in the associated `ResourceKey`.

Registering with BootstrapContext

The `#register` method in the `BootstrapContext` provided by the builder can be used to register objects. It takes in the `ResourceKey` representing the registry name of the object, the object to register, and an optional `Lifecycle` argument to indicate the registry object's current lifecycle status.

```

public static final ResourceKey<ConfiguredFeature<?, ?>> EXAMPLE_CONFIGURED_FEATURE = ResourceKey.create(
    Registries.CONFIGURED_FEATURE,
    new ResourceLocation(MOD_ID, "example_configured_feature")
);

// In some constant location or argument
new RegistrySetBuilder()
    // Create configured features
    .add(Registries.CONFIGURED_FEATURE, bootstrap -> {
        // Register configured features here
        bootstrap.register(
            // The resource key for the configured feature
            EXAMPLE_CONFIGURED_FEATURE,
            new ConfiguredFeature<>(
                Feature.ORE, // Create an ore feature
                new OreConfiguration(
                    List.of(), // Does nothing
                    8 // in veins of at most 8
                )
            )
        );
    });
// Create placed features
.add(Registries.PLACED_FEATURE, bootstrap -> {
    // Register placed features here
});

```

Datapack Registry Object Lookup

Sometimes datapack registry objects may want to use other datapack registry objects or tags containing datapack registry objects. In those cases, you can look up another datapack registry using `BootstrapContext#lookup` to get a `HolderGetter`. From there, you can get a `Holder$Reference` to the datapack registry object or a `HolderSet$Named` for the tag via `#getOrThrow` by passing in the associated key.

```

public static final ResourceKey<ConfiguredFeature<?, ?>> EXAMPLE_CONFIGURED_FEATURE = ResourceKey.create(
    Registries.CONFIGURED_FEATURE,
    new ResourceLocation(MOD_ID, "example_configured_feature")
);

public static final ResourceKey<PlacedFeature> EXAMPLE_PLACED_FEATURE = ResourceKey.create(
    Registries.PLACED_FEATURE,
    new ResourceLocation(MOD_ID, "example_placed_feature")
);

// In some constant location or argument
new RegistrySetBuilder()
    // Create configured features
    .add(Registries.CONFIGURED_FEATURE, bootstrap -> {
        // Register configured features here
        bootstrap.register(
            // The resource key for the configured feature
            EXAMPLE_CONFIGURED_FEATURE,

```

```

        new ConfiguredFeature(/* ... */)
    );
}
// Create placed features
.add(Registries.PLACED_FEATURE, bootstrap -> {
    // Register placed features here

    // Get configured feature registry
    HolderGetter<ConfiguredFeature<?, ?>> configured = bootstrap.lookup(Registries.CONFIGURED_FEATURE);

    bootstrap.register(
        // The resource key for the placed feature
        EXAMPLE_PLACED_FEATURE,
        new PlacedFeature(
            configured.getOrThrow(EXAMPLE_CONFIGURED_FEATURE), // Get the configured feature
            List.of() // and do nothing to the placement location
        )
    );
});

```

Configuration

Configurations define settings and consumer preferences that can be applied to a mod instance. Forge uses a configuration system using TOML files and read with NightConfig.

Creating a Configuration

A configuration can be created using a subtype of `IConfigSpec`. Forge implements the type via `ForgeConfigSpec` and enables its construction through `ForgeConfigSpec$Builder`. The builder can separate the config values into sections via `Builder#push` to create a section and `Builder#pop` to leave a section. Afterwards, the configuration can be built using one of two methods:

| Method | Description |
|------------------------|--|
| <code>build</code> | Creates the <code>ForgeConfigSpec</code> . |
| <code>configure</code> | Creates a pair of the class holding the config values and the <code>ForgeConfigSpec</code> . |

!!! note `ForgeConfigSpec$Builder#configure` is typically used with a `static` block and a class that takes in `ForgeConfigSpec$Builder` as part of its constructor to attach and hold the values:

```

```java
// In some config class
ExampleConfig(ForgeConfigSpec.Builder builder) {
 // Define values here in final fields
}

// Somewhere the constructor is accessible
static {
 Pair<ExampleConfig, ForgeConfigSpec> pair = new ForgeConfigSpec.Builder()
 .configure(ExampleConfig::new);
 // Store pair values in some constant field
}
```

```

Each config value can be supplied with additional context to provide additional behavior. Contexts must be defined before the config value is fully built:

| Method | Description |
|---------------------------|---|
| <code>comment</code> | Provides a description of what the config value does. Can provide multiple strings for a multiline comment. |
| <code>translation</code> | Provides a translation key for the name of the config value. |
| <code>worldRestart</code> | The world must be restarted before the config value can be changed. |

ConfigValue

Config values can be built with the provided contexts (if defined) using any of the `#define` methods.

All config value methods take in at least two components:

- A path representing the name of the variable: a . separated string representing the sections the config value is in
- The default value when no valid configuration is present

The `ConfigValue` specific methods take in two additional components:

- A validator to make sure the deserialized object is valid
- A class representing the data type of the config value

```
// For some ForgeConfigSpec$Builder builder
ConfigValue<T> value = builder.comment("Comment")
    .define("config_value_name", defaultValue);
```

The values themselves can be obtained using `ConfigValue#get`. The values are additionally cached to prevent multiple readings from files.

Additional Config Value Types

- **Range Values**
 - Description: Value must be between the defined bounds
 - Class Type: `Comparable<T>`
 - Method Name: `#defineInRange`
 - Additional Components:
 - * The minimum and maximum the config value may be
 - * A class representing the data type of the config value

!!! note `DoubleValues`, `IntValues`, and `LongValues` are range values which specify the class as `Double`, `Integer`, and `Long` respectively.

- **Whitelisted Values**
 - Description: Value must be in supplied collection
 - Class Type: `T`
 - Method Name: `#defineInList`
 - Additional Components:
 - * A collection of the allowed values the configuration can be
- **List Values**
 - Description: Value is a list of entries
 - Class Type: `List<T>`
 - Method Name: `#defineList`, `#defineListAllowEmpty` if list can be empty
 - Additional Components:
 - * A validator to make sure a deserialized element from the list is valid
- **Enum Values**
 - Description: An enum value in the supplied collection
 - Class Type: `Enum<T>`
 - Method Name: `#defineEnum`
 - Additional Components:
 - * A getter to convert a string or integer into an enum
 - * A collection of the allowed values the configuration can be

- **Boolean Values**
 - Description: A boolean value
 - Class Type: `Boolean`
 - Method Name: `#define`

Registering a Configuration

Once a `ForgeConfigSpec` has been built, it must be registered to allow Forge to load, track, and sync the configuration settings as required. Configurations should be registered in the mod constructor via `ModLoadingContext#registerConfig`. A configuration can be registered with a given type representing the side the config belongs to, the `ForgeConfigSpec`, and optionally a specific file name for the configuration.

```
// In the mod constructor with a ForgeConfigSpec CONFIG
ModLoadingContext.get().registerConfig(Type.COMMON, CONFIG);
```

Here is a list of the available configuration types:

| Type | Loaded | Synced to Client | Client Location | Server Location | Default File Suffix |
|--------|------------------|------------------|--|---|----------------------------|
| CLIENT | Client Side Only | No | <code>.minecraft/config</code> | N/A | <code>-client</code> |
| COMMON | On Both Sides | No | <code>.minecraft/config</code> | <code><server_folder>/config/common</code> | <code>common</code> |
| SERVER | Server Side Only | Yes | <code>.minecraft/saves/<level_name>/server/config</code> | <code><server_folder>/config/server config</code> | <code>server config</code> |

!!! tip Forge documents the config types within their codebase.

Configuration Events

Operations that occur whenever a config is loaded or reloaded can be done using the `ModConfigEvent$Loading` and `ModConfigEvent$Reloading` events. The events must be registered to the mod event bus.

!!! warning These events are called for all configurations for the mod; the `ModConfig` object provided should be used to denote which configuration is being loaded or reloaded.

Key Mappings

A key mapping, or key binding, defines a particular action that should be tied to an input: mouse click, key press, etc. Each action defined by a key mapping can be checked whenever the client can take an input. Furthermore, each key mapping can be assigned to any input through the Controls option menu.

Registering a KeyMapping

A `KeyMapping` can be registered by listening to the `RegisterKeyMappingsEvent` on the **mod event bus** only on the physical client and calling `#register`.

```
// In some physical client only class

// Key mapping is lazily initialized so it doesn't exist until it is registered
public static final Lazy<KeyMapping> EXAMPLE_MAPPING = Lazy.of(() -> /*...*/);

// Event is on the mod event bus only on the physical client
@SubscribeEvent
public void registerBindings(RegisterKeyMappingsEvent event) {
    event.register(EXAMPLE_MAPPING.get());
}
```

Creating a KeyMapping

A `KeyMapping` can be created using its constructor. The `KeyMapping` takes in a translation key defining the name of the mapping, the default input of the mapping, and the translation key defining the category the mapping will be put within in the Controls option menu.

!!! tip A `KeyMapping` can be added to a custom category by providing a category translation key not provided by vanilla. Custom category translation keys should contain the mod id (e.g. `key.categories.examplemod.examplecategory`).

Default Inputs

Each key mapping has a default input associated with it. This is provided through `InputConstants$Key`. Each input consists of an `InputConstants$Type`, which defines what device is providing the input, and an integer, which defines the associated identifier of the input on the device.

Vanilla provides three types of inputs: `KEYSYM`, which defines a keyboard through the provided `GLFW` key tokens, `SCANCODE`, which defines a keyboard through the platform-specific scancode, and `MOUSE`, which defines a mouse.

!!! note It is highly recommended to use `KEYSYM` over `SCANCODE` for keyboards as `GLFW` key tokens are not tied to any particular system. You can read more on the `GLFW` docs.

The integer is dependent on the type provided. All input codes are defined in `GLFW`: `KEYSYM` tokens are prefixed with `GLFW_KEY_*` while `MOUSE` codes are prefixed with `GLFW_MOUSE_*`.

```
new KeyMapping(  
    "key.examplemod.example1", // Will be localized using this translation key  
    InputConstants.Type.KEYSYM, // Default mapping is on the keyboard  
    GLFW.GLFW_KEY_P, // Default key is P  
    "key.categories.misc" // Mapping will be in the misc category  
)
```

!!! note If the key mapping should not be mapped to a default, the input should be set to `InputConstants#UNKNOWN`. The vanilla constructor will require you to extract the input code via `InputConstants$Key#getValue` while the Forge constructor can be supplied the raw input field.

IKeyConflictContext

Not all mappings are used in every context. Some mappings are only used in a GUI, while others are only used purely in game. To avoid mappings of the same key used in different contexts conflicting with each other, an `IKeyConflictContext` can be assigned.

Each conflict context contains two methods: `#isActive`, which defines if the mapping can be used in the current game state, and `#conflicts`, which defines whether the mapping conflicts with a key in the same or different conflict context.

Currently, Forge defines three basic contexts through `KeyConflictContext`: `UNIVERSAL`, which is the default meaning the key can be used in every context, `GUI`, which means the mapping can only be used when a `Screen` is open, and `IN_GAME`, which means the mapping can only be used if a `Screen` is not open. New conflict contexts can be created by implementing `IKeyConflictContext`.

```
new KeyMapping(  
    "key.examplemod.example2",  
    KeyConflictContext.GUI, // Mapping can only be used when a screen is open  
    InputConstants.Type.MOUSE, // Default mapping is on the mouse  
    GLFW.GLFW_MOUSE_BUTTON_LEFT, // Default mouse input is the left mouse button  
    "key.categories.examplemod.examplecategory" // Mapping will be in the new example category  
)
```

KeyModifier

Modders may not want mappings to have the same behavior if a modifier key is held at the same (e.g. `G` vs `CTRL + G`). To remedy this, Forge adds an additional parameter to the constructor to take in a `KeyModifier` which can apply control (`KeyModifier#CONTROL`), shift (`KeyModifier#SHIFT`), or alt (`KeyModifier#ALT`) to any input. `KeyModifier#NONE` is the default and will apply no modifier.

A modifier can be added in the controls option menu by holding down the modifier key and the associated input.

```
new KeyMapping(  
    "key.examplemod.example3",  
    KeyConflictContext.UNIVERSAL,
```

```

KeyModifier.SHIFT, // Default mapping requires shift to be held down
InputConstants.Type.KEYSYM, // Default mapping is on the keyboard
GLFW.GLFW_KEY_G, // Default key is G
"key.categories.misc"
)

```

Checking a KeyMapping

A `KeyMapping` can be checked to see whether it has been clicked. Depending on when, the mapping can be used in a conditional to apply the associated logic.

Within the Game

Within the game, a mapping should be checked by listening to `ClientTickEvent` on the **Forge event bus** and checking `KeyMapping#consumeClick` within a while loop. `#consumeClick` will return `true` only the number of times the input was performed and not already previously handled, so it won't infinitely stall the game.

```

// Event is on the Forge event bus only on the physical client
public void onClientTick(ClientTickEvent event) {
    if (event.phase == TickEvent.Phase.END) { // Only call code once as the tick event is called twice every tick
        while (EXAMPLE_MAPPING.get().consumeClick()) {
            // Execute logic to perform on click here
        }
    }
}

```

!!! warning Do not use the `InputEvents` as an alternative to `ClientTickEvent`. There are separate events for keyboard and mouse inputs only, so they wouldn't handle any additional inputs.

Inside a GUI

Within a GUI, a mapping can be checked within one of the `GuiEventListener` methods using `IForgeKeyMapping#isActiveAndMatches`. The most common methods which can be checked are `#keyPressed` and `#mouseClicked`.

`#keyPressed` takes in the GLFW key token, the platform-specific scan code, and a bitfield of the held down modifiers. A key can be checked against a mapping by creating the input using `InputConstants#getKey`. The modifiers are already checked within the mapping methods itself.

```

// In some Screen subclass
@Override
public boolean keyPressed(int key, int scancode, int mods) {
    if (EXAMPLE_MAPPING.get().isActiveAndMatches(InputConstants.getKey(key, scancode))) {
        // Execute logic to perform on key press here
        return true;
    }
    return super.keyPressed(x, y, button);
}

```

!!! note If you do not own the screen which you are trying to check a **key** for, you can listen to the **Pre** or **Post** events of `ScreenEvent$KeyPressed` on the **Forge event bus** instead.

`#mouseClicked` takes in the mouse's x position, y position, and the button clicked. A mouse button can be checked against a mapping by creating the input using `InputConstants$Type#getOrCreate` with the `MOUSE` input.

```

// In some Screen subclass
@Override
public boolean mouseClicked(double x, double y, int button) {
    if (EXAMPLE_MAPPING.get().isActiveAndMatches(InputConstants.Type.MOUSE.getOrCreate(button))) {
        // Execute logic to perform on mouse click here
        return true;
    }
    return super.mouseClicked(x, y, button);
}

```

!!! note If you do not own the screen which you are trying to check a **mouse** for, you can listen to the **Pre** or **Post** events of `ScreenEvent$MouseButtonPressed` on the **Forge event bus** instead.

Game Tests

Game Tests are a way to run in-game unit tests. The system was designed to be scalable and in parallel to run large numbers of different tests efficiently. Testing object interactions and behaviors are simply a few of the many applications of this framework.

Creating a Game Test

A standard Game Test follows three basic steps:

1. A structure, or template, is loaded holding the scene on which the interaction or behavior is tested.
2. A method conducts the logic to perform on the scene.
3. The method logic executes. If a successful state is reached, then the test succeeds. Otherwise, the test fails and the result is stored within a lectern adjacent to the scene.

As such, to create a Game Test, there must be an existing template holding the initial start state of the scene and a method which provides the logic of execution.

The Test Method

A Game Test method is a `Consumer<GameTestHelper>` reference, meaning it takes in a `GameTestHelper` and returns nothing. For a Game Test method to be recognized, it must have a `@GameTest` annotation:

```
public class ExampleGameTests {
    @GameTest
    public static void exampleTest(GameTestHelper helper) {
        // Do stuff
    }
}
```

The `@GameTest` annotation also contains members which configure how the game test should run.

```
// In some class
@GameTest(
    setupTicks = 20L, // The test spends 20 ticks to set up for execution
    required = false // The failure is logged but does not affect the execution of the batch
)
public static void exampleConfiguredTest(GameTestHelper helper) {
    // Do stuff
}
```

Relative Positioning All `GameTestHelper` methods translate relative coordinates within the structure template scene to its absolute coordinates using the structure block's current location. To allow for easy conversion between relative and absolute positioning, `GameTestHelper#absolutePos` and `GameTestHelper#relativePos` can be used respectively.

The relative position of a structure template can be obtained in-game by loading the structure via the test command, placing the player at the wanted location, and finally running the `/test pos` command. This will grab the coordinates of the player relative to the closest structure within 200 blocks of the player. The command will export the relative position as a copyable text component in the chat to be used as a final local variable.

!!! tip The local variable generated by `/test pos` can specify its reference name by appending it to the end of the command:

```
```bash
/test pos <var> # Exports 'final BlockPos <var> = new BlockPos(...);'
```
```

Successful Completion A Game Test method is responsible for one thing: marking the test was successful on a valid completion. If no success state was achieved before the timeout is reached (as defined by `GameTest#timeoutTicks`), then the test automatically fails.

There are many abstracted methods within `GameTestHelper` which can be used to define a successful state; however, four are extremely important to be aware of.

| Method | Description |
|---------------------------------|--|
| <code>#succeed</code> | The test is marked as successful. |
| <code>#succeedIf</code> | The supplied <code>Runnable</code> is tested immediately and succeeds if no <code>GameTestAssertException</code> is thrown. If the test does not succeed on the immediate tick, then it is marked as a failure. |
| <code>#succeedWhen</code> | The supplied <code>Runnable</code> is tested every tick until timeout and succeeds if the check on one of the ticks does not throw a <code>GameTestAssertException</code> . |
| <code>#succeedOnTickWhen</code> | The supplied <code>Runnable</code> is tested on the specified tick and will succeed if no <code>GameTestAssertException</code> is thrown. If the <code>Runnable</code> succeeds on any other tick, then it is marked as a failure. |

!!! important Game Tests are executed every tick until the test is marked as a success. As such, methods which schedule success on a given tick must be careful to always fail on any previous tick.

Scheduling Actions Not all actions will occur when a test begins. Actions can be scheduled to occur at specific times or intervals:

| Method | Description |
|-----------------------------|---|
| <code>#runAtTickTime</code> | The action is ran on the specified tick. |
| <code>#runAfterDelay</code> | The action is ran x ticks after the current tick. |
| <code>#onEachTick</code> | The action is ran every tick. |

Assertions At any time during a Game Test, an assertion can be made to check if a given condition is true. There are numerous assertion methods within `GameTestHelper`; however, it simplifies to throwing a `GameTestAssertException` whenever the appropriate state is not met.

Generated Test Methods

If Game Test methods need to be generated dynamically, a test method generator can be created. These methods take in no parameters and return a collection of `TestFunctions`. For a test method generator to be recognized, it must have a `@GameTestGenerator` annotation:

```
public class ExampleGameTests {
    @GameTestGenerator
    public static Collection<TestFunction> exampleTests() {
        // Return a collection of TestFunctions
    }
}
```

TestFunction A `TestFunction` is the boxed information held by the `@GameTest` annotation and the method running the test.

!!! tip Any methods annotated using `@GameTest` are translated into a `TestFunction` using `GameTestRegistry#turnMethodIntoTestFunction`. That method can be used as a reference for creating `TestFunctions` without the use of the annotation.

Batching

Game Tests can be executed in batches instead of registration order. A test can be added to a batch by having the same supplied `GameTest#batch` string.

On its own, batching does not provide anything useful. However, batching can be used to perform setup and teardown states on the current level the tests are running in. This is done by annotating a method with either `@BeforeBatch` for setup or `@AfterBatch` for takedown. The `#batch` methods must match the string supplied to the game test.

Batch methods are `Consumer<ServerLevel>` references, meaning they take in a `ServerLevel` and return nothing:

```
public class ExampleGameTests {
    @BeforeBatch(batch = "firstBatch")
    public static void beforeTest(ServerLevel level) {
        // Perform setup
    }

    @GameTest(batch = "firstBatch")
    public static void exampleTest2(GameTestHelper helper) {
        // Do stuff
    }
}
```

Registering a Game Test

A Game Test must be registered to be ran in-game. There are two methods of doing so: via the `@GameTestHolder` annotation or `RegisterGameTestsEvent`. Both registration methods still require the test methods to be annotated with either `@GameTest`, `@GameTestGenerator`, `@BeforeBatch`, or `@AfterBatch`.

GameTestHolder

The `@GameTestHolder` annotation registers any test methods within the type (class, interface, enum, or record). `@GameTestHolder` contains a single method which has multiple uses. In this instance, the supplied `#value` must be the mod id of the mod; otherwise, the test will not run under default configurations.

```
@GameTestHolder(MODID)
public class ExampleGameTests {
    // ...
}
```

RegisterGameTestsEvent

`RegisterGameTestsEvent` can also register either classes or methods using `#register`. The event listener must be added to the mod event bus. Test methods registered this way must supply their mod id to `GameTest#templateNamespace` on every method annotated with `@GameTest`.

```
// In some class
public void registerTests(RegisterGameTestsEvent event) {
    event.register(ExampleGameTests.class);
}

// In ExampleGameTests
@GameTest(templateNamespace = MODID)
public static void exampleTest3(GameTestHelper helper) {
    // Perform setup
}
```

!!! note The value supplied to `GameTestHolder#value` and `GameTest#templateNamespace` can be different from the current mod id. The configuration within the buildscript would need to be changed.

Structure Templates

Game Tests are performed within scenes loaded by structures, or templates. All templates define the dimensions of the scene and the initial data (blocks and entities) that will be loaded. The template must be stored as an `.nbt` file within `data/<namespace>/structures`.

!!! tip A structure template can be created and saved using a structure block.

The location of the template is specified by a few factors:

- If the namespace of the template is specified.
- If the class should be prepended to the name of the template.
- If the name of the template is specified.

The namespace of the template is determined by `GameTest#templateNamespace`, then `GameTestHolder#value` if not specified, then `minecraft` if neither is specified.

The simple class name is not prepended to the name of the template if the `@PrefixGameTestTemplate` is applied to a class or method with the test annotations and set to `false`. Otherwise, the simple class name is made lowercase and prepended and followed by a dot before the template name.

The name of the template is determined by `GameTest#template`. If not specified, then the lowercase name of the method is used instead.

```
// Modid for all structures will be MODID
@GameTestHolder(MODID)
public class ExampleGameTests {

    // Class name is prepended, template name is not specified
    // Template Location at 'modid:examplegametests.exampletest'
    @GameTest
    public static void exampleTest(GameTestHelper helper) { /*...*/ }

    // Class name is not prepended, template name is not specified
    // Template Location at 'modid:examplegametests2'
    @PrefixGameTestTemplate(false)
    @GameTest
    public static void exampleTest2(GameTestHelper helper) { /*...*/ }

    // Class name is prepended, template name is specified
    // Template Location at 'modid:examplegametests.test_template'
    @GameTest(template = "test_template")
    public static void exampleTest3(GameTestHelper helper) { /*...*/ }

    // Class name is not prepended, template name is specified
    // Template Location at 'modid:test_template2'
    @PrefixGameTestTemplate(false)
    @GameTest(template = "test_template2")
    public static void exampleTest4(GameTestHelper helper) { /*...*/ }
}
```

Running Game Tests

Game Tests can be run using the `/test` command. The `test` command is highly configurable; however, only a few are of importance to running tests:

| Subcommand | Description |
|------------------------|---|
| <code>run</code> | Runs the specified test: <code>run <test_name></code> . |
| <code>runall</code> | Runs all available tests. |
| <code>runthis</code> | Runs the nearest test to the player within 15 blocks. |
| <code>runthese</code> | Runs tests within 200 blocks of the player. |
| <code>runfailed</code> | Runs all tests that failed in the previous run. |

!!! note Subcommands follow the test command: `/test <subcommand>`.

Buildscript Configurations

Game Tests provide additional configuration settings within a buildscript (the `build.gradle` file) to run and integrate into different settings.

Enabling Other Namespaces

If the buildscript was setup as recommended, then only Game Tests under the current mod id would be enabled. To enable other namespaces to load Game Tests from, a run configuration must set the property

`forge.enabledGameTestNamespaces` to a string specifying each namespace separated by a comma. If the property is empty or not set, then all namespaces will be loaded.

```
// Inside a run configuration
property 'forge.enabledGameTestNamespaces', 'modid1,modid2,modid3'
```

!!! warning There must be no spaces in-between namespaces; otherwise, the namespace will not be loaded correctly.

Game Test Server Run Configuration

The Game Test Server is a special configuration which runs a build server. The build server returns an exit code of the number of required, failed Game Tests. All failed tests, whether required or optional, are logged. This server can be run using `gradlew runGameTestServer`.

Enabling Game Tests in Other Run Configurations

By default, only the `client`, `server`, and `gameTestServer` run configurations have Game Tests enabled. If another run configuration should run Game Tests, then the `forge.enableGameTest` property must be set to `true`.

```
// Inside a run configuration
property 'forge.enableGameTest', 'true'
```

Forge Update Checker

Forge provides a very lightweight, opt-in, update-checking framework. If any mods have an available update, it will show a flashing icon on the ‘Mods’ button of the main menu and mod list along with the respective changelogs. It *does not* download updates automatically.

Getting Started

The first thing you want to do is specify the `updateJSONURL` parameter in your `mods.toml` file. The value of this parameter should be a valid URL pointing to an update JSON file. This file can be hosted on your own web server, GitHub, or wherever you want as long as it can be reliably reached by all users of your mod.

Update JSON format

The JSON itself has a relatively simple format as follows:

```
{
  "homepage": "<homepage/download page for your mod>",
  "<mcversion>": {
    "<modversion>": "<changelog for this version>",
    // List all versions of your mod for the given Minecraft version, along with their changelogs
    // ...
  },
  "promos": {
    "<mcversion>-latest": "<modversion>",
    // Declare the latest "bleeding-edge" version of your mod for the given Minecraft version
    "<mcversion>-recommended": "<modversion>",
    // Declare the latest "stable" version of your mod for the given Minecraft version
    // ...
  }
}
```

This is fairly self-explanatory, but some notes:

- The link under `homepage` is the link the user will be shown when the mod is outdated.
- Forge uses an internal algorithm to determine whether one version string of your mod is “newer” than another. Most versioning schemes should be compatible, but see the `ComparableVersion` class if you are concerned about whether your scheme is supported. Adherence to Maven versioning is highly recommended.
- The changelog string can be separated into lines using `\n`. Some prefer to include a abbreviated changelog, then link to an external site that provides a full listing of changes.

- Manually inputting data can be chore. You can configure your `build.gradle` to automatically update this file when building a release as Groovy has native JSON parsing support. Doing this is left as an exercise to the reader.
- Some examples can be found here for `nocubes`, `Corail Tombstone` and `Chisels & Bits 2`.

Retrieving Update Check Results

You can retrieve the results of the Forge Update Checker using `VersionChecker#getResult(IModInfo)`. You can obtain your `IModInfo` via `ModContainer#getModInfo`. You can get your `ModContainer` using `ModLoadingContext.get().getActiveContainer()` inside your constructor, `ModList.get().getModContainerById(<your modId>)`, or `ModList.get().getModContainerByObject(<your mod instance>)`. You can obtain any other mod's `ModContainer` using `ModList.get().getModContainerById(<modId>)`. The returned object has a method `#status` which indicates the status of the version check.

| | Status | Description |
|--|---------------|---|
| | FAILED | The version checker could not connect to the URL provided. |
| | UP_TO_DATE | The current version is equal to the recommended version. |
| | AHEAD | The current version is newer than the recommended version if there is not latest version. |
| | OUTDATED | There is a new recommended or latest version. |
| | BETA_OUTDATED | There is a new latest version. |
| | BETA | The current version is equal to or newer than the latest version. |
| | PENDING | The result requested has not finished yet, so you should try again in a little bit. |

The returned object will also have the target version and any changelog lines as specified in `update.json`.

Debug Profiler

Minecraft provides a Debug Profiler that provides system data, current game settings, JVM data, level data, and sided tick information to find time consuming code. Considering things like `TickEvents` and ticking `BlockEntities`, this can be very useful for modders and server owners that want to find a lag source.

Using the Debug Profiler

The Debug Profiler is very simple to use. It requires the debug keybind `F3 + L` to start the profiler. After 10 seconds, it will automatically stop; however, it can be stopped earlier by pressing the keybind again.

!!! note Naturally, you can only profile code paths that are actually being reached. `Entities` and `BlockEntities` that you want to profile must exist in the level to show up in the results.

After you have stopped the debugger, it will create a new zip within the `debug/profiling` sub-directory in your run directory. The file name will be formatted with the date and time as `yyyy-mm-dd_hh_mi_ss-WorldName-VersionNumber.zip`

Reading a Profiling result

Within each sided folder (`client` and `server`), you will find a `profiling.txt` file containing the result data. At the top, it first tells you how long in milliseconds it was running and how many ticks ran in that time.

Below that, you will find information similar to the snippet below:

```
[00] levels - 96.70%/96.70%
[01] |   Level Name - 99.76%/96.47%
[02] |   |   tick - 99.31%/95.81%
[03] |   |   |   entities - 47.72%/45.72%
```

```
[04] | | | | regular - 98.32%/44.95%
[04] | | | | blockEntities - 0.90%/0.41%
[05] | | | | unspecified - 64.26%/0.26%
[05] | | | | minecraft:furnace - 33.35%/0.14%
[05] | | | | minecraft:chest - 2.39%/0.01%
```

Here is a small explanation of what each part means

| [02] | tick | 99.31% | 95.81% |
|--------------------------|-------------------------|--|---|
| The Depth of the section | The Name of the Section | The percentage of time it took in relation to it's parent. For Layer 0, it is the percentage of the time a tick takes. For Layer 1, it is the percentage of the time its parent takes. | The second percentage tells you how much time it took from the entire tick. |

Profiling your own code

The Debug Profiler has basic support for `Entity` and `BlockEntity`. If you would like to profile something else, you may need to manually create your sections like so:

```
ProfilerFiller#push(yourSectionName : String);
//The code you want to profile
ProfilerFiller#pop();
```

You can obtain the `ProfilerFiller` instance from a `Level`, `MinecraftServer`, or `Minecraft` instance. Now you just need to search the results file for your section name.

Access Transformers

Access Transformers (ATs for short) allow for widening the visibility and modifying the `final` flags of classes, methods, and fields. They allow modders to access and modify otherwise inaccessible members in classes outside their control.

The specification document can be viewed on the [Minecraft Forge GitHub](#).

Adding ATs

Adding an Access Transformer to your mod project is as simple as adding a single line into your `build.gradle`:

```
// This block is where your mappings version is also specified
minecraft {
    accessTransformer = file('src/main/resources/META-INF/accesstransformer.cfg')
}
```

After adding or modifying the Access Transformer, the gradle project must be refreshed for the transformations to take effect.

During development, the AT file can be anywhere specified by the line above. However, when loading in a non-development environment, Forge will only search for the exact path of `META-INF/accesstransformer.cfg` in your JAR file.

Comments

All text after a `#` until the end of the line will be treated as a comment and will not be parsed.

Access Modifiers

Access modifiers specify to what new member visibility the given target will be transformed to. In decreasing order of visibility:

- **public** - visible to all classes inside and outside its package
- **protected** - visible only to classes inside the package and subclasses
- **default** - visible only to classes inside the package
- **private** - visible only to inside the class

A special modifier **+f** and **-f** can be appended to the aforementioned modifiers to either add or remove respectively the **final** modifier, which prevents subclassing, method overriding, or field modification when applied.

!!! warning Directives only modify the method they directly reference; any overriding methods will not be access-transformed. It is advised to ensure transformed methods do not have non-transformed overrides that restrict the visibility, which will result in the JVM throwing an error.

Examples of methods that can be safely transformed are ``private` methods`, ``final` methods` (or methods in ```

Targets and Directives

!!! important When using Access Transformers on Minecraft classes, the SRG name must be used for fields and methods.

Classes

To target classes:

`<access modifier> <fully qualified class name>`

Inner classes are denoted by combining the fully qualified name of the outer class and the name of the inner class with a `$` as separator.

Fields

To target fields:

`<access modifier> <fully qualified class name> <field name>`

Methods

Targeting methods require a special syntax to denote the method parameters and return type:

`<access modifier> <fully qualified class name> <method name>(<parameter types>)<return type>`

Specifying Types Also called “descriptors”: see the Java Virtual Machine Specification, SE 8, sections 4.3.2 and 4.3.3 for more technical details.

- **B** - **byte**, a signed byte
- **C** - **char**, a Unicode character code point in UTF-16
- **D** - **double**, a double-precision floating-point value
- **F** - **float**, a single-precision floating-point value
- **I** - **integer**, a 32-bit integer
- **J** - **long**, a 64-bit integer
- **S** - **short**, a signed short
- **Z** - **boolean**, a **true** or **false** value
- **[** - references one dimension of an array
 - Example: `[[S` refers to `short[] []`
- **L<class name>;** - references a reference type
 - Example: `Ljava/lang/String;` refers to `java.lang.String` reference type (*note the use of slashes instead of periods*)
- **(** - references a method descriptor, parameters should be supplied here or nothing if no parameters are present

- Example: `<method>(I)Z` refers to a method that requires an integer argument and returns a boolean
- `V` - indicates a method returns no value, can only be used at the end of a method descriptor
 - Example: `<method>()V` refers to a method that has no arguments and returns nothing

Examples

```
# Makes public the ByteArrayToKeyFunction interface in Crypt
public net.minecraft.util.Crypt$ByteArrayToKeyFunction

# Makes protected and removes the final modifier from 'random' in MinecraftServer
protected-f net.minecraft.server.MinecraftServer f_129758_ #random

# Makes public the 'makeExecutor' method in Util,
# accepting a String and returns an ExecutorService
public net.minecraft.Util m_137477_(Ljava/lang/String;)Ljava/util/concurrent/ExecutorService; #makeExecutor

# Makes public the 'leastMostToIntArray' method in UUIDUtil,
# accepting two longs and returning an int[]
public net.minecraft.core.UUIDUtil m_235872_(JJ)[I #leastMostToIntArray
```

Getting Started

If you have decided to contribute to Forge, you will have to take some special steps to get started with developing. A simple mod development environment will not suffice to work with Forge’s codebase directly. Instead, you can use the following guide to help you with your setup and get you started with improving Forge!

Forking and Cloning the Repository

Like most major open source projects you will find, Forge is hosted on GitHub. If you have contributed to another project before, you will know this process already and can skip right ahead to the next section.

For those who are beginners when it comes to collaboration via Git, here are two easy steps to get you started.

!!! note This guide assumes that you already have a GitHub account set up. If you do not, visit their registration page to create an account. Furthermore, this guide is not a tutorial for git’s usage. Please consult different sources first if you are struggling to get it working.

Forking

First of all, you have to “fork” the MinecraftForge repository by clicking the “Fork” button in the upper right hand corner. If you are in an organization, select the account you want your fork to be hosted on.

Forking the repository is necessary since not every GitHub user can have free access to every repository. Instead, you create a copy of the original repository to later contribute your changes via a so called Pull Request, which you will learn more about later.

Cloning

After forking the repository, it is time to get local access to actually make some changes. For this, you need to clone the repository onto your local machine.

Using your favorite git client, simply clone your fork into a directory of your choice. As general example, here is a command line snippet that should work on all correctly configured systems and clones the repository into a directory called “MinecraftForge” under the current directory (note that you have to replace `<User>` with your username):

```
git clone https://github.com/<User>/MinecraftForge
```

Checking out the Correct Branch

Forking and cloning the repository are the only mandatory steps to develop for Forge. However, to ease the process of creating Pull Requests for you, it is best to work with branches.

It is recommended to create and check out a branch for each PR you plan to submit. This way, you can always keep around the latest changes of Forge for new PRs while you still work on older patches.

After completing this step, you are ready to go and set up your development environment.

Setting Up the Environment

Depending on your favorite IDE, there is a different set of recommended steps you have to follow to successfully set up a development environment.

Eclipse

Due to the way Eclipse workspaces work, ForgeGradle can do most of the work involved to get you started with a Forge workspace.

1. Open a terminal/command prompt and navigate to the directory of your cloned fork.
2. Type `./gradlew setup` and hit enter. Wait until ForgeGradle is done.
3. Type `./gradlew genEclipseRuns` and hit enter. Once again, wait until ForgeGradle is done.
4. Open your Eclipse workspace and go to **File -> Import -> General -> Existing Gradle Project**.
5. Browse to the repo directory for the “Project root directory” option in the dialog that opens.
6. Complete the import by clicking the “Finish” button.

That is all it takes to get you up and running with Eclipse. There is no extra steps required to get the test mods running. Simply hit “Run” like in any other project and select the appropriate run configuration.

IntelliJ IDEA

JetBrains’ flagship IDE comes with great integrated support for Gradle: Forge’s build system of choice. Due to some peculiarities of Minecraft mod development, however, there are additional steps required to get everything to work properly.

IDEA 2021 onwards

1. Start IntelliJ IDEA 2021.
 - If you already have another project open, close the project with the **File -> Close project** option.
2. In the projects tab of the “Welcome to IntelliJ IDEA” window, click the “Open” button on the top right and select the `MinecraftForge` folder you cloned earlier.
3. Click “Trust Project” if prompted.
4. After IDEA is done importing the project and indexing its files, run the Gradle setup task. You can do this by:
 - Open the Gradle sidebar on the right hand side of your screen, then open the forge project tree, select **Tasks**, then other and double-click the `setup` task (may also appear as `MinecraftForge[Setup]`) found in **Forge -> Tasks -> other -> setup**.
5. Generate the run configurations:
 - Open the Gradle sidebar on the right hand side of your screen, then open the forge project tree, select **Tasks**, then other and double-click the `genIntelliJRuns` task (may also appear as `MinecraftForge[genIntelliJRuns]`) found in **Forge -> Tasks -> forgegradle runs -> genIntelliJRuns**.
 - If you get a licensing error during build before making any changes, running the `updateLicenses` task may help. This task is found in **Forge -> Tasks -> other** as well.

IDEA 2019-2020 There are a few minor differences between IDEA 2021 and these versions for setup.

1. Import Forge’s `build.gradle` as an IDEA project. For this, simply click **Import Project** from the **Welcome to IntelliJ IDEA** splash screen, then select the `build.gradle` file.
2. After IDEA is done importing the project and indexing the files, run the Gradle setup task. Either:

1. Open the Gradle sidebar on the right hand side of your screen, then open the **forge** project tree, select **Tasks**, then **other** and double-click the **setup** task (may also appear as **MinecraftForge[Setup]**). Or alternatively:
2. Tap the CTRL key twice, and type **gradle setup** in the Run command window that pops up.

You can then run Forge using the **forge_client** gradle task (Tasks -> **fg_runs** -> **forge_client**): right-click the task and select either **Run** or **Debug** as desired.

You should now be able to work with your mod using the changes you introduce to the Forge and Vanilla codebase.

Making Changes and Pull Requests

Once you have set up your development environment, it is time to make some changes to Forge’s codebase. There are, however, some pitfalls you have to avoid when editing the project’s code.

The most important thing to note is that if you wish to edit Minecraft source code, you must only do so in the “Forge” sub-project. Any changes in the “Clean” project will mess with ForgeGradle and generating the patches. This can have disastrous consequences and might render your environment completely useless. If you wish to have a flawless experience, make sure you only edit code in the “Forge” project!

Generating Patches

After you have made changes to the code base and tested them thoroughly, you may go ahead and generate patches. This is only necessary if you work on the Minecraft code base (i.e. in the “Forge” project), but this step is vital for your changes to work elsewhere. Forge works by injecting only changed things into Vanilla Minecraft and hence needs those changes available in an appropriate format. Thankfully, ForgeGradle is capable of generating the changeset for you to commit it.

To initiate the patch generation, simply run the **genPatches** Gradle task from your IDE or the command line. After its completion, you can commit all your changes (make sure you do not add any unnecessary files) and submit your Pull Request!

Pull Requests

The last step before your contribution is added to Forge is a Pull Request (PR in short). This is a formal request to incorporate your fork’s changes into the live code base. Creating a PR is easy. Simply go to this [GitHub](#) page and follow the proposed steps. It is now that a good setup with branches pays off, since you are able to select precisely the changes you want to submit.

!!! note Pull Requests are bound to rules; not every request will blindly be accepted. Follow this document to get further information and to ensure the best quality of your PR! If you want to maximize the chances of your PR getting accepted, follow these PR guidelines!

Pull Request Guidelines

Mods are built on top of Forge, but there are some things that Forge does not support, and that limits what mods can do.

When modders run into something like that, they can make a change to Forge to support it, and submit that change as a Pull Request on Github.

To make the best use of both your and the Forge team’s time, it is recommended to follow some rough guidelines when preparing a Pull Request. The following points are the most important aspects to keep in mind when it comes to writing a good Pull Request.

What Exactly is Forge?

At a high level, Forge is a mod compatibility layer on top of Minecraft.

Early mods edited Minecraft’s code directly (like coremods do now), but they ran into conflicts with each other when they edited the same things. They also ran into issues when one mod changed behavior in ways that the other mods could not anticipate (like coremods do now), causing mysterious issues and lots of headaches.

By using something like Forge, mods can centralize common changes and avoid conflicts. Forge also includes supporting structures for common mod features like Capabilities, Registries, and others that allow mods to work together better.

When writing a good Forge Pull Request, you also have to know what Forge is at a lower level. There are two main types of code in Forge: Minecraft patches, and Forge code.

Patches

Patches are applied as direct changes to Minecraft's source code, and aim to be as minimal as possible. Every time Minecraft code changes, all the Forge patches need to be looked over carefully and applied correctly to the new code.

This means that large patches that change lots of things are difficult to maintain, so Forge aims to avoid those and keep patches as small as possible.

In addition to making sure the code makes sense, reviews for patches will focus on minimizing the size.

There are many strategies to make small patches, and reviews will often point out better methods to do things. Forge patches often insert a single line that fires an event or a code hook, which affects the code after it if the event meets some condition.

This allows most of the code to exist outside of the patch, which keeps the patch small and simple.

For more detailed information about creating patches, see the GitHub wiki.

Forge Code

Aside from the patches, Forge code is just normal Java code. It can be event code, compatibility features, or anything else that is not directly editing Minecraft code. When Minecraft updates, Forge code has to update just like everything else. However, it is much easier because it is not directly entangled in the Minecraft code.

Because this code stands on its own, there is no size restriction like there is with the patches.

In addition to making sure the code makes sense, reviews will focus on making the code clean: with proper formatting and Java documentation.

Explain Yourself

All Pull Requests need to answer the question: why is this necessary?

Any code added to Forge needs to be maintained, and more code means more potential for bugs, so solid justification is needed for adding code.

A common Pull Request issue is offering no explanation, or giving cryptic examples for how the Pull Request might theoretically be used. This only delays the Pull Request process.

A clear explanation for the general case is good, but also give a concrete example of how your mod needs this Pull Request.

Sometimes there is a better way to do what you wanted, or a way to do it without a Pull Request at all. Code changes can not be accepted until those possibilities have been completely ruled out.

Show that it Works

The code you submit to Forge should work perfectly, and it is up to you to convince the reviewers that it does.

One of the best ways to do that is to add an example mod or JUnit test to Forge that makes use of your new code and shows it working.

To set up and run a Forge Environment with the example mods, see this guide.

Breaking Changes in Forge

Forge cannot make changes that break the mods that depend on it.

This means that Pull Requests have to ensure that they do not break binary compatibility with previous Forge

versions.

A change that breaks binary compatibility is called a Breaking Change.

There are some exceptions to this:

- Forge accepts Breaking Changes at the beginning of new Minecraft versions, where Minecraft itself already causes Breaking Changes for modders.
- Sometimes an emergency breaking change is required outside of that time window, but it is rare and can cause dependency headaches for everyone in the modded Minecraft community.

Outside of those exceptional times, Pull Requests with breaking changes are not accepted. They must be adapted to support the old behavior or wait for the next Minecraft version.

Be Patient, Civil, and Empathetic

When submitting Pull Requests, you will often have to survive code review and make several changes before it is the best Pull Request possible.

Keep in mind that code review is not judgement against you. Bugs in your code are not personal. Nobody is perfect, and that is why we are working together.

Negativity will not help. Threatening to give up on your Pull Request and write a coremod instead will just make people upset and make the modded ecosystem worse.

It is important that while working together you assume the best intentions of the people who are reviewing your Pull Request and not take things personally.

Review

If you do your best to understand the slow and perfectionistic nature of the Pull Request process, we will do our best to understand your point of view as well.

After your Pull Request has been reviewed and cleaned up to the best of everyone's ability, it will be marked for a final review by Lex, who has the final say on what is included in the project or not.

Documentation for Legacy Versions

Forge has existed for years, and you can still easily access builds of Forge for Minecraft versions as old as Minecraft 1.1. There are significant differences between each and every version, and it would be an impossible task to support so many different versions. Therefore, Forge uses an LTS system where a previous major Minecraft version is deemed as "LTS" (Long Term Support). Only the latest version and any current LTS versions will have easily accessible documentation and be included in the version dropdown in the sidebar. However, some older versions were LTS once or the latest version at some point and had documentation written. Links to old sites with documentation for those versions can be found here.

!!! important These old documentation sites are for reference purposes only. Do not ask for help with old versions on the Forge discord or the Forge forums. **You will not receive support when you are using older versions.**

List of Previously Documented versions

Unfortunately, not all versions were used for a significant amount of time, and the documentation for that version may be incomplete. Whenever a new version is released, the documentation from the previous version is copied and adjusted over time to include new and updated information. When a version wasn't supported for long, the information was never updated. The accuracy percentages represent how much of the information that should have been updated was actually updated.

| Version | Accuracy | Link |
|---------|----------|---|
| 1.12.x | 100% | https://docs.minecraftforge.net/en/1.12.x/ |
| 1.13.x | 10% | https://docs.minecraftforge.net/en/1.13.x/ |
| 1.14.x | 10% | https://docs.minecraftforge.net/en/1.14.x/ |
| 1.15.x | 85% | https://docs.minecraftforge.net/en/1.15.x/ |

| Version | Accuracy | Link |
|---------|----------|---|
| 1.16.x | 85% | https://docs.minecraftforge.net/en/1.16.x/ |
| 1.17.x | 85% | https://docs.minecraftforge.net/en/1.17.x/ |
| 1.18.x | 90% | https://docs.minecraftforge.net/en/1.18.x/ |
| 1.19.2 | 90% | https://docs.minecraftforge.net/en/1.19.2/ |
| 1.19.x | 90% | https://docs.minecraftforge.net/en/1.19.x/ |

RetroGradle

RetroGradle is an archival initiative to update the older ForgeGradle 1.x to 2.3 toolchains and their Minecraft versions to use the modern ForgeGradle 4.x and above toolchain. The goal is to preserve all past released versions of Minecraft Forge by moving them to a verifiably working and modern toolchain which is data-driven and not hardcoded for version-specific workflows.

If any developer wishes to contribute to this archival effort, please visit The Forge Project discord server and ask for directions to the designated channel. Please note that this initiative only aims to *preserve* these old versions for the benefit of the community, ***not to support developing mods for these old, unsupported versions***. There will not be any support for using or developing for unsupported versions.

Porting to Minecraft 1.20

Here you can find a list of primers on how to port from old versions to the current version. Some versions are lumped together since that particular version never saw much usage.

| From -> To | Primer |
|-------------------|---------------------------|
| 1.12 -> 1.13/1.14 | Primer by williewillus |
| 1.14 -> 1.15 | Primer by williewillus |
| 1.15 -> 1.16 | Primer by 50ap5ud5 |
| 1.16 -> 1.17 | Primer by 50ap5ud5 |
| 1.19.2 -> 1.19.3 | Primer by ChampionAsh5357 |
| 1.19.3 -> 1.19.4 | Primer by ChampionAsh5357 |
| 1.19.4 -> 1.20 | Primer by ChampionAsh5357 |